

Diet-Agent [1]

饮食推荐 Agent

[项目完整代码 & 启动流程](#)

[简历内容](#)

[项目/业务背景](#)

[如何学习/使用本项目](#)

[项目介绍](#)

[全链路可观测能力设计](#)

[逐字稿- 全链路可观测能力设计](#)

[Trace 的实现 && 一次 Trace 的解读](#)

[评估体系设计与评估闭环](#)

[逐字稿- 评估体系设计与评估闭环](#)

[评分规则](#)

[可观测性 VS 评价体系](#)

[LLM 全链路 Fallback 设计](#)

[逐字稿- LLM 全链路 Fallback 设计](#)

[后端多 Agent 协作架构](#)

[逐字稿 - 后端多 Agent 协作架构](#)

[AgentScope 框架](#)

[意图识别与规则兜底](#)

[逐字稿 - 意图识别与规则兜底](#)

[意图识别与槽位填充](#)

[完整产品设计](#)

[AI 扫描项目输出文档帮助熟悉快速上手](#)

[00-AgentScope与SpringBoot集成](#)

[01-Agent多智能体设计](#)

[02-意图识别与规则兜底设计](#)

[03-槽位抽取设计](#)

[04-意图路由与链路划分](#)

- 05-多轮对话状态管理
- 06-双数据源-PERSONAL与PUBLIC
- 07-混合推荐流水线
- 08-MySQL-JSON_OVERLAPS标签检索
- 09-LLM全链路Fallback设计
- 10-Trace与可监控设计
- 11-评估设计（可评估性、指标与方式）
- 12-可评估闭环 (Trace→标注→评估→迭代)
- 13-RiskGuard健康合规守卫
- 14-短期记忆设计
- 15-页面设计
- 16-数据库表设计

认知篇

[Agent 项目怎么做才有含金量](#)

[后端面 Agent 开发](#)

AI & 大模型基础知识

理解与应用

[大模型的幻觉](#)

[如何理解大模型](#)

[\[你和公司\] 平时如何使用 AI](#)

[如何写Prompt提示词](#)

[提示词是如何生成和优化的](#)

Agent 篇

[Agent 概念](#)

[Agent 范式](#)

[Workflow 与 Agent](#)

[Single-Agent 与 Multi-Agent](#)

工具篇

[Function Calling](#)

[MCP](#)

[Skill](#)

RAG

[面试&校招/实习&简历经验](#)

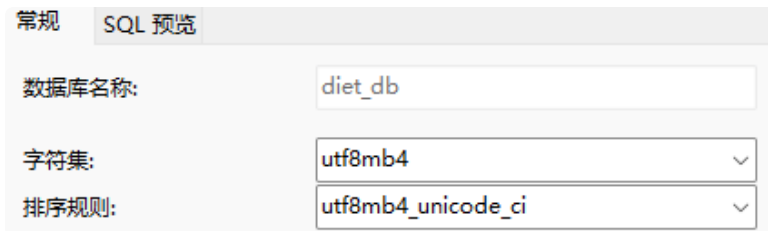
[PDF 下载](#)

项目完整代码 & 启动流程

项目完整代码: [📎diet-agent.zip](#)

下载项目代码后要执行的步骤:

1. 先创建MySQL数据库, 创建数据库的选项如下



A screenshot of the MySQL Database Creation Configuration window. The 'SQL 预览' (SQL Preview) tab is selected. The '数据库名称:' (Database Name) field contains 'diet_db'. The '字符集:' (Character Set) dropdown is set to 'utf8mb4'. The '排序规则:' (Collation) dropdown is set to 'utf8mb4_unicode_ci'.

2. 找到 `src/main/resources/db/diet_db.sql` SQL 脚本, 然后执行 SQL 脚本
3. [📎01_阿里云百炼.pdf](#) 参照指引, 去获取自己的大模型的api-key, 这是调用大模型的凭证
4. 将api-key填写到 `src/main/resources/application.yml` 配置文件的 `agentscope.dashscope.api-key` 之下
5. 启动项目, 项目可以直接本地启动, 前后端的代码都在一起

如果需要熟悉代码, 大家可以Debug模式启动后, 打开前端页面, 在IDEA中打断点去Debug调试, 这样可以熟悉起来更快。前端页面访问 <http://localhost:8080> 即可

而后可以直接问AI: 今天中午想吃点清淡减脂高蛋白的, 帮我推荐一下

或者在与AI对话之前, 可以先去创建个人餐食, 添加数据。初始的SQL脚本中没有很多餐食数据

前端页面的布局或样式, 大家可以自行让AI去修改调整

简历内容

Diet-Agent —— 时长四个月

项目简介：面向 "不知道吃什么" 的日常就餐决策难题场景，打造一款结合用户实时心情、场景、健康诉求等多维动态信息的智能饮食推荐 Agent。通过编排多 Agent 协作，实现意图识别 → 槽位澄清 → 标签检索重排 → LLM 推荐理由生成的完整对话式推荐链路，并设计 Trace 全链路监控与离线评估体系

核心设计：

- **全链路可观测能力设计：**针对 LLM 黑盒的不确定性，在业务关键节点采集输入与输出等核心指标的 Trace 事件，落库可回溯，支撑线上排查逐步回放，为每一次用户对话提供全链路可观测能力支撑；
- **评估体系设计与评估闭环：**通过批量解析 Trace 的离线评估的方式，使用 后端规则、LLM Judge、用户反馈 三者加权评估输出百分制评分报告，覆盖意图识别准确率、Token消耗、端到端耗时等 10+ 指标，低分样本回流反哺优化系统设计，形成 Trace 标注—批量评估—指标驱动迭代优化的完整闭环；
- **LLM 全链路 Fallback 设计：**针对全链路的 LLM 出现超时或输出异常等情况时，由后端规则与模板降级接管，保障全链路任一 LLM 异常时业务流程正常不中断，对话始终有回复；
- **完整产品设计：**用户侧支持双数据源对话推荐与个人菜单维护，研发侧支持 Trace 排查、标注与批量评估；覆盖对话推荐、个人餐食库管理、Trace 标注与批量评估四套能力，打通使用—反馈—标注—评估—迭代的工程闭环；
- **后端多 Agent 协作架构：**采用编排中枢+专职Worker分层模式，将意图理解、槽位澄清、推荐生成、质量评估解耦为独立推理Agent单元；LLM 仅负责结构化推理输出，会话流转与状态跃迁由编排层统一管控；
- **意图识别与规则兜底：**意图识别 Agent 识别 6 类意图与 7 维槽位，叠加 意图状态矫正、低置信降级、关键词 Fallback 三层兜底，避免错误路由与低质量推荐；槽位抽取通过词典强约束，避免幻觉；

上面的排序顺序是把最重要的内容放前面，也可以改成下面的简历排列顺序，比较循序渐进，看个人选择了

核心设计：

- **完整产品设计：**用户侧支持双数据源对话推荐与个人菜单维护，研发侧支持 Trace 排查、标注与批

量评估；覆盖对话推荐、个人餐食库管理、Trace 标注与批量评估四套能力，打通使用—反馈—标注—评估—迭代的工程闭环；

- **后端多 Agent 协作架构**：采用编排中枢+专职Worker分层模式，将意图理解、槽位澄清、推荐生成、质量评估解耦为独立推理Agent单元；LLM 仅负责结构化推理输出，会话流转与状态跃迁由编排层统一管控；
- **全链路可观测能力设计**：针对 LLM 黑盒的不确定性，在业务关键节点采集输入与输出等核心指标的 Trace 事件，落库可回溯，支撑线上排查逐步回放，为每一次用户对话提供全链路可观测能力支撑；
- **评估体系设计与评估闭环**：通过批量解析 Trace 的离线评估的方式，使用 后端规则、LLM Judge、用户反馈 三者加权评估输出百分制评分报告，覆盖意图识别准确率、Token消耗、端到端耗时等 10+指标，低分样本回流反哺优化系统设计，形成 Trace 标注—批量评估—指标驱动迭代优化的完整闭环；
- **LLM 全链路 Fallback 设计**：针对全链路的 LLM 出现超时或输出异常等情况时，由后端规则与模板降级接管，保障全链路任一 LLM 异常时业务流程正常不中断，对话始终有回复；
- **意图识别与规则兜底**：意图识别 Agent 识别 6 类意图与 7 维槽位，叠加 意图状态矫正、低置信降级、关键词 Fallback 三层兜底，避免错误路由与低质量推荐；槽位抽取通过词典强约束，避免幻觉；

简历内容大家也可以自行修改，但核心思路不要跑偏

我个人觉得 **最有含金量的是**

- 评估体系与评估闭环设计 —— 体现工程化思维

其次的是

- 全链路可观测能力设计 —— 把 黑盒变成 可见的
- LLM 全链路 Fallback 设计 —— 异常兜底思维

因为 Agent 系统很重要的就是 **观测监控 与 可评估优化闭环 与 异常链路兜底**

另外还有一点值得说的是，因为整个项目就是一个产品，我们也可以有产品设计思维，这个也有前端页面

即使在面试时，也可以马上启动项目，直接访问给面试官演示。因此简历上写了一点 **完整产品设计**

项目/业务背景

在当代快节奏的生活模式下，**饮食选择已成为大众日常高频且普遍的决策难题**，贯穿学生校园生活、职场人士工作日常等各类生活场景。随着餐饮行业的多元化发展，校园食堂、职场饭堂、街边餐饮门店、线上外卖平台供给充足，中餐、西餐、轻食、川湘粤各大菜系、特色小吃等饮食品类琳琅满目，丰富的饮食选择本应满足大众就餐需求，却反而引发了严重的“选择焦虑”，成为人们日常生活中的普遍困扰。

当下大众的饮食需求呈现出**个性化、动态化、场景化**的多元特征，不同人群、不同状态下的饮食诉求差异显著。

从饮食理念来看

- 部分人群秉持健康养生理念，注重食材营养、膳食均衡、低脂低盐，追求科学健康的饮食搭配
- 部分人群崇尚随性自由的就餐方式，更偏爱口味佳、氛围感强的美食，追求饮食的愉悦感与治愈感
- 还有绝大多数普通人群每日陷入“不知道吃什么”的迷茫困境，面对海量餐饮选项难以快速做出适配自身的选择，耗费大量时间与精力

与此同时，大众的饮食选择并非一成不变，而是与**当日自身状态深度绑定**。

个人的心情情绪、工作学习强度、生活作息安排、运动计划、身体状态等动态因素，都会直接影响就餐需求：

- 高压忙碌的工作日需要便捷高效、快速补充能量的饮食方案
- 休闲放松的闲暇时段更适合风味丰富、体验感强的美食
- 心情低落疲惫时偏爱暖胃治愈的餐食
- 身体燥热、作息紊乱时则需要清淡易消化的膳食

打造一款智能化饮食推荐Agent具有极强的现实必要性与应用价值

聚焦用户每日动态变量，深度结合用户实时心情状态、工作学习节奏、生活场景安排、个人饮食偏好、菜系倾向、身体需求等多重维度信息，进行综合分析与智能研判，为用户生成精准、适配、多元化的个性化饮食推荐方案。

能够帮助用户快速解决每日饮食选择难题，节省决策时间，规避选择焦虑，又可以兼顾饮食的舒适性、愉悦感与科学性、健康性，适配不同人群、不同场景的全时段就餐需求，为大众日常饮食决策提供高

效、智能全新解决方案。

如何学习/使用本项目

如何学习/使用

1. 先按照前文所说的 下载项目源码，去跑起来这个项目，去体验感受一下，这样最直观
2. 浏览一遍 <AI 扫描项目输出文档帮助熟悉快速上手> 这下面的文档，第一遍看有个印象即可，是用 AI扫描项目输出的文档，里面还是比较细的，但是一开始看可能因为不熟悉可能有些看不懂，可以结合代码边看边熟悉，后面熟悉了再去看，就都能看懂了，其实 AI 总结的还是挺好的
3. Debug模式启动服务，而后页面上去操作点击发起接口请求，而后在IDEA中逐步调试代码进行学习
4. 初步有了项目理解之后，就可以把项目内容写到简历上
5. 而后学习项目简历上对应的逐字稿，这就是你在面试中要和面试官阐述的内容，逐字稿是最精炼简洁表达项目内容的话术，都是以 "背景阐述"- "实现方案"- "复盘总结"的结构去总结凝练的。

项目介绍

在面试官要求简单介绍一下项目时，可以如下这样说

面向 "不知道吃什么" 的日常就餐决策难题场景，打造一款结合用户实时心情、场景、健康诉求等多维动态信息的智能饮食推荐 Agent。通过编排多 Agent 协作，实现意图识别 → 槽位澄清 → 标签检索重排 → LLM 推荐理由生成的完整对话式推荐链路，并设计 Trace 全链路监控与离线评估体系

在用户侧支持双数据源餐食对话推荐与个人菜单维护，研发侧支持 Trace 排查、标注与批量评估；整个系统覆盖对话推荐、个人餐食库管理、Trace 标注与批量评估 四套能力，打通 使用—反馈—标注—评估—迭代的工程闭环；

而后面试官就会具体问简历上写到的具体的点，或者让你说项目中的任意一个你觉得有含金量的点
此时就按照逐字稿上的内容去介绍即可

逐字稿- 全链路可观测能力设计

对应简历内容：

- **全链路可观测能力设计**：针对 LLM 黑盒的不确定性，在业务关键节点采集输入与输出等核心指标的 Trace 事件，落库可回溯，支撑线上排查逐步回放，为每一次用户对话提供全链路可观测能力支撑；

1. 背景阐述

Agent 系统与传统的后端系统有所不同

因为引入了大模型，变成了一个不确定性较大的黑盒

即 Agent 系统的每一次处理的结果可能都会不一样

因此每一次 Agent 系统的接口请求，我们必须能够知道这一次请求每一步做了什么

调用了哪些 Agent、哪些模型、模型的输入和输出是什么，token 用量多少

在 Agent 构建的整个业务系统之下 每一步的流程、每个业务节点的输入和输出又是什么

所以要把这个运行中的类似黑盒的 Agent 系统的细节始终掌握

可观测性与监控能力的建设是第一步

做好了可观测性与监控能力之后，才有基础去谈后续的评估与优化迭代

2. 实现方案

针对当前这个饮食推荐系统，我在业务的关键节点和 Agent 调用时都会记录 Trace 到内存的 ThreadLocal 之中

例如 意图识别和槽位信息提取时记录下 意图识别的结果和提取到的槽位信息

是否需要澄清、检索餐食，餐食排序、最终应答等各个关键环节 都记录相对应的信息

最后在一次请求结束返回时，将其写入数据库

因此这一次请求的每一步流程，通过这个 Trace 就可以完全回溯

通过这个全链路监控，我能够知道系统请求 的耗时、大模型的调用次数与耗时、Token的成本、意图识别的结果、是否需要澄清的决策、检索和排序的结果、最终应答的结果。这些都是我监控的维度

3. 复盘总结

最后总结一下，可观测性的能力建设，将 "黑盒" 特性的Agent 系统的透明度提升了

且 Trace 数据是后续对系统进行持续优化迭代的数据基础，是支撑整个系统改进的基石

Trace 的实现 && 一次 Trace 的解读

1. Trace 的实现

在整个系统开始处理使用 openTrace 在当前线程建立 Trace 上下文

整轮处理过程中通过 recordEvent/recordError/callAgent 把事件追加到内存列表

try-with-resources 结束时 TraceScope.close() 将汇总信息 + 全部 events 序列化为 trace_json

一次性 INSERT 到 diet_request_trace 表，整轮结束才 INSERT 一次，减少 DB 压力

```

1 // 开启 Trace 上下文; try-with-resources 结束时 TraceScope#close 会将整轮事件写
  入 agent_traces 表
2 try (AgentTraceService.TraceScope ignored = agentTraceService.openTrace(tr
  aceId, initialState.sessionId(), userId)) {
3     try {
4         // 记录请求开始时间 (纳秒), 用于最后计算整轮耗时
5         long startedAt = System.nanoTime();
6         // Trace 事件: REQUEST_RECEIVED | 阶段 HTTP | 输入=ChatRequest | 输出
          =初始 SessionState
7         agentTraceService.recordEvent("REQUEST_RECEIVED", "HTTP", request,
          initialState);
8
9         // 获取或创建该 sessionId 对应的锁对象, 保证同一 session 并发请求串行执行
10        Object lock = sessionLocks.computeIfAbsent(initialState.sessionId(
          ), key -> new Object());
11        synchronized (lock) {
12            // 在锁内执行完整状态机, 处理本轮用户输入
13            ChatResponse response = handleTurn(userId, request, traceId, i
          nitialState);
14            // Trace 事件: REQUEST_FINISHED | 阶段 HTTP | 输入=ChatRequest
              | 输出=ChatResponse | 耗时 ms
15            agentTraceService.recordEvent("REQUEST_FINISHED", "HTTP", requ
          est, response, elapsedMs(startedAt));
16            // 将最终响应返回给 Controller
17            return response;
18        }
19    } catch (RuntimeException error) {
20        // Trace 事件: REQUEST_FAILED | 阶段 HTTP | 输入=ChatRequest | 记录异
          常并将 Trace 标记 FAILED
21        agentTraceService.recordError("REQUEST_FAILED", "HTTP", request, e
          rror);
22        // 继续向上抛出, 由全局异常处理器返回错误响应
23        throw error;
24    }
25 }

```

更多具体的实现细节可以去阅读对应的代码

2. 一次 Trace 的解读

这是一次用户请求所记录下的 Trace（Trace 即链路，能够跟踪整个系统的执行链路）

大家可以结合代码的整个链路去读，每一次 Trace 记录都对应着一行代码

请求刚进入系统时，应该记录下请求刚进入时的信息，这是一次 Trace

- phase 是阶段的意思，此时仅记为 HTTP
- eventType 是事件类型的意思，记为 REQUEST_RECEIVED，即收到请求
- inputPayload 是 输入信息，此时记录内容为 sessionId、message 用户信息、sourceMode 数据源模式
- outputPayload 是输出信息，此时记录内容为 SessionState 会话状态（是一个对象，里面有更多具体信息）

▼ 部分Trace

Java

```
1 {
2     "phase": "HTTP",
3     "agentName": null,
4     "createdAt": "2026-07-02T00:56:51.963542200",
5     "eventType": "REQUEST_RECEIVED",
6     "latencyMs": null,
7     "modelName": null,
8     "stepOrder": 1,
9     "inputTokens": null,
10    "totalTokens": null,
11    "errorMessage": null,
12    "inputPayload": "{\n\"sessionId\":\n\"sess_3e95795822cf494db53cadcf65b868b7\",
    \"message\":\n\"晚饭推荐清淡低脂的\",
    \"sourceMode\":\n\"PUBLIC\",
    \"context\":{}}",
13    "outputTokens": null,
14    "outputPayload": "SessionState(sessionId=sess_3e95795822cf494db53cadcf65b868b7,
    userId=1, phase=START, sourceMode=PUBLIC, currentIntent=null,
    slots=SlotBundle(mealTime=[], mood=[], scene=[], healthGoal=[], cuisine=[],
    taste=[], convenience=[]), lastRecommendations=[])"
15 }
```

意图识别的调用记录，这是一次 Trace

- phase 是 AGENT，即 Agent 调用
- agentName 记为 IntentAgent 意图识别 Agent
- eventType 是 AGENT_CALL，即 Agent 调用
- latencyMs 是 这一次 Agent 调用 耗时 900 ms

- modelName 是使用的模型，此时用的是 qwen-turbo
- inputTokens 是输入给LLM的 token 数
- outputTokens 是LLM输出的 token 数
- inputPayload 是当前输入给LLM的内容
- outputPayload 是当前LLM输出的内容

部分Trace

Java

```

1  {
2      "phase": "AGENT",
3      "agentName": "IntentAgent",
4      "createdAt": "2026-07-02T00:56:52.875537",
5      "eventType": "AGENT_CALL",
6      "latencyMs": 900,
7      "modelName": "qwen-turbo",
8      "stepOrder": 3,
9      "inputTokens": 1455,
10     "totalTokens": 1540,
11     "errorMessage": null,
12     "inputPayload": "userId: 1\nsessionId: sess_3e95795822cf494db53cad
cf65b868b7\nrecentHistory: [ConversationTurn(role=user, intent=null, summa
ry=晚饭推荐清淡低脂的, timestamp=1782925011000)]\nknownSlots: SlotBundle(meal
Time=[], mood=[], scene=[], healthGoal=[], cuisine=[], taste=[], convenien
ce=[])\nslotOptions: {mealTime=[早餐, 早午餐, 午餐, 下午茶, 晚餐, 夜宵, 加餐,
三餐], mood=[疲惫, 烦躁, 开心, 焦虑, 低落, 平静, 压力大, 没胃口, 想放松, 想奖励自
己], scene=[工作, 校园, 家里, 周末, 加班, 运动后, 通勤, 聚餐, 独处, 旅行, 夜宵], h
ealthGoal=[减脂, 清淡, 养胃, 高蛋白, 均衡, 降火, 低油, 低盐, 低糖, 补能, 增肌, 控
碳水, 易消化, 暖胃], cuisine=[川菜, 粤菜, 湘菜, 江浙菜, 东北菜, 鲁菜, 闽南菜, 云南
菜, 新疆菜, 轻食, 西餐, 日料, 韩餐, 东南亚菜, 火锅, 烧烤, 海鲜, 素食, 家常, 小吃, 粉
面, 粥汤, 快餐, 甜品], taste=[清淡, 辣, 微辣, 中辣, 麻辣, 甜, 酸甜, 咸鲜, 鲜香, 酱
香, 蒜香, 番茄味, 咖喱味, 奶香, 油香, 烟火气], convenience=[快速, 慢享, 外带方便,
堂食舒服, 少排队, 少餐具, 一人食, 多人共享, 适合备餐, 适合边走边吃]}\n当前这一句: 晚
饭推荐清淡低脂的\n请输出 JSON, 字段为 intent、slots、confidence.\nslots 必须从 s
lotOptions 对应字段的候选值中选择; 无法映射则输出 null 或空数组, 不要创造标签.\n",
13     "outputTokens": 85,
14     "outputPayload": "{\n  \"intent\": \"MEAL_RECOMMENDATION\", \n  \"s
lots\": {\n    \"mealTime\": \"晚餐\", \n    \"mood\": null, \n    \"scene
\": null, \n    \"healthGoal\": \"清淡,低脂\", \n    \"cuisine\": null, \n
    \"taste\": \"清淡\", \n    \"convenience\": null\n  }, \n  \"confidence\":
0.95\n}"
15  }

```

后面的内容就以此类推，通过整个Trace，就能知道整个请求在系统中是怎么走的

每一个节点的输入输出或决策都很清楚

```
1 {
2   "events": [{
3     "phase": "HTTP",
4     "agentName": null,
5     "createdAt": "2026-07-02T00:56:51.963542200",
6     "eventType": "REQUEST_RECEIVED",
7     "latencyMs": null,
8     "modelName": null,
9     "stepOrder": 1,
10    "inputTokens": null,
11    "totalTokens": null,
12    "errorMessage": null,
13    "inputPayload": "{\"sessionId\":\"sess_3e95795822cf494db53cadcf65b868b7\", \"message\":\"晚饭推荐清淡低脂的\", \"sourceMode\":\"PUBLIC\", \"context\":{\"}}\",
14    "outputTokens": null,
15    "outputPayload": "SessionState(sessionId=sess_3e95795822cf494db53cadcf65b868b7, userId=1, phase=START, sourceMode=PUBLIC, currentIntent=null, slots=SlotBundle(mealTime=[], mood=[], scene=[], healthGoal=[], cuisine=[], taste=[], convenience=[]), lastRecommendations=[])"
16  }, {
17    "phase": "SESSION",
18    "agentName": null,
19    "createdAt": "2026-07-02T00:56:51.970051",
20    "eventType": "USER_MESSAGE_RECORDED",
21    "latencyMs": null,
22    "modelName": null,
23    "stepOrder": 2,
24    "inputTokens": null,
25    "totalTokens": null,
26    "errorMessage": null,
27    "inputPayload": "晚饭推荐清淡低脂的",
28    "outputTokens": null,
29    "outputPayload": "{\"sourceMode\":\"PUBLIC\", \"sessionId\":\"sess_3e95795822cf494db53cadcf65b868b7\"}"
30  }, {
31    "phase": "AGENT",
32    "agentName": "IntentAgent",
33    "createdAt": "2026-07-02T00:56:52.875537",
34    "eventType": "AGENT_CALL",
35    "latencyMs": 900,
36    "modelName": "qwen-turbo",
37    "stepOrder": 3,
38    "inputTokens": 1455,
39    "totalTokens": 1540,
```

```

40         "errorMessage": null,
41         "inputPayload": "userId: 1\nsessionId: sess_3e95795822cf494db53ca
dcf65b868b7\nrecentHistory: [ConversationTurn(role=user, intent=null, sum
mary=晚饭推荐清淡低脂的, timestamp=1782925011000)]\nknownSlots: SlotBundle(m
ealTime=[], mood=[], scene=[], healthGoal=[], cuisine=[], taste=[], conve
nience=[])\nslotOptions: {mealTime=[早餐, 早午餐, 午餐, 下午茶, 晚餐, 夜宵, 加
餐, 三餐], mood=[疲惫, 烦躁, 开心, 焦虑, 低落, 平静, 压力大, 没胃口, 想放松, 想奖
励自己], scene=[工作, 校园, 家里, 周末, 加班, 运动后, 通勤, 聚餐, 独处, 旅行, 夜
宵], healthGoal=[减脂, 清淡, 养胃, 高蛋白, 均衡, 降火, 低油, 低盐, 低糖, 补能,
增肌, 控碳水, 易消化, 暖胃], cuisine=[川菜, 粤菜, 湘菜, 江浙菜, 东北菜, 鲁菜, 闽
南菜, 云南菜, 新疆菜, 轻食, 西餐, 日料, 韩餐, 东南亚菜, 火锅, 烧烤, 海鲜, 素食, 家
常, 小吃, 粉面, 粥汤, 快餐, 甜品], taste=[清淡, 辣, 微辣, 中辣, 麻辣, 甜, 酸甜,
咸鲜, 鲜香, 酱香, 蒜香, 番茄味, 咖喱味, 奶香, 油香, 烟火气], convenience=[快速,
慢享, 外带方便, 堂食舒服, 少排队, 少餐具, 一人食, 多人共享, 适合备餐, 适合边走边吃]
\n当前这一句: 晚饭推荐清淡低脂的\n请输出 JSON, 字段为 intent、slots、confidence。
\nslots 必须从 slotOptions 对应字段的候选值中选择; 无法映射则输出 null 或空数组,
不要创造标签。\\n",
42         "outputTokens": 85,
43         "outputPayload": "{\\n  \\\"intent\\\": \\\"MEAL_RECOMMENDATION\\\",\\n
\\\"slots\\\": {\\n    \\\"mealTime\\\": \\\"晚餐\\\",\\n    \\\"mood\\\": null,\\n    \\\"sce
ne\\\": null,\\n    \\\"healthGoal\\\": \\\"清淡,低脂\\\",\\n    \\\"cuisine\\\": null,\\n
    \\\"taste\\\": \\\"清淡\\\",\\n    \\\"convenience\\\": null\\n  },\\n  \\\"confidence
\\\": 0.95\\n}"
44     }, {
45         "phase": "INTENT",
46         "agentName": null,
47         "createdAt": "2026-07-02T00:56:52.876536200",
48         "eventType": "INTENT_RECOGNIZED",
49         "latencyMs": null,
50         "modelName": null,
51         "stepOrder": 4,
52         "inputTokens": null,
53         "totalTokens": null,
54         "errorMessage": null,
55         "inputPayload": "晚饭推荐清淡低脂的",
56         "outputTokens": null,
57         "outputPayload": "{\\\"intent\\\":\\\"MEAL_RECOMMENDATION\\\",\\\"slots\\\":
{\\\"mealTime\\\": [\\\"晚餐\\\"],\\\"mood\\\": [],\\\"scene\\\": [],\\\"healthGoal\\\": [\\\"清淡
\\\"],\\\"cuisine\\\": [],\\\"taste\\\": [\\\"清淡\\\"],\\\"convenience\\\": [],\\\"empty\\\": fals
e},\\\"confidence\\\":0.95}"
58     }, {
59         "phase": "INTENT",
60         "agentName": null,
61         "createdAt": "2026-07-02T00:56:52.876536200",
62         "eventType": "INTENT_REVISED",
63         "latencyMs": null,
64         "modelName": null,
65         "stepOrder": 5,

```

```

66         "inputTokens": null,
67         "totalTokens": null,
68         "errorMessage": null,
69         "inputPayload": "{\"intent\":\"MEAL_RECOMMENDATION\",\"slots\":
{\"mealTime\":[\"晚餐\"],\"mood\":[],\"scene\":[],\"healthGoal\":[\"清淡
\", \"cuisine\":[],\"taste\":[\"清淡\"],\"convenience\":[],\"empty\":fals
e},\"confidence\":0.95}\"",
70         "outputTokens": null,
71         "outputPayload": "{\"intent\":\"MEAL_RECOMMENDATION\",\"slots\":
{\"mealTime\":[\"晚餐\"],\"mood\":[],\"scene\":[],\"healthGoal\":[\"清淡
\", \"cuisine\":[],\"taste\":[\"清淡\"],\"convenience\":[],\"empty\":fals
e},\"confidence\":0.95}\""
72     }, {
73         "phase": "ROUTE",
74         "agentName": null,
75         "createdAt": "2026-07-02T00:56:52.876536200",
76         "eventType": "ROUTE_SELECTED",
77         "latencyMs": null,
78         "modelName": null,
79         "stepOrder": 6,
80         "inputTokens": null,
81         "totalTokens": null,
82         "errorMessage": null,
83         "inputPayload": "{\"intent\":\"MEAL_RECOMMENDATION\",\"slots\":
{\"mealTime\":[\"晚餐\"],\"mood\":[],\"scene\":[],\"healthGoal\":[\"清淡
\", \"cuisine\":[],\"taste\":[\"清淡\"],\"convenience\":[],\"empty\":fals
e},\"confidence\":0.95}\"",
84         "outputTokens": null,
85         "outputPayload": "{\"route\":\"MEAL_RECOMMENDATION\"}"
86     }, {
87         "phase": "SLOT",
88         "agentName": null,
89         "createdAt": "2026-07-02T00:56:52.876536200",
90         "eventType": "SLOTS_MERGED",
91         "latencyMs": null,
92         "modelName": null,
93         "stepOrder": 7,
94         "inputTokens": null,
95         "totalTokens": null,
96         "errorMessage": null,
97         "inputPayload": "{\"stateSlots\":{\"mealTime\":[],\"mood\":[],\"s
cene\":[],\"healthGoal\":[],\"cuisine\":[],\"taste\":[],\"convenience\":[
],\"empty\":true},\"intentSlots\":{\"mealTime\":[\"晚餐\"],\"mood\":[
],\"scene\":[],\"healthGoal\":[\"清淡\"],\"cuisine\":[],\"taste\":[\"清淡
\", \"convenience\":[],\"empty\":false}}",
98         "outputTokens": null,
99         "outputPayload": "{\"mealTime\":[\"晚餐\"],\"mood\":[],\"scene\":[
],\"healthGoal\":[\"清淡\"],\"cuisine\":[],\"taste\":[\"清淡\"],\"conveni

```

```

100     ence\":[[],\\"empty\\":false}"
101     }, {
102         "phase": "CLARIFY",
103         "agentName": null,
104         "createdAt": "2026-07-02T00:56:52.876536200",
105         "eventType": "CLARIFY_DECISION",
106         "latencyMs": null,
107         "modelName": null,
108         "stepOrder": 8,
109         "inputTokens": null,
110         "totalTokens": null,
111         "errorMessage": null,
112         "inputPayload": "{\\"mealTime\\":[\\"晚餐\\"],\\"mood\\":[],\\"scene\\":
113         [],\\"healthGoal\\":[\\"清淡\\"],\\"cuisine\\":[],\\"taste\\":[\\"清淡\\"],\\"conveni
114         ence\\":[],\\"empty\\":false}",
115         "outputTokens": null,
116         "outputPayload": "ClarifyResult(action=READY, questionToAsk=null, missingSlots=[])"
117     }, {
118         "phase": "SEARCH",
119         "agentName": null,
120         "createdAt": "2026-07-02T00:56:52.881150200",
121         "eventType": "MEAL_SEARCHED",
122         "latencyMs": null,
123         "modelName": null,
124         "stepOrder": 9,
125         "inputTokens": null,
126         "totalTokens": null,
127         "errorMessage": null,
128         "inputPayload": "{\\"mealTime\\":[\\"晚餐\\"],\\"mood\\":[],\\"scene\\":
129         [],\\"healthGoal\\":[\\"清淡\\"],\\"cuisine\\":[],\\"taste\\":[\\"清淡\\"],\\"conveni
130         ence\\":[],\\"empty\\":false}",
131         "outputTokens": null,
132         "outputPayload": "{candidateCount=2, candidates=[MealItem(id=1, s
133         ourceType=PUBLIC, ownerUserId=null, name=番茄鸡蛋面, slots=SlotBundle(meal
134         Time=[午餐, 晚餐, 三餐], mood=[疲惫, 低落], scene=[工作, 校园, 家里], healthGo
135         al=[清淡, 养胃, 易消化], cuisine=[家常, 粉面], taste=[清淡, 番茄味], convenien
136         ce=[快速, 一人食]), matchScore=0.0), MealItem(id=2, sourceType=PUBLIC, own
137         erUserId=null, name=清汤馄饨, slots=SlotBundle(mealTime=[早餐, 午餐, 晚餐,
138         三餐], mood=[疲惫, 没胃口], scene=[工作, 校园, 家里], healthGoal=[清淡, 养胃,
139         暖胃], cuisine=[小吃, 粥汤], taste=[清淡, 咸鲜], convenience=[快速, 少餐具]),
140         matchScore=0.0)]}"
141     }, {
142         "phase": "RANK",
143         "agentName": null,
144         "createdAt": "2026-07-02T00:56:52.882181",
145         "eventType": "MEAL_RANKED",
146         "latencyMs": null,

```



```

134         "modelName": null,
135         "stepOrder": 10,
136         "inputTokens": null,
137         "totalTokens": null,
138         "errorMessage": null,
139         "inputPayload": "{\"excludeMealIds\":[]}",
140         "outputTokens": null,
141         "outputPayload": "{rankedCount=2, ranked=[MealItem(id=1, sourceTy
pe=PUBLIC, ownerUserId=null, name=番茄鸡蛋面, slots=SlotBundle(mealTime=[午
餐, 晚餐, 三餐], mood=[疲惫, 低落], scene=[工作, 校园, 家里], healthGoal=[清
淡, 养胃, 易消化], cuisine=[家常, 粉面], taste=[清淡, 番茄味], convenience=[快
速, 一人食]), matchScore=0.42857142857142855), MealItem(id=2, sourceType=P
UBLIC, ownerUserId=null, name=清汤馄饨, slots=SlotBundle(mealTime=[早餐, 午
餐, 晚餐, 三餐], mood=[疲惫, 没胃口], scene=[工作, 校园, 家里], healthGoal=[清
淡, 养胃, 暖胃], cuisine=[小吃, 粥汤], taste=[清淡, 咸鲜], convenience=[快速,
少餐具]), matchScore=0.42857142857142855)]}"
142     }, {
143         "phase": "AGENT",
144         "agentName": "RecommendResponseAgent",
145         "createdAt": "2026-07-02T00:56:57.121084600",
146         "eventType": "AGENT_CALL",
147         "latencyMs": 4238,
148         "modelName": "qwen-max",
149         "stepOrder": 11,
150         "inputTokens": 736,
151         "totalTokens": 865,
152         "errorMessage": null,
153         "inputPayload": "用户原话：晚饭推荐清淡低脂的\n数据源模式：PUBLIC\n本轮
槽位：SlotBundle(mealTime=[晚餐], mood=[], scene=[], healthGoal=[清淡], cui
sine=[], taste=[清淡], convenience=[])\n候选餐食：[MealItem(id=1, sourceTyp
e=PUBLIC, ownerUserId=null, name=番茄鸡蛋面, slots=SlotBundle(mealTime=[午
餐, 晚餐, 三餐], mood=[疲惫, 低落], scene=[工作, 校园, 家里], healthGoal=[清
淡, 养胃, 易消化], cuisine=[家常, 粉面], taste=[清淡, 番茄味], convenience=[快
速, 一人食]), matchScore=0.42857142857142855), MealItem(id=2, sourceType=P
UBLIC, ownerUserId=null, name=清汤馄饨, slots=SlotBundle(mealTime=[早餐, 午
餐, 晚餐, 三餐], mood=[疲惫, 没胃口], scene=[工作, 校园, 家里], healthGoal=[清
淡, 养胃, 暖胃], cuisine=[小吃, 粥汤], taste=[清淡, 咸鲜], convenience=[快速,
少餐具]), matchScore=0.42857142857142855)]\n请输出 JSON, 包含 recommendation
s 数组 (每项 mealId + reason) 和 speechText, 不要编造候选之外的餐食。",
154         "outputTokens": 129,
155         "outputPayload": "{\n  \"recommendations\": [\n    {\"mealId\":
1, \"reason\": \"番茄鸡蛋面清淡可口, 富含维生素C和蛋白质, 有助于晚餐时轻松养胃
\"},\n    {\"mealId\": 2, \"reason\": \"清汤馄饨低脂暖胃, 适合晚餐食用, 能够快
速补充体力而不增加负担\"}\n  ],\n  \"speechText\": \"想要晚饭吃得清淡点, 可以试试
番茄鸡蛋面, 它不仅口感清爽还能帮你补充营养; 或者来一碗清汤馄饨, 既暖胃又不会太油腻。想换
更特别的还是就这几款? \"\n}"
156     }, {
157         "phase": "RECOMMEND",

```

```

158         "agentName": null,
159         "createdAt": "2026-07-02T00:56:57.124081400",
160         "eventType": "RECOMMEND_RESULT_BUILT",
161         "latencyMs": null,
162         "modelName": null,
163         "stepOrder": 12,
164         "inputTokens": null,
165         "totalTokens": null,
166         "errorMessage": null,
167         "inputPayload": "{ranked=[MealItem(id=1, sourceType=PUBLIC, owner
    UserId=null, name=番茄鸡蛋面, slots=SlotBundle(mealTime=[午餐, 晚餐, 三餐],
    mood=[疲惫, 低落], scene=[工作, 校园, 家里], healthGoal=[清淡, 养胃, 易消化],
    cuisine=[家常, 粉面], taste=[清淡, 番茄味], convenience=[快速, 一人食]), matc
    hScore=0.42857142857142855), MealItem(id=2, sourceType=PUBLIC, ownerUserI
    d=null, name=清汤馄饨, slots=SlotBundle(mealTime=[早餐, 午餐, 晚餐, 三餐], mo
    od=[疲惫, 没胃口], scene=[工作, 校园, 家里], healthGoal=[清淡, 养胃, 暖胃], cu
    isine=[小吃, 粥汤], taste=[清淡, 咸鲜], convenience=[快速, 少餐具]), matchSco
    re=0.42857142857142855)], strategy=MEAL_RECOMMENDATION}",
168         "outputTokens": null,
169         "outputPayload": "RecommendResult(recommendations=[RecommendedMea
    lOption(itemId=1, sourceType=PUBLIC, name=番茄鸡蛋面, reason=番茄鸡蛋面清淡可
    口, 富含维生素C和蛋白质, 有助于晚餐时轻松养胃, matchScore=0.42857142857142855, m
    atchedSlots=SlotBundle(mealTime=[午餐, 晚餐, 三餐], mood=[疲惫, 低落], scene
    =[工作, 校园, 家里], healthGoal=[清淡, 养胃, 易消化], cuisine=[家常, 粉面], ta
    ste=[清淡, 番茄味], convenience=[快速, 一人食])), RecommendedMealOption(item
    Id=2, sourceType=PUBLIC, name=清汤馄饨, reason=清汤馄饨低脂暖胃, 适合晚餐食用,
    能够快速补充体力而不增加负担, matchScore=0.42857142857142855, matchedSlots=Sl
    otBundle(mealTime=[早餐, 午餐, 晚餐, 三餐], mood=[疲惫, 没胃口], scene=[工作,
    校园, 家里], healthGoal=[清淡, 养胃, 暖胃], cuisine=[小吃, 粥汤], taste=[清
    淡, 咸鲜], convenience=[快速, 少餐具]))], needDisclaimer=false)"
170     }, {
171         "phase": "RESPONSE",
172         "agentName": null,
173         "createdAt": "2026-07-02T00:56:57.126081500",
174         "eventType": "RESPONSE_AGENT_RESULT",
175         "latencyMs": null,
176         "modelName": null,
177         "stepOrder": 13,
178         "inputTokens": null,
179         "totalTokens": null,
180         "errorMessage": null,
181         "inputPayload": "RecommendResult(recommendations=[RecommendedMeal
    Option(itemId=1, sourceType=PUBLIC, name=番茄鸡蛋面, reason=番茄鸡蛋面清淡可
    口, 富含维生素C和蛋白质, 有助于晚餐时轻松养胃, matchScore=0.42857142857142855, m
    atchedSlots=SlotBundle(mealTime=[午餐, 晚餐, 三餐], mood=[疲惫, 低落], scene
    =[工作, 校园, 家里], healthGoal=[清淡, 养胃, 易消化], cuisine=[家常, 粉面], ta
    ste=[清淡, 番茄味], convenience=[快速, 一人食])), RecommendedMealOption(item
    Id=2, sourceType=PUBLIC, name=清汤馄饨, reason=清汤馄饨低脂暖胃, 适合晚餐食用,

```

```

182    能够快速补充体力而不增加负担, matchScore=0.42857142857142855, matchedSlots=SlotBundle(mealTime=[早餐, 午餐, 晚餐, 三餐], mood=[疲惫, 没胃口], scene=[工作, 校园, 家里], healthGoal=[清淡, 养胃, 暖胃], cuisine=[小吃, 粥汤], taste=[清淡, 咸鲜], convenience=[快速, 少餐具]))], needDisclaimer=false)",
183     "outputTokens": null,
184     "outputPayload": "ResponseResult(speechText=想要晚饭吃得清淡点, 可以试试番茄鸡蛋面, 它不仅口感清爽还能帮你补充营养; 或者来一碗清汤馄饨, 既暖胃又不会太油腻。想换更特别的还是就这几款?, displayBlocks=[MealResponse(id=1, sourceType=PUBLIC, name=番茄鸡蛋面, mealTime=[午餐, 晚餐, 三餐], mood=[疲惫, 低落], scene=[工作, 校园, 家里], healthGoal=[清淡, 养胃, 易消化], cuisine=[家常, 粉面], taste=[清淡, 番茄味], convenience=[快速, 一人食], matchScore=0.42857142857142855), MealResponse(id=2, sourceType=PUBLIC, name=清汤馄饨, mealTime=[早餐, 午餐, 晚餐, 三餐], mood=[疲惫, 没胃口], scene=[工作, 校园, 家里], healthGoal=[清淡, 养胃, 暖胃], cuisine=[小吃, 粥汤], taste=[清淡, 咸鲜], convenience=[快速, 少餐具], matchScore=0.42857142857142855)], nextAction=WAIT_USER)"
185 }, {
186     "phase": "GUARD",
187     "agentName": null,
188     "createdAt": "2026-07-02T00:56:57.127081300",
189     "eventType": "NUTRITION_GUARD_CHECKED",
190     "latencyMs": null,
191     "modelName": null,
192     "stepOrder": 14,
193     "inputTokens": null,
194     "totalTokens": null,
195     "errorMessage": null,
196     "inputPayload": "{intent=MEAL_RECOMMENDATION, response=ResponseResult(speechText=想要晚饭吃得清淡点, 可以试试番茄鸡蛋面, 它不仅口感清爽还能帮你补充营养; 或者来一碗清汤馄饨, 既暖胃又不会太油腻。想换更特别的还是就这几款?, displayBlocks=[MealResponse(id=1, sourceType=PUBLIC, name=番茄鸡蛋面, mealTime=[午餐, 晚餐, 三餐], mood=[疲惫, 低落], scene=[工作, 校园, 家里], healthGoal=[清淡, 养胃, 易消化], cuisine=[家常, 粉面], taste=[清淡, 番茄味], convenience=[快速, 一人食], matchScore=0.42857142857142855), MealResponse(id=2, sourceType=PUBLIC, name=清汤馄饨, mealTime=[早餐, 午餐, 晚餐, 三餐], mood=[疲惫, 没胃口], scene=[工作, 校园, 家里], healthGoal=[清淡, 养胃, 暖胃], cuisine=[小吃, 粥汤], taste=[清淡, 咸鲜], convenience=[快速, 少餐具], matchScore=0.42857142857142855)], nextAction=WAIT_USER)}",
197     "outputTokens": null,
198     "outputPayload": "RiskGuardResult(passed=true, riskLevel=LOW, blockedReasons=[], rewriteSuggestion=null)"
199 }, {
200     "phase": "GUARD",
201     "agentName": null,
202     "createdAt": "2026-07-02T00:56:57.128654900",
203     "eventType": "COMPLIANCE_GUARD_REWRITTEN",
204     "latencyMs": null,
205     "modelName": null,
206     "stepOrder": 15,

```

```

206         "inputTokens": null,
207         "totalTokens": null,
208         "errorMessage": null,
209         "inputPayload": null,
210         "outputTokens": null,
211         "outputPayload": "ResponseResult(speechText=想要晚饭吃得清淡点，可以
试试番茄鸡蛋面，它不仅口感清爽还能帮你补充营养；或者来一碗清汤馄饨，既暖胃又不会太油腻。想换更特别的还是就这几款？，displayBlocks=[MealResponse(id=1, sourceType=PUBL
IC, name=番茄鸡蛋面, mealTime=[午餐, 晚餐, 三餐], mood=[疲惫, 低落], scene=[工
作, 校园, 家里], healthGoal=[清淡, 养胃, 易消化], cuisine=[家常, 粉面], taste=
[清淡, 番茄味], convenience=[快速, 一人食], matchScore=0.4285714285714285
5), MealResponse(id=2, sourceType=PUBLIC, name=清汤馄饨, mealTime=[早餐, 午
餐, 晚餐, 三餐], mood=[疲惫, 没胃口], scene=[工作, 校园, 家里], healthGoal=[清
淡, 养胃, 暖胃], cuisine=[小吃, 粥汤], taste=[清淡, 咸鲜], convenience=[快速,
少餐具], matchScore=0.42857142857142855)], nextAction=WAIT_USER)"
212     }, {
213         "phase": "RESPONSE",
214         "agentName": null,
215         "createdAt": "2026-07-02T00:56:57.146906300",
216         "eventType": "RESPONSE_READY",
217         "latencyMs": null,
218         "modelName": null,
219         "stepOrder": 16,
220         "inputTokens": null,
221         "totalTokens": null,
222         "errorMessage": null,
223         "inputPayload": "SessionState(sessionId=sess_3e95795822cf494db53c
adcf65b868b7, userId=1, phase=RECOMMEND, sourceMode=PUBLIC, currentIntent
=MEAL_RECOMMENDATION, slots=SlotBundle(mealTime=[晚餐], mood=[], scene=
[], healthGoal=[清淡], cuisine=[], taste=[清淡], convenience=[]), lastReco
mmendations=[1, 2])",
224         "outputTokens": null,
225         "outputPayload": "{\"sessionId\":\"sess_3e95795822cf494db53cadcf6
5b868b7\",\"traceId\":\"trace_73862c0f31074c7786b74a6b4932f0a7\",\"respon
seType\":\"ANSWER\",\"speechText\":\"想要晚饭吃得清淡点，可以试试番茄鸡蛋面，它
不仅口感清爽还能帮你补充营养；或者来一碗清汤馄饨，既暖胃又不会太油腻。想换更特别的还是就
这几款？\",\"displayBlocks\": [{\"id\":1,\"sourceType\":\"PUBLIC\",\"name
\":\"番茄鸡蛋面\",\"mealTime\": [\"午餐\",\"晚餐\",\"三餐\"],\"mood\": [\"疲惫
\",\"低落\"],\"scene\": [\"工作\",\"校园\",\"家里\"],\"healthGoal\": [\"清淡
\",\"养胃\",\"易消化\"],\"cuisine\": [\"家常\",\"粉面\"],\"taste\": [\"清淡
\",\"番茄味\"],\"convenience\": [\"快速\",\"一人食\"],\"matchScore\":0.42857
142857142855},{\"id\":2,\"sourceType\":\"PUBLIC\",\"name\":\"清汤馄饨
\",\"mealTime\": [\"早餐\",\"午餐\",\"晚餐\",\"三餐\"],\"mood\": [\"疲惫
\",\"没胃口\"],\"scene\": [\"工作\",\"校园\",\"家里\"],\"healthGoal\": [\"清淡
\",\"养胃\",\"暖胃\"],\"cuisine\": [\"小吃\",\"粥汤\"],\"taste\": [\"清淡
\",\"咸鲜\"],\"convenience\": [\"快速\",\"少餐具\"],\"matchScore\":0.4285714
2857142855}],\"nextAction\":\"WAIT_USER\",\"clarifyQuestion\":null,\"miss
ingSlots\": []}"

```

```

226     }, {
227         "phase": "HTTP",
228         "agentName": null,
229         "createdAt": "2026-07-02T00:56:57.146906300",
230         "eventType": "REQUEST_FINISHED",
231         "latencyMs": 5183,
232         "modelName": null,
233         "stepOrder": 17,
234         "inputTokens": null,
235         "totalTokens": null,
236         "errorMessage": null,
237         "inputPayload": "{\"sessionId\":\"sess_3e95795822cf494db53cadcf65b868b7\", \"message\":\"晚饭推荐清淡低脂的\", \"sourceMode\":\"PUBLIC\", \"context\":{\"}}\",
238         "outputTokens": null,
239         "outputPayload": "{\"sessionId\":\"sess_3e95795822cf494db53cadcf65b868b7\", \"traceId\":\"trace_73862c0f31074c7786b74a6b4932f0a7\", \"responseType\":\"ANSWER\", \"speechText\":\"想要晚饭吃得清淡点，可以试试番茄鸡蛋面，它不仅口感清爽还能帮你补充营养；或者来一碗清汤馄饨，既暖胃又不会太油腻。想换更特别的还是就这几款？\", \"displayBlocks\": [{\"id\":1, \"sourceType\":\"PUBLIC\", \"name\":\"番茄鸡蛋面\", \"mealTime\": [\"午餐\", \"晚餐\", \"三餐\"], \"mood\": [\"疲惫\", \"低落\"], \"scene\": [\"工作\", \"校园\", \"家里\"], \"healthGoal\": [\"清淡\", \"养胃\", \"易消化\"], \"cuisine\": [\"家常\", \"粉面\"], \"taste\": [\"清淡\", \"番茄味\"], \"convenience\": [\"快速\", \"一人食\"], \"matchScore\":0.42857142857142855}, {\"id\":2, \"sourceType\":\"PUBLIC\", \"name\":\"清汤馄饨\", \"mealTime\": [\"早餐\", \"午餐\", \"晚餐\", \"三餐\"], \"mood\": [\"疲惫\", \"没胃口\"], \"scene\": [\"工作\", \"校园\", \"家里\"], \"healthGoal\": [\"清淡\", \"养胃\", \"暖胃\"], \"cuisine\": [\"小吃\", \"粥汤\"], \"taste\": [\"清淡\", \"咸鲜\"], \"convenience\": [\"快速\", \"少餐具\"], \"matchScore\":0.42857142857142855}], \"nextAction\":\"WAIT_USER\", \"clarifyQuestion\":null, \"missingSlots\": []}\"
240     },
241     "status": "SUCCESS",
242     "userId": 1,
243     "traceId": "trace_73862c0f31074c7786b74a6b4932f0a7",
244     "sessionId": "sess_3e95795822cf494db53cadcf65b868b7",
245     "durationMs": 5183
246 }

```

逐字稿- 评估体系设计与评估闭环

对应简历内容：

- **评估体系设计与评估闭环**：通过批量解析 Trace 的离线评估的方式，使用 后端规则、LLM Judge、用户反馈 三者加权评估输出百分制评分报告，覆盖意图识别准确率、Token消耗、端到端耗时等 10+指标，低分样本回流反哺优化系统设计，形成 Trace 标注—批量评估—指标驱动迭代优化的完整闭环；

1. 背景阐述

Agent 系统有一个棘手的问题：这个系统到底怎么样？

这不能由主观感受来判断，系统的好坏 需要量化才能说清楚

2. 实现方案

所以我在 有了 Trace 的可观测性之上，又建了一套离线评估体系：把线上请求的 Trace 当作评估数据源，设置指标规则进行批量打分。评分由后端规则、LLM Judge、用户反馈三者加权，输出百分制报告，以此评估系统质量

具体地来说

- 第一步，Trace 标注：就是对 Trace，补充「标准答案」——比如这条请求正确的意图是什么、槽位应该填什么、该不该澄清。这些关键性的决策结果，需要人工来给出评定
- 第二步，批量离线评估：选定一个时间窗口，批量拉取 Trace，自动解析出系统当时的各项数据，再和人工标注、用户反馈对照，逐条算出各维度得分，最后汇总成百分制评估报告。

评分来自三个渠道，按权重合并：

- 后端规则侧，权重占比 60%，评定的是可客观判定的硬指标，覆盖 意图识别准确率、槽位准确率、澄清决策是否正确、Token 消耗、端到端耗时、安全合规等 10 余项指标——能写死规则的走规则，成本低、可复现
- LLM Judge 侧，权重占比 10%，评定的是规则不好量化的主观质量，比如回复解释是否清楚、表达是否自然

- 用户反馈侧，权重占比30%，评定的是真实使用中的满意度，来自推荐后的点赞、差评等操作
- 三者分工，兼顾客观性、主观质量和真实满意度，通过量化的数据，才可以评估当前系统的质量

且评估不仅仅只是为了评估，而是为了持续改进，比如我们看到某一条Trace的最终得分比较低，就可以去看这条Trace的评分明细，是意图识别错了？还是端到端的延迟太久了？由此我们可以发现低分样本的问题，反过来再去改进系统，比如是改提示词还是改模型还是改业务流程或是优化系统性能。

3. 复盘总结

最后总结一下，我认为，在可观测性之上，需要建立系统的评估体系，由后端规则+LLM Judge+用户反馈三方面结合，去覆盖多项指标，得到系统的评分，再通过低分样本回流，反推进改动优化系统。以此形成闭环

和可观测性有些类似，评估体系也是在构建 Agent 这个「黑盒」中可度量、可对比、可迭代的系统能力
可观测性解决「看得见」，评估体系解决「评得准、改得对」。

评分规则

后端规则一共 9 项参与规则分。所有规则分都是 0~1，最后对非 null 项取平均得到 `ruleScore`。

具体计算可以查看代码实现，这里仅列举

一、参与规则分的 9 项指标

1. 意图准确率 (intentAccuracy)

数据来源：Trace 里意图修正事件的 `intent`，与人工标注的 `expected_intent` 对比。

评分：

- 未标注 → `null`（不参与平均）
 - 完全一致 → 1.0
 - 不一致 → 0.0
-

2. 槽位准确率 (slotAccuracy)

数据来源：Trace 里槽位合并/修正事件的 `slots`，与人工标注的 `expected_slots` 对比。

7 个标准槽位：mealTime、mood、scene、healthGoal、cuisine、taste、convenience

评分：

- 未标注 → `null`
- 只比较人工标注了非空值的槽位（空标注跳过）
- 每个槽位用集合比较（忽略顺序，不忽略多/少值）
- 得分 = 命中槽位数 / 有标注的槽位数

例：标了 3 个槽位，对 2 个 → 0.67

3. 澄清必要性准确率 (clarifyNecessityAccuracy)

数据来源： Trace 里澄清决策事件的 `action`（ASK / READY），与人工标注的 `expected_clarify_action` 对比。

评分：

- 未标注 → `null`
- 完全一致 → 1.0
- 不一致 → 0.0

4. Token 成本分 (tokenCostScore)

数据来源： Trace 里所有 Agent 调用事件的 `totalTokens` 累加。

评分（分段线性）：

Token 总量	得分
≤ 1000	1.0（满分）
1000 ~ 3000	线性衰减： $(3000 - \text{tokenCost}) / 2000$
≥ 3000	0.0

无 token 数据 → `null`

5. 延迟分 (latencyScore)

数据来源： Trace 记录的整轮 `duration_ms`（端到端耗时）。

评分（分段线性）：

耗时	得分
≤ 3 秒	1.0
3 ~ 8 秒	线性衰减： $(8000 - \text{latencyMs}) / 5000$
≥ 8 秒	0.0

无耗时数据 → `null`

6. 降级兜底分 (fallbackScore)

数据来源： 是否触发 fallback/失败，满足任一即为 fallback：

- Trace 状态为 `FAILED`
- 任意事件带 `errorMessage`
- 出现 `REQUEST_FAILED` 事件

评分：

- 未 fallback → 1.0
 - 发生 fallback → 0.0
-

7. 安全合规 (safetyCompliance)

数据来源： 最终回复文本 (`RESPONSE_AGENT_RESULT` / `RESPONSE_READY` 的 `speechText`) 。

规则： 不含以下禁用短语 → 合规

`治好` 、 `治疗` 、 `保证` 、 `一定能瘦` 、 `根治` 、 `替代医生`

评分：

- 不含禁用词 → 1.0
 - 含任一禁用词 → 0.0
-

8. 幻觉控制 (hallucinationControl)

数据来源：

- 重排候选：排序事件 `ranked[].id`
- 最终展示：响应事件 `displayBlocks[].id`

规则： 最终展示的餐食 ID 必须全部来自重排候选集合 (`responseIds ⊆ rankedIds`) 。无展示卡片时默认通过。

评分：

- 全部来自候选 → 1.0
 - 出现候选外 ID → 0.0
-

9. 多轮一致性 (multiTurnConsistency)

数据来源：仅 MEAL_ADJUST（换一批/调整推荐）场景计算：

- 调整上下文事件里的 `excludeMealIds`（应排除的历史推荐）
- 最终展示卡片 ID

评分：

- 非调整场景 → `null`（不参与平均）
 - 是调整场景但无排除列表 → 0.0
 - 有排除列表，且新推荐未命中任何排除 ID → 1.0
 - 新推荐命中了排除 ID → 0.0
-

二、规则分 (ruleScore) 怎么汇总

```
1      Double ruleScore = groupAverage(metrics, List.of(  
2          "intentAccuracy",  
3          "slotAccuracy",  
4          "clarifyNecessityAccuracy",  
5          "tokenCostScore",  
6          "latencyScore",  
7          "fallbackScore",  
8          "safetyCompliance",  
9          "hallucinationControl",  
10         "multiTurnConsistency"  
11     ));
```

- 对上述 9 项做算术平均
 - `null` 跳过，不算 0 分（缺标注、缺数据、非调整场景等）
 - 结果 0~1，展示时 ×100 为百分制
-

三、快速对照表

指标	类型	依赖	计分方式
intentAccuracy	规则分	人工标注	完全一致 1，否则 0
slotAccuracy	规则分	人工标注	有标注槽位的命中比例
clarifyNecessityAccuracy	规则分	人工标注	ASK/READY 一致 1，否则 0
tokenCostScore	规则分	Trace 自动	$\leq 1k \rightarrow 1$ ； $\geq 3k \rightarrow 0$ ；中间线性
latencyScore	规则分	Trace 自动	$\leq 3s \rightarrow 1$ ； $\geq 8s \rightarrow 0$ ；中间线性
fallbackScore	规则分	Trace 自动	未降级 1，降级 0
safetyCompliance	规则分	Trace 自动	无禁用词 1，有则 0
hallucinationControl	规则分	Trace 自动	展示 ID 全在候选内 1，否则 0
multiTurnConsistency	规则分	Trace 自动	仅调整场景；新推荐未重复旧 ID $\rightarrow 1$
tokenCost	仅展示	Trace 自动	原始 token 数
latencyMs	仅展示	Trace 自动	原始毫秒数
fallbackRate	仅展示	Trace 自动	单条 0/1，区间平均为降级率

四、和总分的关系

规则分只占单条 Trace 总分的 60%（LLM Judge 10%、用户反馈 30%）。

某块缺失时，会按实际存在的权重重新归一化，避免缺 Judge 或缺反馈把总分拉低。

可观测性 VS 评价体系

可观测性 vs 评价体系 核心区别

维度	可观测性	评价体系
核心目标	看清系统运行全过程，排查故障	量化Agent任务质量，评估业务效果
数据来源	原始运行日志、Trace链路、性能指标	基于观测数据做二次判定、打分、标注
运行时机	任务执行中实时采集	任务执行完成后自动/人工打分
目的	解决稳定性问题	解决效果优化问题
产出物	监控大盘、链路日志、告警信息	评测分数、问题样本库、版本优化报告
依赖关系	评价体系必须依赖可观测性采集的数据；可观测性可独立存在	不能脱离观测数据单独搭建评价体系

逐字稿- LLM 全链路 Fallback 设计

对应简历内容：

- **LLM 全链路 Fallback 设计**：针对全链路的 LLM 出现超时或输出异常等情况时，由后端规则与模板降级接管，保障全链路任一 LLM 异常时业务流程正常不中断，对话始终有回复；

1. 背景阐述

在生产环境里，Agent 系统的大模型并不是稳定可靠的依赖。

超时、限流、输出空文本、幻觉——这些情况在实际调用LLM中都会发生。

如果链路里任意一个 LLM 节点一旦异常就直接抛错，整条对话就中断，用户无法得到回复

这对于一个对话式的 Agent 系统我认为是不可接受的

在饮食推荐的系统中，无论 LLM 是否可用，整个系统流程都应该可以继续推进，让用户得到回复

这就要求我们必须做好对于异常情况的降级兜底处理（即 Fallback 设计）

2. 实现方案

所以我在设计上的原则是：主路径走 LLM，异常路径走 Java 规则与模板降级

在推荐的主链路上，一共有三个Agent调用，分别是意图识别、澄清追问、应答生成

每个环节都有独立的降级策略

- 第一环：理解用户想干什么（意图识别）
 - LLM 正常时，用它识别用户意图和关键信息。
 - 一旦超时或输出异常，就切到关键词规则：比如提到「胃疼」判定为健康风险提示的意图，提到「换一批」判定为调整推荐的意图，提到「吃什么」判定为推荐意图，其余默认判定为澄清意图。
 - 同时给这次识别打一个低置信度标记，后续不会贸然进入推荐，而是先追问补信息。
- 第二环：信息够不够（澄清追问）

- 要不要追问、缺什么信息 这件事，始终由后端规则决定，LLM 只负责生成口语化的追问话术
- 如果 LLM 挂了或返回空，就用固定模板追问，比如「这顿是早餐、午餐还是晚餐？」
- 第三环：给出推荐（推荐应答）
 - 推荐列表本身来自检索和排序，不依赖 LLM。LLM 只负责写推荐理由和口语化回复
 - 如果 LLM 异常，就用模板拼出理由和话术，但真实的推荐卡片照常返回，用户感知是「回复稍微模板化」，而不是系统报错。

3. 复盘总结

最后总结一下，Fallback 设计的核心价值，是把 大模型 不可靠 的系统风险给兜底住了

在一个对话式的 Agent 系统中，虽然 使用 大模型 让系统变得更加智能灵活，大模型虽然承担了一部分业务逻辑，但我认为它应该是一个可选的增强层，即使大模型异常了，后端规则与模板都能接管，保障对话始终有回复、业务流程正常推进。

逐字稿 - 后端多 Agent 协作架构

对应简历内容：

- **后端多 Agent 协作架构**：采用编排中枢+专职Worker分层模式，将意图理解、槽位澄清、推荐生成、质量评估解耦为独立推理Agent单元；LLM 仅负责结构化推理输出，会话流转与状态跃迁由编排层统一管控

1. 背景阐述

饮食推荐是典型的多轮对话场景：用户往往不会一次说清需求，系统需要先理解意图、再追问用户补齐关键的信息、最后才给出推荐。

如果把这些能力全部塞进 一个大 Prompt + 一个大模型 ，会有两个明显问题：

- 一是职责混在一起，意图识别、追问、推荐文案互相干扰，改一处容易牵一发而动全身；
- 二是流程不可控，模型可能跳过澄清直接推荐，或在不该推荐时自由发挥，业务状态难以保证一致。

因此需要把 "理解" 和 "决策" 分开：模型负责推理与生成，代码负责流程与状态。

2. 实现方案

整体采用 编排中枢 + 专职 Worker 的分层模式。

编排中枢是系统的「调度员」：接收用户请求后，按会话当前阶段决定下一步该做什么——是识别意图、追问缺失信息，还是进入推荐链路。会话阶段怎么流转、状态怎么更新，都由编排层统一管控，不交给模型自行决定。

Worker Agent 是各司其职的专家单元，彼此解耦：

- 意图理解 Agent：从用户话里提取意图和槽位，输出结构化结果
- 槽位澄清 Agent：在需要追问时，生成自然、友好的补充提问
- 推荐生成 Agent：基于已确认的候选餐食，生成推荐理由和口语化回复
- 质量评估 Agent：在离线链路里对回复质量打分，支撑后续优化

每个 Worker 只做一件事：接收上下文，输出结构化 JSON，不直接读写会话状态，也不决定下一步去哪。

检索、排序、合规校验等确定性逻辑放在 Java 规则层，不走模型——保证召回可控、结果可预期。

一句话概括分工：LLM 负责 "想明白、说清楚"，编排层负责 "走对路、守规矩"。

3. 复盘总结

最后总结一下

核心思路是把 Agent 系统从 "一个不可控的黑盒" 拆成 "可控的流程 + 可替换的能力单元"。

职责解耦后，单个 Agent 的 Prompt 和模型可以独立迭代，不影响整体链路；

由编排层统一管控，多轮对话的路径清晰、行为可预期，也更便于排查和观测。

不是让模型当 "总指挥"，而是让模型当 "专职顾问"，总指挥始终是确定性编排层——这是在 LLM 能力之上保证业务稳定性的关键设计。

AgentScope 框架

1. 基础定位

AgentScope 是阿里通义实验室开源、面向生产级的多智能体开发框架，主打工程化、可观测、分布式、高可靠，区别于 AutoGen、LangGraph 这类偏向原型实验的框架，目标直接支撑企业落地复杂智能体系统

2. 智能体核心层 Agent Engine

内置标准化智能体，可自定义扩展：

- **ReActAgent**：框架默认核心，完整推理-行动循环，支持任务规划 PlanNotebook；
- 支持单智能体、主从编排 Orchestrator-Worker、多智能体辩论、Agent 任务转交；
- 钩子（Hook）中间件机制，可拦截消息、校验权限、自定义预处理。

3. 特性 Hook

AgentScope 的 Hook 就类似于 AOP 切面，Hook 可以在 Agent 执行的任意节点插入自定义逻辑

AgentScope 的 Hook = 基于「事件监听机制」实现的 AOP 切面

3.1. Hook 是什么

可以把 Agent 的整个运行过程 想象成一条流水线，代码按步骤依次执行。

Hook（钩子） 就是这条流水线上提前开好的“小窗口 / 预留接口”

不用改动原有流水线代码，就能在任意环节插进你自己的逻辑

与 Java 的 AOP 切面非常类似：不改主干代码，只做额外增强、拦截、监控、补逻辑。

举个例子：Agent 正常流程是「接收消息 → 思考回答 → 输出结果」，利用 Hook 就能做到：

执行前：记录日志、校验参数

执行中：拦截内容、修改数据

执行后：统计耗时、上报状态

3.2. AgentScope 里具体怎么用 Hook

在 AgentScope 框架中用了统一的事件机制：

AgentScope 框架在 Agent 每一步运行时，都会自动抛出对应的事件

（比如“调用 LLM 推理之前”“LLM 推理完成之后”“工具执行之前”“工具执行完成之后”等）

我们只需要实现框架规定的 Hook 接口，重写里面的 onEvent 方法；

在这个方法里判断一下“当前是什么事件”，只处理我们关心的事件类型就行；

框架会自动在对应时机，触发对应代码。

3.3. 总结

AgentScope 的 Hook = 流水线预留的拦截点位 + 统一事件通知

不用改 Agent 原本的运行代码

写一段统一的监听代码，框架跑到对应的步骤时就会被监听到，此时按需加日志、改数据、做拦截都可以

4. 适用场景

- 复杂多智能体协作：多人辩论、分工式任务拆解（规划+检索+总结）；
- 企业业务 Agent：知识库问答、数据分析、金融投研、代码审查、自动化流程；
- 高并发在线服务：多用户会话隔离、生产级长时运行智能体服务；
- 多模态交互应用：语音客服、图文分析机器人；

- 大规模智能体仿真、科研多Agent实验。

逐字稿 - 意图识别与规则兜底

对应简历内容：

- **意图识别与规则兜底**：意图识别 Agent 识别 6 类意图与 7 维槽位，叠加 意图状态矫正、低置信降级、关键词 Fallback 三层兜底，避免错误路由与低质量推荐；槽位抽取通过词典强约束，避免幻觉；

1. 背景阐述

饮食推荐 Agent 的第一步，是把用户这句话判清楚：是要推荐、要调整、要规划，还是信息不够、存在健康风险、或根本与饮食无关。

这一步如果判错，后面整条链路都会偏，例如应该澄清追问的却直接去推荐，该拦截的风险内容却进了推荐，该调整换一批的却当成新需求，体验和结果都会很差。

大模型做意图识别，理解力好，但输出不稳定：可能超时、格式不对、置信度虚高，或出现幻觉。

所以不能只靠 Agent 一次判断就路由。需要在 LLM 主路径之外，加一层可预期、可兜底的规则体系，保证路由尽量正确、推荐质量可控。

2. 实现方案

整体思路是：**Agent 负责理解，规则负责兜底。**

主路径上，用专门的意图识别 Agent，对用户当前这句话做意图识别，系统一共定义 6 类意图：餐食推荐、信息不足需澄清、调整上轮推荐、多餐规划、健康风险、其他闲聊。意图决定后续走哪条业务分支；同时抽取 7 个维度的偏好槽位——餐次、心情、场景、健康诉求、菜系、口味、便捷性等。槽位决定检索和排序用什么条件。

在 Agent 输出之后、真正路由之前，叠加三层兜底：

1. **意图状态矫正**——结合会话上下文做二次修正。例如：用户说「换一批」，但上一轮其实还没推荐过，就不应走调整，而应回到推荐；健康风险类表述一律优先拦截，不走正常推荐。
2. **低置信降级**——Agent 给出较低置信度时，即使判成「推荐」，也不直接进入推荐，而是降级为「需要澄清」，先追问再推荐，避免低质量结果。
3. **关键词 Fallback**——Agent 调用失败、返回无法解析时，用关键词规则做保守推断，保证流程不中

断；同时返回低置信度，便于后续层继续降级处理。

槽位抽取同样不能只信模型。合法标签事先维护在词典里，Prompt 要求只能从词典中选；解析后再做一次校验过滤，模型多写、写错的标签一律丢弃。

这样形成「理解 → 矫正 → 降级 → 规则兜底」的链路：主路径尽量智能，异常路径尽量可控。

3. 复盘总结

最后总结一下，意图识别与规则兜底，解决的是 Agent 系统里「第一步不能错」的问题。

LLM 负责语义理解和复杂场景判断，规则负责边界清晰、风险可控的场景。两者叠加，既保留灵活性，又避免错误路由和低质量推荐。对饮食这类对安全性和准确性有要求的场景，「Agent + 规则」比纯 Agent 或纯规则都更稳，也是后续可观测、评估和迭代的基础。

意图识别与槽位填充

1. 概念理解

意图识别 (Intent Recognition) 与槽填充 (Slot Filling)

- 意图识别 旨在理解用户对话中的目标
- 槽位填充 关注于从对话中提取关键信息

示例： 用户说："提醒我下周一上午10点开会"

- 意图识别结果为："设置提醒"
- 槽位填充结果为：时间槽="下周一上午10点"，事件槽="开会"在本项目中，意图识别与槽位填充由

2. 意图分类 (6 类)

系统用 `Intent` 枚举定义 6 种意图，作为 Orchestrator 状态机路由的唯一依据：

意图	含义	典型用户说法
MEAL_RECOMMENDATION	请求餐食推荐	「中午吃什么」 「推荐清淡的」
CLARIFY_NEEDED	有就餐意向，但信息不足，需追问	「帮我推荐一下」 （无餐次/目标）
MEAL_ADJUST	调整上一轮推荐	「换一批」 「清淡点」 「不要太油」
MEAL_PLAN	多餐/多时段规划	「规划早中晚三餐」 「一周清淡饮食」
HEALTH_RISK	医疗诊断、极端节食等健康风险	「胃疼吃什么能治好」 「绝食减肥」
OTHER	与饮食无关	「你是 AI 吗」 「推荐电影」

关键判别规则（来自 `intent.txt`）：

- 最近 3 轮已推荐过餐食 + 当前句含「换一批/不太想吃」 → MEAL_ADJUST

- 未提到餐次和健康诉求 → CLARIFY_NEEDED
- 含「三餐/早中晚/一周饮食」→ MEAL_PLAN
- 医疗诊断/极端节食/特殊人群 → HEALTH_RISK

LLM 失败时会走关键词兜底；输出后还有 IntentReviseService 做二次矫正（如低置信度降级、无历史推荐时 MEAL_ADJUST 降级等）。

3. 槽位信息（7 维）

槽位模型为 SlotBundle，与餐食标签体系一一对应，共 7 个维度：

槽位字段	含义	示例标签
mealTime	餐次	早餐、午餐、晚餐、三餐...
mood	心情/状态	疲惫、烦躁、开心、没胃口...
scene	就餐场景	工作、校园、加班、聚餐...
healthGoal	健康诉求	减脂、清淡、养胃、高蛋白...
cuisine	菜系	川菜、轻食、粉面、粥汤...
taste	口味	清淡、辣、麻辣、咸鲜...
convenience	便捷需求	快速、一人食、外带方便...

约束要点：

- 合法标签存于 diet_slot_option 表（约 93 条），LLM 只能从字典中选，不能自创标签
- 支持语义归一：如「午饭」→「午餐」，「减肥」→「减脂」
- 单字段可多值（逗号/顿号分隔）
- 多轮对话通过 SlotMergeService 累积：本轮有值则更新，为空则保留历史

推荐所需最低槽位（ClarifyRuleService）：

- mealTime 必填
- healthGoal 条件必填：若已有菜系/口味/场景/便捷性等强偏好，可省略

4. 完整示例（饮食场景）

示例 1：餐食推荐

用户说：「加班好累，中午在公司想吃点清淡快速的」

维度	结果
意图识别	MEAL_RECOMMENDATION
mealTime	午餐
mood	疲惫
scene	工作
healthGoal	清淡
convenience	快速

→ 路由到推荐链路：检索 → 排序 → 生成推荐理由。

示例 2：信息不足

用户说：「帮我推荐一下」

维度	结果
意图识别	CLARIFY_NEEDED
槽位	各维度为空或极少

→ 路由到 ClarifyAgent 追问：「您是想吃哪一餐？有什么口味或健康上的偏好吗？」

示例 3：调整推荐（需上下文）

上下文：上一轮 assistant 已推荐 3 款餐食

用户说：「换一批，再清淡点」

维度	结果
意图识别	MEAL_ADJUST
healthGoal	清淡
taste	清淡

→ 基于上轮推荐结果重新检索/排序，而非全新推荐。

示例 4：多餐规划

用户说：「帮我规划今天早中晚三餐，要清淡一点」

维度	结果
意图识别	MEAL_PLAN
mealTime	三餐
healthGoal	清淡

示例 5：健康风险

用户说：「胃疼吃什么能治好」

维度	结果
意图识别	HEALTH_RISK
槽位	通常不进入推荐

→ 走 `RiskGuardService` 保守回复，不给出医疗建议。

5. IntentAgent 输出格式示例

```
1 {  
2   "intent": "MEAL_RECOMMENDATION",  
3   "slots": {  
4     "mealTime": "午餐",  
5     "mood": "疲惫",  
6     "scene": "工作",  
7     "healthGoal": "减脂, 高蛋白",  
8     "cuisine": null,  
9     "taste": "清淡",  
10    "convenience": "快速"  
11  },  
12  "confidence": 0.9  
13 }
```

合并后的槽位会持久化到 `diet_sessions.slots`

并用于餐食检索（`JSON_OVERLAPS`）、排序打分和推荐理由生成。

完整产品设计

对应简历内容：

- **完整产品设计：**用户侧支持双数据源对话推荐与个人菜单维护，研发侧支持 Trace 排查、标注与批量评估；覆盖对话推荐、个人餐食库管理、Trace 标注与批量评估四套能力，打通使用—反馈—标注—评估—迭代的工程闭环；

可以直接屏幕共享，启动本地服务，访问系统页面。给面试官边演示边讲解，这样我觉得还是不错的



如图所示，首页上有标识 有Trace 页面、有评估页面，所以这当然是一个研发角色登录才会展示成这样

如果是正常用户登录，就只会有前面的 首页、聊天推荐、个人餐食、公共餐食 这几个页面
不会看得到 Trace 和 评估 页面

但登录和权限从来不是我们校招生的项目重点，因此不需要真的代码实现登录和权限功能，像上面所说解释即可

页面上展示的功能都是基本实现了的

00-AgentScope与SpringBoot集成

1. 技术选型

组件	版本/选型	用途
agentscope-spring-boot-starter	1.0.11	Agent 框架 + Spring 自动配置
DashScopeChatModel	通义千问 API	LLM 后端
ReActAgent	AgentScope 核心	各 Worker Agent 运行时
Spring Boot	3.3.13	Web / DI / 配置
Java	21	语言运行时

POM 依赖：

```
1 <dependency>
2   <groupId>io.agentscope</groupId>
3   <artifactId>agentscope-spring-boot-starter</artifactId>
4   <version>1.0.11</version>
5 </dependency>
```

2. 模型 Bean 配置

`DietAgentScopeConfig` 声明两个 Spring Bean：

```

1  @Bean("DietMainChatModel")
2  public Model DietMainChatModel() {
3      return DashScopeChatModel.builder()
4          .apiKey(apiKey)
5          .modelName(mainModelName)    // qwen-max
6          .build();
7  }
8
9  @Bean("DietLightChatModel")
10 public Model DietLightChatModel() {
11     return DashScopeChatModel.builder()
12         .apiKey(apiKey)
13         .modelName(lightModelName)    // qwen-turbo
14         .build();
15 }

```

配置项：

```

1  agentscope:
2    dashscope:
3      api-key: sk-xxx
4  diet:
5    llm:
6      main-model: qwen-max
7      light-model: qwen-turbo

```

双 Bean 策略：按任务复杂度注入不同 Model，Builder 通过 `@Qualifier` 区分。

3. Builder 模式构建 Agent

每个 Agent 独立 `AgentBuilder` Component：

Builder	Model	Agent name
IntentAgentBuilder	DietLightChatModel	diet_intent_agent
ClarifyAgentBuilder	DietLightChatModel	diet_clarify_agent
RecommendResponseAgentBuilder	DietMainChatModel	diet_recommend_response_agent
EvaluationJudgeAgentBuilder	DietLightChatModel	diet_evaluation_judge_agent

统一构建模式：

```
1 ReActAgent.builder()
2     .name("...")
3     .model(xxxModel)
4     .sysPrompt(promptLoader.load("..."))
5     .memory(new InMemoryMemory())
6     .build();
```

4. Agent 调用方式

项目未使用 AgentScope 的复杂 Tool/ReAct 循环做业务工具调用，而是 **单轮 LLM 调用 + JSON 输出**：

```
1 Msg response = agent.call(Msg.builder()
2     .role(MsgRole.USER)
3     .textContent(inputText)
4     .build()).block();
```

包装在 `AgentTraceService.callAgent()` 中：

- 同步 block 等待
- 记录 latency、token usage
- 异常向上抛或由各 Service catch fallback

这是「AgentScope 作 LLM 网关 + 项目自管编排」的轻量用法，而非全自动 ReAct 工具链。

5. Memory 使用策略

每个 Agent 使用 `InMemoryMemory()`，但业务上 **不依赖 Agent 记忆做多轮**，而是自己实现记忆

```
1 agent.getMemory().clear(); // 每轮调用前
```

多轮上下文由 Orchestrator 拼进 user prompt：

- recentHistory 最近对话内容
- knownSlots 已知的槽位信息
- slotOptions 可选的槽位选项

原因：

- SessionState 是权威状态源
- 避免 Agent memory 与 DB 双写不一致

6. 与业务层的分层

```
1  Spring Controller
2      → DietOrchestratorService (纯 Java 状态机)
3          → XxxAgentService (解析 JSON、fallback)
4              → AgentTraceService.callAgent
5                  → ReActAgent (AgentScope)
6                      → DashScopeChatModel
```

AgentScope 仅占据 「模型调用 + sysPrompt」 层，不参与路由、检索、合规。

7. 集成优势与边界

优势：

- Starter 简化 DashScope 配置
- ReActAgent API 统一，换模型只改 Config
- 与 Spring 生态（MyBatis、Trace、Evaluation）无缝共存

边界（诚实说明）：

- 未使用 AgentScope 多 Agent 对话/MsgHub 自动协作
- 协作逻辑在自研 Orchestrator
- EvaluationJudgeAgent 未纳入 AgentFactory AgentSet（评估独立 build）

8. 关键代码索引

文件	说明
----	----

<code>pom.xml</code>	agentscope 依赖
<code>DietAgentScopeConfig</code>	Model Bean
<code>com.diet.agent.builder.*</code>	Agent 构建
<code>AgentTraceService#callAgent</code>	统一调用
<code>application.yml</code>	api-key / model 名

01-Agent多智能体设计

1. 总体架构

本项目采用 **Orchestrator + Worker Agent** 模式：由 `DietOrchestratorService` 统一驱动状态机，各 Worker Agent 只负责产出结构化结果，**不直接读写 SessionState**。



核心设计原则：

- LLM 负责理解与生成，Java 负责状态、路由、检索与合规
- Agent 无状态调用：每次调用前 `agent.getMemory().clear()`，上下文由 Orchestrator 通过 prompt 注入
- 会话级 Agent 隔离：`AgentFactory` 按 `sessionId + promptVersion` 缓存 AgentSet，避免多用户记忆串话

2. Agent 清单与职责

Agent	构建器	模型	职责
IntentAgent	<code>IntentAgentBuilder</code>	qwen-turbo（轻量）	6 类意图分类 + 7 维槽位抽取，输出 JSON

ClarifyAgent	ClarifyAgentBuilder	qwen-turbo（轻量）	槽位不足时生成自然语言追问（是否追问由规则决定）
RecommendResponseAgent	RecommendResponseAgentBuilder	qwen-max（主模型）	基于 Top3 候选一次性生成推荐理由 + speechText
EvaluationJudgeAgent	EvaluationJudgeAgentBuilder	qwen-turbo（轻量）	离线评估：解释质量、自然度（1-5 分）

Prompt 均外置在 `src/main/resources/diet/prompts/`，通过 `PromptLoader` 加载，支持 `diet.prompt.version` 配置版本隔离。

3. AgentFactory：会话级缓存

```
1 // AgentFactory.java
2 AgentSet get(String sessionId) // intent + clarify + recommendResponse 三件套
3 void remove(String sessionId) // 会话结束时释放
```

设计要点：

- LRU 淘汰：最大缓存 1000 个会话 AgentSet（`LinkedHashMap` + `removeEldestEntry`）
- Prompt 版本键：`sessionId::promptVersion`，Prompt 升级后不会复用旧 Agent
- AgentSet 结构：一个会话内共享三个 ReActAgent 实例，但每轮调用前清空 memory

4. 双模型策略

`DietAgentScopeConfig` 配置两个 DashScope 模型 Bean：

Bean	默认模型	使用场景
DietLightChatModel	qwen-turbo	IntentAgent、ClarifyAgent、EvaluationJudgeAgent

DietMainChatModel	qwen-max	RecommendResponseAgent
-------------------	----------	------------------------

目的：分类/追问/评估用轻量模型降延迟和成本；最终推荐文案用主模型保质量。

5. Agent 调用统一入口

所有 Agent 调用经 `AgentTraceService.callAgent()` 包装：

- 同步 `agent.call(...).block()` 等待 LLM 返回
- 自动记录 `AGENT_CALL` 事件：agentName、modelName、input、output、latencyMs、token 用量
- 异常时 markFailed 并向上抛出

各 Service 在调用前统一：

1. `agentFactory.get(sessionId).xxx()` 取实例
2. `agent.getMemory().clear()` 清记忆
3. 构造结构化 user prompt（含 session 上下文、槽位字典等）
4. `agentTraceService.callAgent(...)` 发起调用

6. LLM 与 Java 的分工边界

环节	LLM	Java 规则/服务
意图识别	IntentAgent 主识别	IntentReviseService 二次矫正；关键词 fallback
槽位是否足够	不决策	ClarifyRuleService 判 ASK/READY
追问文案	ClarifyAgent 生成	ClarifyRuleService 模板兜底
餐食召回	不参与	MealSearchService (JSON_OVERLAPS)

餐食排序	不参与	MealRankService (7 维 overlap 打分)
推荐理由	RecommendResponseAgent	候选 ID 白名单校验；模板兜底
健康合规	不参与	RiskGuardService 关键词拦截 + 保守回复

这是典型的「Agent 做理解，Orchestrator 做决策」多 Agent 协作模式，避免 LLM 直接操控业务状态。

7. 关键代码入口

- 编排入口: `DietOrchestratorService#dietChat`
- Agent 工厂: `AgentFactory`
- Agent 构建: `com.diet.agent.builder.*`
- 模型配置: `DietAgentScopeConfig`
- Prompt 加载: `PromptLoader`

02-意图识别与规则兜底设计

1. 意图体系

系统定义 6 类意图（ Intent 枚举），作为 Orchestrator 状态机路由的唯一依据：

意图	含义	典型用户说法
MEAL_RECOMMENDATION	餐食推荐	「中午吃什么」「推荐清淡的」
CLARIFY_NEEDED	信息不足需追问	「帮我推荐一下」（无餐次/目标）
MEAL_ADJUST	调整上轮推荐	「换一批」「清淡点」「不要太油」
MEAL_PLAN	多餐规划	「规划早中晚三餐」「一周清淡饮食」
HEALTH_RISK	健康/医疗风险	「胃疼吃什么能治好」「绝食减肥」
OTHER	与饮食无关	「你是 AI 吗」「推荐电影」

2. 意图识别主链路

```

1  用户输入
2  |
3  ▼
4  IntentAgentService.recognize()
5  |—— 加载 slotOptions (DB 槽位字典)
6  |—— 注入: userId / sessionId / recentHistory(3轮) / knownSlots / slotOptions
7  |—— IntentAgent (qwen-turbo) → JSON { intent, slots, confidence }
8  |—— parseResult() 解析 + 字典校验
9  |
10 ▼
11 IntentReviseService.revise() ← 规则二次矫正
12 |
13 ▼
14 Orchestrator switch(intent) 路由

```

2.1 IntentAgent 输入上下文

Prompt 要求 Agent 只对「当前这一句」分类，但会附带：

- **recentHistory**: 最近 3 轮对话摘要（用于识别「换一批」等上下文相关调整）
- **knownSlots**: 会话历史槽位
- **slotOptions**: DB 合法标签字典（约束槽位输出）

2.2 Prompt 关键判别规则 (intent.txt)

- 最近 3 轮 assistant 已推荐过餐食 + 当前句含「换一批/不太想吃」→ **必须 MEAL_ADJUST**
- 未提到餐次和健康诉求 → **CLARIFY_NEEDED**
- 含「三餐/早中晚/一周饮食」→ **MEAL_PLAN**
- 医疗诊断/极端节食/特殊人群 → **HEALTH_RISK**

3. 三层规则兜底

3.1 第一层：IntentAgent 解析兜底

`IntentAgentService.parseIntent()` :

- `intent` 字段缺失或非法枚举 → 走 `fallbackIntent()` 关键词规则

- LLM 整体异常（超时/JSON 解析失败）→ `fallback()` 返回低置信度结果（confidence=0.2）

关键词 `fallback` 优先级（`fallbackIntent`）：

1. 空输入 → `CLARIFY_NEEDED`
2. 健康风险词（胃疼、糖尿病、孕妇、治好、绝食...）→ `HEALTH_RISK`
3. 调整词（换一批、清淡点、便宜点...）→ `MEAL_ADJUST`
4. 规划词（三餐、早中晚、一周）→ `MEAL_PLAN`
5. 闲聊词（你是谁、你好）→ `OTHER`
6. 推荐词（吃什么、推荐、晚饭...）→ `MEAL_RECOMMENDATION`
7. 默认 → `CLARIFY_NEEDED`

3.2 第二层：IntentReviseService 状态感知矫正

LLM 输出后、路由前，`IntentReviseService.revise()` 结合 `SessionState` 做二次修正：

规则	条件	矫正结果
健康风险优先	intent=HEALTH_RISK 或用户输入含健康风险关键词	强制 HEALTH_RISK
无历史推荐不可调整	intent=MEAL_ADJUST 且 lastRecommendations 为空	降级为 MEAL_RECOMMENDATION
低置信度不进推荐	intent=MEAL_RECOMMENDATION 且 confidence < 0.4	降级为 CLARIFY_NEEDED

健康风险关键词比 IntentAgent fallback 更全（含儿童、高血压、诊断、处方、一天不吃等）。

3.3 第三层：Orchestrator 路由层兜底

- **PERSONAL 空库**：无个人餐食时直接返回引导文案，跳过意图识别后的推荐链路
- **HEALTH_RISK 分支**：不走推荐，返回 `RiskGuardService.conservativeMessage()`
- **OTHER 分支**：固定引导文案，不调用 LLM
- **推荐链路内 ClarifyRule**：即使 LLM 判 MEAL_RECOMMENDATION，规则层仍可能 ASK 追问

注释原文：「不能完全依靠 agent 的意图识别，在进入推荐之前，规则层面上也需要判断是否有足够的信息。」

4. 置信度机制

- IntentAgent 输出 `confidence` (默认 0.5)
 - `< 0.4` 时 IntentRevise 将 MEAL_RECOMMENDATION 降级为 CLARIFY_NEEDED
 - LLM 完全失败时 `confidence` 固定 0.2, 触发低置信度降级或澄清
-

5. Trace 可观测点

意图相关 Trace 事件:

- `INTENT_RECOGNIZED` : IntentAgent 原始输出
- `INTENT_REVISED` : 规则矫正后最终 intent
- `ROUTE_SELECTED` : 路由目标

便于对比 LLM 原始判断 vs 规则矫正 vs 最终路由, 支撑评估中的 `intentAccuracy` 指标。

6. 关键代码

- `IntentAgentService` : LLM 识别 + parse + fallback
- `IntentReviseService` : 状态感知二次矫正
- `DietOrchestratorService#handleTurn` : 路由 switch
- Prompt: `diet/prompts/intent.txt`

03-槽位抽取设计

1. 槽位模型

系统采用 7 维标准槽位（ SlotBundle ），与餐食标签体系一一对应：

槽位字段	含义	示例标签
mealTime	餐次	早餐、午餐、晚餐、三餐...
mood	心情	疲惫、烦躁、开心、没胃口...
scene	场景	工作、校园、加班、聚餐...
healthGoal	健康诉求	减脂、清淡、养胃、高蛋白...
cuisine	菜系	川菜、轻食、粉面、粥汤...
taste	口味	清淡、辣、麻辣、咸鲜...
convenience	便捷需求	快速、一人食、外带方便...

合法标签全部存储在 diet_slot_option 表，通过 SlotOptionService.findAllOptions() 加载，LLM 只能从字典中选，不能创造新标签。

2. 槽位抽取链路

```

1  IntentAgent 输出 slots JSON
2      |
3      ▼
4  SlotJsonPicker.pick() ← 逐字段过滤非法值、支持多值分隔
5      |
6      ▼
7  SlotBundle (本轮槽位)
8      |
9      ▼
10 SlotMergeService.merge(history, current) ← 多轮累积
11      |
12      ▼
13 SessionState.slots (持久化到 diet_sessions)

```

2.1 IntentAgent 内抽取

`IntentAgentService.buildUserPrompt()` 将 `slotOptions` 字典传入 Prompt, 要求:

- 语义归一 (如「午饭」→「午餐」, 「减肥」→「减脂」)
- 无合适候选则输出 null, 不创造标签
- 支持单字段多值 (逗号/顿号分隔)

`parseSlots()` 通过 `SlotJsonPicker` 对 7 个字段逐一提取。

2.2 SlotJsonPicker: 结构化校验

```

1  // 支持 array 或 string 两种 JSON 格式
2  // 多值分隔符: , , 、 ; ; / |
3  // 只保留 allowed 字典内的值, 去重

```

这是槽位抽取的 **硬约束层**, 即使 LLM 幻觉出非法标签也会被过滤掉。

2.3 SlotMergeService: 多轮累积策略

```

1  merge(history, current):
2      每个维度: current 非空 → 用 current; current 空 → 保留 history

```

设计意图: 用户多轮补充信息时增量累积, 本轮未提及的维度不被清空。

3. 槽位字典管理

3.1 数据来源

- 表: `diet_slot_option` (slot_name + option_value + sort_order + enabled)
- 预置约 93 条标签 (7 维 × 各候选值)
- API: `GET /api/v1/diet/slot-options` 供前端餐食录入表单使用

3.2 校验入口

- Intent 抽取后: SlotJsonPicker 静默过滤
- 个人餐食 CRUD: `SlotOptionService.validate()` 严格校验, 非法标签抛 `DietException`

4. 澄清规则与槽位完整性

`ClarifyRuleService.missingSlots()` 定义「推荐所需最低槽位」:

规则	说明
mealTime 必填	未识别餐次 → 必须追问
healthGoal 条件必填	无 healthGoal 且无强偏好 (cuisine/taste/scene/convenience 任一非空) → 追问 healthGoal
有强偏好可缺 healthGoal	用户已明确菜系/口味/场景/便捷性时, healthGoal 可省略

这与 IntentAgent 的 CLARIFY_NEEDED 意图形成 **双保险**: 意图层和信息完整性层分别判断。

5. 槽位在推荐链路中的使用

阶段	使用方式
----	------

MealSearch	MySQL <code>JSON_OVERLAPS</code> 按 7 维标签召回 (任一有交集即命中, 空维度不参与过滤)
MealRank	7 维 overlap 比例求平均, 按 matchScore 降序 取 Top10
RecommendResponseAgent	作为 prompt 输入, 生成贴合槽位的推荐理由
RiskGuard	healthGoal 含减脂/低糖/控碳水/养胃时触发免 责声明

6. 持久化

槽位存储在 `diet_sessions.slots` JSON 列, 结构:

```
1 {
2   "mealTime": ["午餐"],
3   "mood": ["疲惫"],
4   "scene": ["工作"],
5   "healthGoal": ["清淡"],
6   "cuisine": [],
7   "taste": [],
8   "convenience": ["快速"],
9   "_meta": {
10    "sourceMode": "PUBLIC",
11    "currentIntent": "MEAL_RECOMMENDATION"
12  }
13 }
```

`_meta` 由 `SessionStateService` 维护, 与 7 维槽位同列存储。

7. 关键代码

- `SlotBundle` : 7 维槽位模型
- `IntentAgentService#parseSlots` : LLM 槽位解析
- `SlotJsonPicker` : 字典过滤与多值解析

- SlotMergeService : 多轮合并
- ClarifyRuleService : 完整性判定
- SlotOptionService : 字典加载与校验

04-意图路由与链路划分

1. 一轮对话完整链路



2. 路由分发 (switch intent)

DietOrchestratorService#handleTurn 在 INTENT_REVISED 后按最终 intent 分发:

路由目标	处理函数	后续链路
MEAL_RECOMMENDATION	handleRecommendation	合并槽位 → Clarify → 推荐流水线
CLARIFY_NEEDED	handleRecommendation	同上 (规则层可能 ASK)
MEAL_ADJUST	handleAdjust	合并槽位 → excludeMealIds → 推荐流水线
MEAL_PLAN	handlePlan	合并槽位 → phase=PLAN → 推荐流水线
HEALTH_RISK	handleHealthRisk	保守文案, 不走推荐

OTHER	handleChitchat	固定引导文案，不调用 LLM
-------	----------------	----------------

3. 推荐主链路 (handleRecommendation → completeRecommendation)

```

1  SlotMergeService.merge()
2      |
3      ▼
4  ClarifyAgentService.decide()
5      ├── ClarifyRuleService.missingSlots() → 规则判 ASK/READY
6      ├── ASK → ClarifyAgent 生成追问 → completeAsk → 返回 CLARIFY 响应
7      └── READY → completeRecommendation
8          |
9          ▼
10 completeRecommendation (推荐流水线) :
11     ① MealSearchService.search()          → JSON_OVERLAPS 召回 ≤50
12     ② MealRankService.rank()              → 7维打分 Top10, exclude 已推荐 ID
13     ③ ranked 为空 → 空库提示
14     ④ RecommendResponseAgent              → Top3 理由 + speechText
15     ⑤ RiskGuardService.check()            → 合规拦截, 必要时改写
16     ⑥ 更新 lastRecommendations + save SessionState
17     ⑦ appendMessage(assistant) + 返回 ANSWER 响应 (含餐食卡片)

```

3.1 调整链路 (MEAL_ADJUST)

```

1  handleAdjust:
2      merge slots
3      excludeMealIds = state.lastRecommendations (累积已推荐 ID)
4      phase = RECOMMEND
5      → completeRecommendation(..., excludeMealIds)

```

Rank 层过滤 excludeMealIds, 实现「换一批」不重复推荐。

3.2 澄清链路 (ASK)


```

1  completeAsk:
2      phase = CLARIFY
3      save SessionState
4      appendMessage(assistant, clarifyQuestion)
5      return ChatResponse.clarify(sessionId, traceId, question, missingSlots)

```

前端 `responseType=CLARIFY`，展示 `missingSlots` 标签 chips。

4. 会话状态机 (SessionPhase)

Phase	含义	触发时机
START	初始	新建会话
CLARIFY	澄清等待	Clarify ASK 返回后
RECOMMEND	推荐中	槽位足够进入推荐
PLAN	多餐规划	MEAL_PLAN 路由

`SessionState` 还维护：

- `currentIntent`：当前/上一轮确认意图
- `slots`：多轮累积 7 维槽位
- `lastRecommendations`：已推荐 mealId 列表（换一批排除用）
- `sourceMode`：PERSONAL / PUBLIC 数据源隔离

5. 并发与一致性

- 会话级锁：`ConcurrentHashMap<sessionId, lock>` + `synchronized`，同 session 串行写状态
- Orchestrator 独占写 `SessionState`：Agent 不直接写 DB
- Trace 贯穿整轮：`try-with-resources TraceScope`，close 时 flush 到 `diet_request_trace`

6. 数据源模式（SourceMode）

模式	检索范围	前置检查
PERSONAL	owner_user_id = 当前用户	无个人餐食 → 引导录入
PUBLIC	source_type = PUBLIC	无额外检查

PERSONAL 与 PUBLIC 禁止自动混查，由请求 `sourceMode` 决定。

7. 响应类型（ChatResponse）

responseType	场景	关键字段
ANSWER	推荐/固定回复/空库提示	speechText, displayBlocks, nextAction
CLARIFY	槽位不足追问	clarifyQuestion, missingSlots, nextAction=ASK_CLARIFY

每条响应携带 `sessionId` + `traceId`，便于前端多轮续聊和后台 Trace 关联。

8. 链路阶段与 Trace phase 对照

Phase	典型 eventType
HTTP	REQUEST_RECEIVED, REQUEST_FINISHED, REQUEST_FAILED
SESSION	USER_MESSAGE_RECORDED
INTENT	INTENT_RECOGNIZED, INTENT_REVISIED
ROUTE	ROUTE_SELECTED, PERSONAL_LIBRARY_EMPTY
SLOT	SLOTS_MERGED

CLARIFY	CLARIFY_DECISION, RESPONSE_READY
SEARCH	MEAL_SEARCHED
RANK	MEAL_RANKED
RECOMMEND	RECOMMEND_RESULT_BUILT, NO_MEAL_MATCHED
RESPONSE	RESPONSE_AGENT_RESULT, RESPONSE_READY
GUARD	NUTRITION_GUARD_CHECKED, NUTRITION_GUARD_REWRITTEN
ADJUST	ADJUST_CONTEXT_RESOLVED
PLAN	PLAN_CONTEXT_RESOLVED
AGENT	AGENT_CALL (Intent/Clarify/RecommendResponse/Judge)

9. 关键代码

- `DietOrchestratorService` : 完整状态机
- `DietChatController` : HTTP 入口 `POST /api/v1/diet/chat`
- `SessionStateService` : 会话状态 load/save
- `SessionService` : 消息落库 `diet_messages`

05-多轮对话状态管理

1. 多轮场景举例

轮次	用户说	系统需要记住
1	「工作很忙，推荐快速的」	scene=工作, convenience=快速
2	「午餐」	mealTime=午餐（合并上轮 scene/convenience）
3	「换一批」	排除上轮 3 个 mealId，保留槽位

没有会话状态，每轮都是「冷启动」，无法实现澄清补全、换一批、上下文调整。

2. SessionState 核心模型

```
1  SessionState {
2      sessionId, userId,
3      phase,           // START | CLARIFY | RECOMMEND | PLAN
4      sourceMode,      // PERSONAL | PUBLIC
5      currentIntent,   // 当前/上一轮确认意图
6      slots,           // SlotBundle 7 维累积槽位
7      lastRecommendations // 已推荐 mealId 列表（累积）
8  }
```

持久化表： `diet_sessions`

- `phase` 独立列
- `slots` JSON 列（含 `_meta.sourceMode` / `_meta.currentIntent`）
- `last_recommendations` JSON 数组列

3. 会话生命周期

```
1 POST /chat (sessionId=null)
2   → SessionStateService.create() → 新 sess_xxx
3
4 POST /chat (sessionId=已有)
5   → loadOrCreate() → 从 DB 恢复 slots/phase/lastRecommendations
6
7 每轮结束
8   → sessionStateService.save()
9   → sessionService.appendMessage() → diet_messages
```

消息表 `diet_messages` 记录每轮 user/assistant 原文 + intent + traceId, 供 IntentAgent 拉取 `recentConversationTurns(sessionId, 3)`。

4. 槽位增量合并: SlotMergeService

```
1 choose(history, current):
2   current 非空 → 用 current
3   current 空   → 保留 history
```

7 维各自独立合并。设计意图:

- 用户第二轮只补「午餐」, 不丢失第一轮的「快速/工作」
- 本轮 IntentAgent 未识别到的维度不会被清空

Trace: `SLOTS_MERGED` (stateSlots + intentSlots → mergedSlots)

5. 换一批: lastRecommendations 累积排除

5.1 写入

`completeRecommendation` 在推荐成功之后:

```

1 List<Long> lastIds = recommend.recommendations().stream().map(option -> option.itemId()).toList();
2 SessionState savedState = state.appendLastRecommendations(lastIds);

```

`appendLastRecommendations` 使用 `LinkedHashSet` 累积去重，保留插入顺序。

5.2 读取与使用

`handleAdjust` (MEAL_ADJUST 路由) :

```

1 List<Long> excludeMealIds = state.lastRecommendations();
2 return completeRecommendation(..., excludeMealIds);

```

`MealRankService.rank` 过滤 `excludeIds`，新推荐不会包含已推荐过的 ID。

5.3 IntentRevise 保护

无 `lastRecommendations` 时，MEAL_ADJUST 降级为 MEAL_RECOMMENDATION——避免「换一批」误触但无历史可排除。

Trace: `ADJUST_CONTEXT_RESOLVED` (mergedSlots + excludeMealIds)

评估指标: `multiTurnConsistency` (新推荐是否命中 `excludeIds`)

6. Phase 状态机

Phase	进入条件	含义
START	新建会话	初始
CLARIFY	Clarify ASK 返回	等待用户补充槽位
RECOMMEND	槽位足够进入推荐 / MEAL_ADJUST	推荐链路中
PLAN	MEAL_PLAN 路由	多餐规划

Phase 写入 DB，便于排查会话卡在哪个阶段。

7. 并发安全：会话级锁

```
1  Object lock = sessionLocks.computeIfAbsent(sessionId, key -> new Object());
2  synchronized (lock) {
3      ChatResponse response = handleTurn(...);
4  }
```

同一 sessionId 并发请求串行执行，避免：

- 两轮同时 merge slots 互相覆盖
- lastRecommendations 竞态丢失

8. Orchestrator 独占写 vs Agent 无状态

设计原则（AgentFactory 注释）：

Worker 只产出结构化结果，不直接读写 SessionState。

角色	读写 SessionState
DietOrchestratorService	唯一写入方
IntentAgentService	只读 knownSlots（prompt 注入）
ClarifyAgentService	只读 mergedSlots
RecommendResponseAgentService	只读 slots

Agent 每轮 `memory.clear()`，状态权威来源是 DB + Orchestrator，避免 Agent 内部记忆与 DB 不一致。

9. 关键代码索引

类	职责
<code>SessionState</code>	不可变式 withXxx / appendLastRecommendations

SessionStateService	loadOrCreate / save / slots JSON 序列化
SessionService	消息落库 + recentConversationTurns
SlotMergeService	多轮槽位合并
DietOrchestratorService	锁 + 状态更新时机

06-双数据源-PERSONAL与PUBLIC

1. 业务动机

有这样的业务背景：用户就餐场景高度个性化——学生有食堂菜单、职场人有公司饭堂、也有人依赖公共外卖库。

单一公共推荐库无法覆盖「我家食堂有什么」的真实约束。因此系统支持两种数据源：

模式	含义	典型场景
PERSONAL	用户自建个人餐食库	录入食堂常吃的菜，Agent 只在其中推荐
PUBLIC	平台公共餐食库	通用推荐，无需事先录入

2. 数据模型隔离

`meal_item` 表通过两字段区分：

```
1  source_type  VARCHAR(16)  -- 'PERSONAL' | 'PUBLIC'
2  owner_user_id BIGINT NULL  -- PERSONAL 时为 userId, PUBLIC 时为 NULL
```

索引设计：

- `idx_public_meal_source(source_type)` — 公共库查询
- `idx_private_meal_source(owner_user_id, source_type)` — 个人库按用户查

硬隔离原则：后端 SQL 层 `<choose>` 分支，PERSONAL 与 PUBLIC 永不混查。

3. 请求级绑定：SourceMode

`ChatRequest.sourceMode` 必填，Orchestrator 校验：

```

1 if (request.sourceMode() == null) {
2     throw new DietException("sourceMode 不能为空, 请选择 PERSONAL 或 PUBLIC");
3 }

```

`SessionState.sourceMode` 持久化在 `diet_sessions.slots._meta.sourceMode` , 多轮对话保持同一数据源。

前端聊天页可切换模式; 切换后新 session 或同 session 下 `_meta` 以 DB 已存值优先 (`SessionStateService.parseSourceMode`) 。

4. 三条前置/后置校验链路

4.1 PERSONAL 空库前置检查

`DietOrchestratorService#handleTurn` 在意图识别 之前:

```

1 if (sourceMode == PERSONAL && !mealService.hasPersonalMeals(userId)) {
2     return completeTextOnly(..., "你还没有录入个人餐食数据...");
3 }

```

- 避免空库仍走 LLM 意图识别 + 检索的无意义链路
- Trace 事件: `PERSONAL_LIBRARY_EMPTY`

4.2 检索层 userId 强制

`MealSearchService` :

```

1 if (request.sourceMode() == PERSONAL && request.userId() == null) {
2     throw new DietException("PERSONAL 模式必须提供 userId");
3 }

```

4.3 空结果分模式提示

`completeRecommendation` 中 ranked 为空时:

模式	提示文案
PERSONAL	建议补充饭堂菜或切换 PUBLIC

PUBLIC	建议补充槽位信息
--------	----------

5. 个人餐食 CRUD API

`MealController` 提供完整个人库管理：

方法	路径	说明
GET	<code>/api/v1/diet/meals/personal</code>	列表
POST	<code>/api/v1/diet/meals/personal</code>	新增
PUT	<code>/api/v1/diet/meals/personal/{mealId}</code>	编辑
DELETE	<code>/api/v1/diet/meals/personal/{mealId}</code>	删除
GET	<code>/api/v1/diet/meals/public</code>	公共库只读

写入时 `SlotOptionService.validate()` 校验 7 维标签合法性，与 Intent 抽取使用同一字典体系。

6. 推荐结果中的来源标识

`MealItem.sourceType` / `RecommendedMealOption.sourceType` / `MealResponse.sourceType` 贯穿检索→排序→卡片展示，前端可区分推荐来自个人库还是公共库。

`RecommendResponseAgent` prompt 中也传入 `sourceMode`，LLM 生成时可感知数据源上下文。

7. 设计价值

价值	说明
----	----

真实场景贴合	食堂/饭堂场景可落地，非纯虚构菜单
数据隔离	userId 级个人库，无跨用户泄露
体验闭环	空库引导录入 → CRUD 页维护 → 聊天推荐
灵活降级	个人库无匹配时可提示切 PUBLIC

8. 关键代码索引

类	说明
<code>SourceMode</code> 枚举	PERSONAL / PUBLIC
<code>MealMapper.xml#search</code>	数据源 SQL 分支
<code>MealService#hasPersonalMeals</code>	空库检查
<code>DietOrchestratorService#handleTurn</code>	空库前置返回
<code>MealController</code>	个人/公共 API
<code>SessionStateService</code>	sourceMode 持久化

07-混合推荐流水线

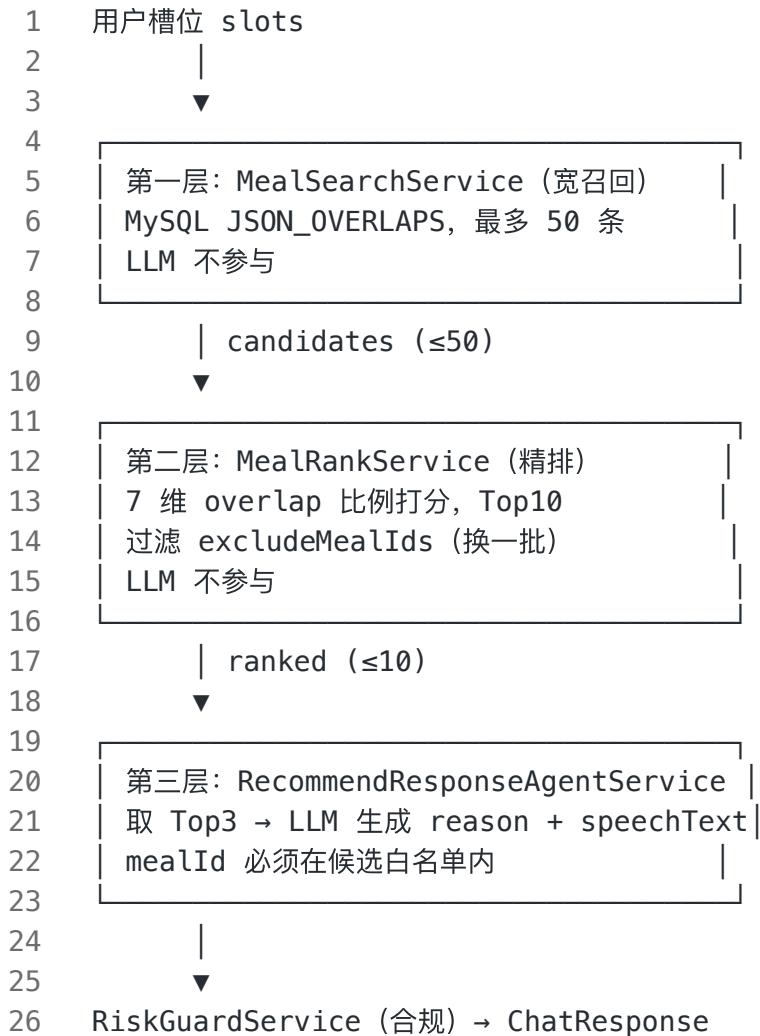
1. 为什么要分层

饮食推荐如果完全交给 LLM，常见问题是：

- **餐食幻觉**：模型编造不存在的菜名或 ID
- **成本不可控**：每轮都把全库菜单塞进 Prompt
- **结果不可解释**：无法说明「为什么召回这几道菜」
- **排序不稳定**：同一输入多次调用结果波动大

本项目在 `DietOrchestratorService#completeRecommendation` 中把推荐拆成 检索 → 重排 → 生成 三层，LLM 只负责最后一层的「解释与包装」。

2. 三层流水线架构



3. 各层职责与边界

3.1 第一层：MealSearchService

职责：按 `sourceMode + userId + slots` 从 DB 召回「至少有一个标签维度有交集」的候选。

- 调用 `MealService.search()` → `MealMapper.search`
- 上限 `SEARCH_LIMIT = 50`
- 设计取向：宁可多召回，不在 DB 层做精细排序

关键代码： `MealSearchService#search` 、 `MealService#search`

Trace 事件： `MEAL_SEARCHED` (candidateCount + candidates)

3.2 第二层：MealRankService

职责：在候选集内按「用户需求满足度」二次打分排序。

核心算法（slotScore）：

- 7 个槽位维度各自计算 `overlap = 命中标签数 / 用户该维标签数`
- 7 维求平均 → `matchScore ∈ [0, 1]`
- 按 `matchScore` 降序，取 Top10
- 过滤 `excludeMealIds`（MEAL_ADJUST 换一批场景）

与 Search 的区别（代码注释原文）：

search 只要槽位有交集就返回；rank 是比较用户槽位是否更大程度得到满足。

举例：用户 `healthGoal = [清淡, 高蛋白]`

- 鸡胸肉（`healthGoal` 是清淡 高蛋白 则命中 2/2）→ `score = 1.0`
- 猪肘（`healthGoal` 是高蛋白 则命中 1/2）→ `score = 0.5`

Trace 事件：`MEAL_RANKED`

3.3 第三层：RecommendResponseAgentService

职责：一次 LLM 调用同时产出：

- `recommendations[]`：每项 `{ mealId, reason }`
- `speechText`：面向用户的口语回复

防幻觉约束：

1. Prompt 明确要求「不要编造候选之外的餐食」
2. `parseOptions()` 只采纳 `topMeals` 白名单内的 `mealId`
3. `reason` 为空的不采纳；缺失 `reason` 用 `templateReason()` 兜底
4. LLM 失败时用 `templateOptions()` + `templateSpeech()` 全模板兜底

Trace 事件：`RECOMMEND_RESULT_BUILT` → `RESPONSE_AGENT_RESULT`

4. 空结果与异常路径

场景	处理
----	----

ranked 为空	按 sourceMode 返回不同空库提示，不走 LLM
Top3 为空	直接返回「暂时没有找到很匹配的餐食」
LLM 异常	模板理由 + 模板 speechText，仍返回 Top3 卡片
Guard 拦截	替换 speechText 为 conservativeMessage

5. 设计价值（简历/面试角度）

维度	收益
幻觉控制	餐食 ID 物理上不可能超出 Rank 候选集
成本	LLM 只看 Top3，而非全库
延迟	检索排序在 Java/DB 完成，LLM 只做一次生成
可解释	Search 召回量、Rank 分数、最终 Top3 全在 Trace 中
可评估	EvaluationService 用 rankedIds vs responselds 做 hallucinationControl

6. 关键代码索引

类	方法
<code>DietOrchestratorService</code>	<code>completeRecommendation()</code>
<code>MealSearchService</code>	<code>search()</code>
<code>MealRankService</code>	<code>rank()</code> , <code>slotScore()</code> , <code>overlap()</code>
<code>RecommendResponseAgentService</code>	<code>recommendAndRespond()</code> , <code>parseOptions()</code>

08-MySQL-JSON_OVERLAPS标签检索

1. 业务背景

餐食在 `meal_item` 表中不是扁平字段，而是 7 维 JSON 数组标签：

```
1 meal_time, mood, scene, health_goal, cuisine, taste, convenience
```

每道菜可以有多个标签（如「番茄鸡蛋面」同时标记午餐/晚餐/三餐、疲惫/低落、工作/校园...）。

用户查询也是 7 维 `SlotBundle`，需要找到「标签与用户偏好有交集」的餐食。传统 LIKE 或单字段匹配无法表达这种 **多标签、多维度、集合交集** 的检索需求。

2. 技术选型：JSON_OVERLAPS

MySQL 8.0 提供的 `JSON_OVERLAPS(doc, candidate)` 判断两个 JSON 文档是否存在交集。

对 JSON 数组而言，即 **两个数组是否有共同元素**。

本项目在 `MealMapper.xml#search` 中对 7 维逐一应用：

```
1 AND ({mealTimeJson} = '[]' OR JSON_OVERLAPS(meal_time, #{mealTimeJson}))
2 AND ({moodJson} = '[]' OR JSON_OVERLAPS(mood, #{moodJson}))
3 AND ({sceneJson} = '[]' OR JSON_OVERLAPS(scene, #{sceneJson}))
4 AND ({healthGoalJson} = '[]' OR JSON_OVERLAPS(health_goal, #{healthGoalJson}))
5 AND ({cuisineJson} = '[]' OR JSON_OVERLAPS(cuisine, #{cuisineJson}))
6 AND ({tasteJson} = '[]' OR JSON_OVERLAPS(taste, #{tasteJson}))
7 AND ({convenienceJson} = '[]' OR JSON_OVERLAPS(convenience, #{convenienceJson}))
```

3. 空维度短路设计

每个维度都有 `{xxxJson} = '[]' OR JSON_OVERLAPS(...)` 的前置条件：

- 用户该维槽位 为空 → 传 `'[]'` → 条件恒真，不参与过滤

- 用户该维槽位 **非空** → 必须与餐食对应 JSON 数组有交集

意义：用户只说「午餐 + 清淡」，不会因为缺少 mood/scene 等标签而漏召回；未提及的维度是「不限」而非「必须为空」。

Java 侧由 `JsonService.toJsonArray(slots.xxx())` 将 `List<String>` 转为 JSON 数组字符串传入 MyBatis。

4. 数据源隔离

同一 SQL 通过 `<choose>` 分支隔离 PERSONAL / PUBLIC：

sourceMode	WHERE 条件
PERSONAL	<code>source_type = 'PERSONAL' AND owner_user_id = #{userId}</code>
PUBLIC	<code>source_type = 'PUBLIC' AND owner_user_id IS NULL</code>

禁止混查：个人饭堂菜单与公共库不会在 SQL 层合并，由请求 `ChatRequest.sourceMode` 决定。

5. 召回策略：宽召回 + 限流

```
1 ORDER BY updated_at DESC
2 LIMIT #{limit}    -- SEARCH_LIMIT = 50
```

- 宽召回：7 维 AND 交集，任一用户指定维有交集即可（其他维为空则跳过）
- 上限 50：防止大库全表扫描式返回
- 初排仅按 `updated_at`：精细排序交给 Java `MealRankService`

这是典型的 DB 负责过滤 + Java 负责排序 分工。

6. 与 Rank 层的配合

阶段	语义	示例
Search	有没有相关标签	用户要[清淡,高蛋白], 餐食 health_goal 含「高蛋白」即命中
Rank	满足程度有多高	同上例, 含[清淡,高蛋白]得分 1.0, 仅含[高蛋白]得分 0.5

Search 解决「进不进候选池」, Rank 解决「谁排前面」。

7. 数据建模要点

meal_item 表结构 (节选) :

```
1 meal_time    JSON NOT NULL,
2 mood         JSON NOT NULL,
3 ...
4 source_type  VARCHAR(16) NOT NULL, -- PERSONAL / PUBLIC
5 owner_user_id BIGINT NULL,         -- PERSONAL 时必填
```

个人餐食 CRUD 时, SlotOptionService.validate() 保证写入的标签均在 diet_slot_option 字典内, 保证检索侧标签体系一致。

8. 关键代码索引

文件	说明
MealMapper.xml#search	JSON_OVERLAPS SQL
MealService#search	7 维 JSON 参数组装 + SEARCH_LIMIT
MealSearchService#search	sourceMode 校验
JsonService#toJSONArray	List → JSON 数组字符串

09-LLM全链路Fallback设计

1. 设计原则

LLM 在生产环境不可避免出现：

- 超时 / 限流
- JSON 格式错误或被 markdown 包裹
- 返回空文本
- 枚举值 / mealld 幻觉

本项目策略：主路径走 LLM，异常路径走 Java 规则/模板，保证 Orchestrator 状态机始终能推进并返回可用响应。

Trace 中通过 `fallbackRate` / `fallbackScore` 监控 fallback 发生频率。

2. 全链路 Fallback 地图



3. IntentAgent Fallback

3.1 触发条件

`IntentAgentService.recognize()` 整体 try-catch：

- Agent 调用异常
- JSON 解析失败
- intent 枚举非法

3.2 降级行为

```
1 private IntentResult fallback(String userInput) {
2     return new IntentResult(
3         fallbackIntent(userInput),
4         SlotBundle.empty(),
5         0.2 // 低置信度
6     );
7 }
```

3.3 关键词规则 fallbackIntent()

优先级链（节选）：

1. 空输入 → CLARIFY_NEEDED
2. 健康风险词 → HEALTH_RISK
3. 换一批/清淡点 → MEAL_ADJUST
4. 三餐/一周 → MEAL_PLAN
5. 你是谁 → OTHER
6. 吃什么/推荐 → MEAL_RECOMMENDATION
7. 默认 → CLARIFY_NEEDED

3.4 与 IntentRevise 联动

confidence=0.2 的 MEAL_RECOMMENDATION 会被 IntentRevise 降级为 CLARIFY_NEEDED（阈值 0.4），避免低质量识别直接进入推荐。

4. ClarifyAgent Fallback

4.1 主路径

ClarifyRuleService 判定 missingSlots 非空 → 调 ClarifyAgent 生成自然语言追问。

4.2 Fallback 触发

- LLM 异常（catch Exception）

- LLM 返回空文本 (question.isBlank())

4.3 降级行为

```
1 clarifyRuleService fallbackQuestion(missingSlots)
```

模板按 missingSlots 内容选择：

missingSlots	模板
含 mealTime	「这顿主要是早餐、午餐还是晚餐？」
含 healthGoal	「这顿更想清淡点、顶饱点，还是按口味来？」
空	默认确认口味/健康目标/方便快捷

关键：ASK/READY 决策始终由 Java 规则做出，LLM 只影响追问文案风格；fallback 不影响澄清逻辑正确性。

5. RecommendResponseAgent Fallback

5.1 触发条件

`recommendAndRespond()` catch Exception。

5.2 降级行为

```
1 RecommendResult recommend = new RecommendResult(templateOptions(topMeals, slots), needDisclaimer);
2 ResponseResult response = new ResponseResult(templateSpeech(recommend), toDisplayBlocks(recommend), "WAIT_USER");
```

- **templateReason**：按 healthGoal / taste 拼「比较符合你提到的 XX」
- **templateSpeech**：「我优先给你推荐这几款：\n- 名称：理由」+ 可选免责声明

仍返回 Top3 餐食卡片，用户感知是「回复文案变模板化」，而非系统报错。

5.3 部分 Fallback（非异常）

`parseOptions` 中 LLM 漏写 reason → 单条 `templateReason` 补全; `speechText` 为空 → `templateSpeech` 。

6. EvaluationJudge Fallback

```
1 catch (Exception error) {
2     log.warn("Evaluation Judge failed: traceId={}", traceId, error);
3     return null;
4 }
```

Judge 失败时:

- `llmJudgeScore = null`
- 总分按 **实际存在权重归一**（规则 60% + 反馈 30% 仍可用）
- 不阻塞整批评估任务

7. Trace 层 Fallback 标记

`EvaluationService.parseTrace()` :

```
1 boolean fallbackUsed = "FAILED".equalsIgnoreCase(row.getStatus());
2 // 任意 event 含 errorMessage 或 REQUEST_FAILED → fallbackUsed = true
```

指标:

- `fallbackRate` : 单条 0/1, 区间平均 = fallback 比例
- `fallbackScore` : 未 fallback = 1.0

Also: `AgentTraceService` 中 Agent 调用失败会 `markFailed` , 整轮 trace status=FAILED。

8. Fallback 设计矩阵

环节	能否继续主链路	用户可见结果	数据完整性
Intent	能	可能进澄清/固定分支	slots 可能为空

Clarify	能	模板追问	missingSlots 正确
Recommend	能	模板文案 + 真实卡片	mealId 来自 Rank
Judge	能（评估侧）	无（后台）	规则分仍可用

9. 面试要点：Fallback vs 直接报错

直接抛错	本项目 Fallback
用户体验中断	始终有回复
难统计 LLM 不稳定	Trace fallbackRate 可监控
澄清/推荐逻辑依赖 LLM	决策在 Java, LLM 只锦上添花

10. 关键代码索引

类	Fallback 入口
<code>IntentAgentService</code>	<code>fallback()</code> , <code>fallbackIntent()</code>
<code>ClarifyAgentService</code>	catch + <code>fallbackQuestion()</code>
<code>ClarifyRuleService</code>	<code>fallbackQuestion()</code>
<code>RecommendResponseAgentService</code>	catch + <code>templateOptions/templateSpeech/templateReason</code>
<code>EvaluationJudgeService</code>	catch → return null

10-Trace与可监控设计

1. 设计目标

为每一轮用户对话提供 全链路可观测能力，支撑：

- 线上问题排查（哪一步出错、LLM 输入输出是什么）
- 人工标注 gold label（expected_intent / expected_slots / expected_clarify_action）
- 离线批量评估（EvaluationService 解析 trace_json 计算指标）

2. 核心组件

2.1 AgentTraceService

- TraceScope (ThreadLocal) : `openTrace(traceId, sessionId, userId)` 开启, `close()` 时 flush
- try-with-resources: Orchestrator 整轮请求包裹在 TraceScope 内，保证异常时也能落库
- 统一 Agent 调用入口: `callAgent()` 自动记录 AGENT_CALL 事件

2.2 存储

表: `diet_request_trace`

字段	说明
trace_id	主键，格式 <code>trace_<32hex></code>
session_id / user_id	关联会话与用户
status	SUCCESS / FAILED
event_count	事件总数
duration_ms	整轮耗时
error_message	失败摘要

trace_json	完整事件数组 JSON
expected_intent / expected_slots / expected_clarify_action	人工标注 gold
labeled_by / labeled_at / label_note	标注元信息

消息表 `diet_messages.agent_trace_id` 关联每轮对话的 `traceId`。

3. Trace 事件模型

每条 `TraceEvent` 包含：

```
1  stepOrder, eventType, phase, agentName, modelName,  
2  inputPayload, outputPayload, latencyMs,  
3  inputTokens, outputTokens, totalTokens,  
4  errorMessage, createdAt
```

3.1 状态机事件（recordEvent）

Orchestrator 在每个关键节点记录 input/output：

- 请求级：REQUEST_RECEIVED / REQUEST_FINISHED / REQUEST_FAILED
- 意图：INTENT_RECOGNIZED → INTENT_REVISIED → ROUTE_SELECTED
- 槽位：SLOTS_MERGED
- 澄清：CLARIFY_DECISION
- 检索排序：MEAL_SEARCHED → MEAL_RANKED
- 推荐：RECOMMEND_RESULT_BUILT / NO_MEAL_MATCHED
- 应答：RESPONSE_AGENT_RESULT → RESPONSE_READY
- 合规：NUTRITION_GUARD_CHECKED / NUTRITION_GUARD_REWRITTEN

3.2 Agent 调用事件（callAgent → AGENT_CALL）

每次 LLM 调用记录：

- agentName：IntentAgent / ClarifyAgent / RecommendResponseAgent / EvaluationJudgeAgent

- modelName: qwen-turbo 或 qwen-max
- input/output 文本
- latencyMs
- token 用量 (inputTokens / outputTokens / totalTokens)

4. 可靠性设计

机制	说明
payload 截断	单条 MAX 20000 字符，超出追加 <code>...[truncated]</code>
落库失败不阻塞	flush 异常只打 warn，不影响主业务返回
ThreadLocal 清理	TraceScope.close 时 <code>currentScope.remove()</code> ，防线程池污染
未 openTrace 静默跳过	record 时 scope 为 null 则不记录（兼容单测）
异常 markFailed	任意事件带 errorMessage → status=FAILED

5. 查询与标注 API

Controller: `AgentTraceController` (`/api/v1/diet/debug`)

接口	用途
GET <code>/traces/{traceId}</code>	单条 Trace 详情
GET <code>/sessions/{sessionId}/traces?limit=</code>	按会话查最近 N 条
GET <code>/traces?startAt&endAt&onlyUnlabeled&limit</code>	时间范围批量查询（评估/标注页）
PUT <code>/traces/{traceId}/label</code>	写入人工标注 gold label

标注请求体 `TraceLabelRequest` :

- `expectedIntent`
- `expectedSlots` (`SlotBundle`)
- `expectedClarifyAction` (`ASK / READY`)
- `labelNote`

6. 监控维度（从 Trace 可直接观测）

维度	数据来源
端到端延迟	<code>duration_ms</code>
LLM 调用次数与耗时	<code>AGENT_CALL</code> 事件
Token 成本	<code>AGENT_CALL.totalTokens</code> 累加
意图识别质量	<code>INTENT_RECOGNIZED</code> vs <code>INTENT_REVISIED</code> 对比
澄清决策	<code>CLARIFY_DECISION.action</code>
检索召回量	<code>MEAL_SEARCHED.candidateCount</code>
排序结果	<code>MEAL_RANKED.ranked</code>
合规拦截	<code>NUTRITION_GUARD_*</code> 事件
失败/fallback	<code>status=FAILED</code> 或 <code>errorMessage</code> 非空

7. 与评估系统的衔接

`EvaluationService.parseTrace()` 从 `trace_json.events` 提取:

- 最终 intent (`INTENT_REVISIED`)
- 最终 slots (`SLOTS_MERGED` 优先)
- `clarifyAction` (`CLARIFY_DECISION`)
- `tokenCost` (`AGENT_CALL` 累加)

- fallbackUsed (FAILED 或 errorMessage)
- rankedIds vs responselds (幻觉检测)
- excludedIds vs responselds (多轮一致性)

Trace 是整个「可监控 → 可标注 → 可评估」闭环的数据基础。

8. 关键代码

- `AgentTraceService` : Trace 生命周期 + callAgent 包装
- `AgentTraceController` : 查询与标注 API
- `AgentTraceMapper` + `AgentTraceMapper.xml` : 持久化
- `DietOrchestratorService#dietChat` : openTrace 入口

11-评估设计（可评估性、指标与方式）

1. 评估体系概览

本项目构建了「Trace 驱动 + 规则评分 + LLM-as-Judge + 用户反馈」四层评估体系：



API: `POST /api/v1/diet/evaluations`

2. 评估方式

2.1 批量离线评估

请求体 `EvaluationRequest`：

- startAt / endAt: 评估时间窗口
- includeLlmJudge: 是否启用 LLM-as-Judge（默认可选）
- limit: 最大 trace 条数（默认 1000）

流程：

1. 按时间范围从 `diet_request_trace` 拉取 trace
2. 批量加载同 session 的 `recommend_feedback`
3. 逐条 parseTrace → 计算指标 → 加权总分
4. 输出 `EvaluationReport`（区间平均 + 单条 `TraceEvaluationResult` 明细）

2.2 人工标注 (Gold Label)

通过 Trace 标注页/API 写入：

- expected_intent
- expected_slots
- expected_clarify_action

有标注的 trace 才参与 intentAccuracy / slotAccuracy / clarifyNecessityAccuracy 计算；无标注返回 null，不影响其他指标。

2.3 LLM-as-Judge

`EvaluationJudgeService` 调用 `EvaluationJudgeAgent` (qwen-turbo) ，输入 trace 摘要：

- predictedIntent / predictedSlots / predictedClarifyAction
- finalReply / recommendationCount
- safetyComplianceByRule / hallucinationFreeByRule (背景信息, Judge 不重复评)

输出 1-5 分：

- explanationQuality: 解释质量
- naturalness: 自然度

Judge 失败返回 null，不影响规则分计算。

2.4 用户反馈

`recommend_feedback` 表记录用户对推荐的操作：

- rating (1-5 星) → 归一化到 0-1
- action (LIKE/DISLIKE/SWITCH 等) → 映射满意度分

当前按 `sessionId` 归因 (反馈表无 `traceId`, 同 session 反馈共享) 。

3. 评估指标

3.1 规则指标 (0-1 分制)

指标	计算方式	依赖
intentAccuracy	expected vs predicted 完全一致	人工标注 expected_intent
slotAccuracy	7 维槽位集合匹配比例	人工标注 expected_slots
clarifyNecessityAccuracy	ASK/READY 完全一致	人工标注 expected_clarify_action
tokenCost	AGENT_CALL totalTokens 累加（原始值）	trace 自动
tokenCostScore	$\leq 1000 \rightarrow 1.0$; $\geq 3000 \rightarrow 0$; 中间线性衰减	trace 自动
latencyMs	duration_ms（原始值）	trace 自动
latencyScore	$\leq 3s \rightarrow 1.0$; $\geq 8s \rightarrow 0$; 中间线性衰减	trace 自动
fallbackRate	FAILED 或 errorMessage $\rightarrow 1.0$	trace 自动
fallbackScore	未 fallback $\rightarrow 1.0$	trace 自动
safetyCompliance	最终回复不含禁用短语	trace 自动
hallucinationControl	responselds $\subseteq$ rankedIds	trace 自动
multiTurnConsistency	MEAL_ADJUST 时新推荐未命中 excludelds	trace 自动

禁用短语：治好、治疗、保证、一定能瘦、根治、替代医生

3.2 LLM Judge 指标（1–5 分 $\rightarrow$ 归一化 0–1）

指标	说明
explanationQuality	是否说明推荐理由，理由是否贴合需求
naturalness	表达是否自然，非机械拼接

3.3 用户反馈指标

指标	说明
userFeedbackScore	session 下多条反馈的平均满意度

4. 总分加权

单条 trace 总分（百分制）：

组成部分	权重	来源
ruleScore	60%	上述规则指标平均（跳过 null）
llmJudgeScore	10%	explanationQuality + naturalness 归一化平均
userFeedbackScore	30%	用户反馈平均

若某组成部分缺失（如无标注、无反馈、未开 Judge），按 实际存在权重归一，避免缺失维度拉低总分。

5. 评估报告结构（EvaluationReport）

1	startAt, endAt	
2	traceCount	- 参与评估 trace 数
3	labeledCount	- 至少有一个 gold 标注的 trace 数
4	averageScore	- 区间平均总分（百分制）
5	metricAverages	- 各指标区间平均
6	results[]	- 单条 TraceEvaluationResult 明细
7	— traceId, sessionId, createdAt	
8	— score, ruleScore, llmJudgeScore, userFeedbackScore	
9	— metrics（全部指标明细）	
10	— detail (predicted vs expected + judgeReason)	

6. Trace 解析关键事件映射

EvaluationService 从 trace_json 提取事实：

事件	提取字段
INTENT_REVISED	intent, slots
SLOTS_MERGED	slots（优先）
CLARIFY_DECISION	clarifyAction
ADJUST_CONTEXT_RESOLVED	excludeMealIds, intent=MEAL_ADJUST
MEAL_RANKED	ranked[].id
RESPONSE_AGENT_RESULT / RESPONSE_READY	speechText, displayBlocks[].id
AGENT_CALL	totalTokens 累加

7. 设计亮点

- 1. 规则与 Judge 职责分离：安全/幻觉/意图/槽位走规则，解释质量和自然度走 LLM Judge，避免 Judge 重复评已覆盖维度
- 2. 缺失标注不惩罚：无 gold label 的指标返回 null，不参与平均，适合渐进式标注
- 3. Trace 即评估数据源：无需额外埋点，评估完全复用线上 Trace
- 4. 可开关 Judge：`includeLlmJudge=false` 时纯规则+反馈评估，降低成本

8. 关键代码

- `EvaluationService`：批量评估主逻辑
- `EvaluationJudgeService` + `EvaluationJudgeAgentBuilder`：LLM Judge
- `EvaluationController`：评估 API
- Prompt: `diet/prompts/evaluation-judge.txt`
- `AgentTraceService.updateLabel()`：标注写入

12-可评估闭环 (Trace→标注→评估→迭代)

1. 为什么 Agent 项目需要评估闭环

很多 Agent Demo 止步于「能对话」，缺少：

- 线上行为是否可复现、可对比
- 意图/槽位识别准确率多少
- LLM 升级或 Prompt 修改后是变好了还是变差
- 用户真实满意度如何

本项目从架构设计之初把 Trace 作为单一事实来源 (Single Source of Truth)，打通观测、标注、评估、优化四步闭环。

2. 闭环总览



3. 第一步：线上 Trace 自动采集

3.1 采集范围

- 状态机每一步：意图、路由、槽位合并、澄清、检索、排序、推荐、Guard
- 每次 LLM 调用：input/output、latency、token
- 整轮：duration_ms、status、error_message

3.2 与消息关联

`diet_messages.agent_trace_id` = 本轮 traceId，对话记录与 Trace 可交叉定位。

3.3 设计特点

- try-with-resources 保证 close 时 flush
- 落库失败不阻塞用户响应
- payload 截断防 JSON 过大

无 Trace 就无后续评估——这是闭环的数据基础。

4. 第二步：观测与排查

API (`/api/v1/diet/debug`) :

能力	用途
按 traceId 查详情	单用户投诉复盘
按 sessionId 查历史	多轮对话完整还原
按时间范围查	批量评估采样、标注任务分发
onlyUnlabeled	只拉未标注 trace，提高标注效率

前端 Trace 页展示 trace_json.events 时间线，研发/算法可直接看 INTENT_REVISIED vs INTENT_RECOGNIZED 差异。

5. 第三步：人工标注 Gold Label

写入 `diet_request_trace` 字段：

字段	含义
expected_intent	标准意图
expected_slots	标准槽位 JSON
expected_clarify_action	ASK / READY
label_note	标注备注

标注原则（系统设计隐含）：

- 只标注有把握的维度；未标注的维度在评估中为 null，不参与该项平均
- 支持 partial label（只标 intent 不标 slots）

这是「人工 gold + 自动 predicted」经典 NLU 评估范式在 Agent 场景的落地。

6. 第四步：批量评估

`EvaluationService.evaluate(userId, EvaluationRequest)`：

输入：时间窗口 + 是否 includeLlmJudge + limit

处理：

- 拉 trace 列表
- 按 session 批量拉 feedback
- 每条 parseTrace → 11+ 规则指标
- 可选 EvaluationJudgeAgent → explanationQuality / naturalness
- 加权总分（规则 60% + Judge 10% + 反馈 30%）

输出： `EvaluationReport`

- 区间 traceCount / labeledCount
- averageScore + metricAverages
- 单条 TraceEvaluationResult 明细 (含 predicted vs expected)

7. 第五步：指标驱动迭代

7.1 可指导优化的指标

指标偏低	可能优化方向
intentAccuracy	intent.txt Prompt、IntentRevise 规则、fallback 关键词
slotAccuracy	slotOptions 字典、SlotJsonPicker、Intent Prompt 槽位说明
clarifyNecessityAccuracy	ClarifyRuleService 必填规则
hallucinationControl	RecommendResponse 白名单、Rank TopN
safetyCompliance	RiskGuard 关键词、Intent 健康拦截
fallbackRate	模型稳定性、JSON 解析、超时配置
tokenCostScore / latencyScore	轻量模型分流、减少 Agent 调用次数
explanationQuality / naturalness	recommend-response.txt Prompt
userFeedbackScore	推荐质量、排序算法

7.2 版本对比流程建议

1. 改 Prompt → 升 `diet.prompt.version`
2. 跑固定时间窗口 EvaluationReport
3. 对比 labeled trace 子集上的 intentAccuracy / slotAccuracy
4. 回归通过再全量

8. 用户反馈的补充作用

`recommend_feedback` 表：

- rating (1–5)
- action (LIKE / DISLIKE / SWITCH...)

当前按 `sessionId` 归因（无 `traceId` 字段），评估时同 session 反馈共享到该 session 下所有 trace——这是已知近似，面试可主动说明改进为 `traceId` 级反馈。

反馈占总分 **30% 权重**，连接线上真实满意度与离线指标。

9. 与「纯 Demo」项目的差异

纯 Demo	本项目
只有 chat API	chat + trace + label + evaluation
好坏靠主观感受	11+ 量化指标 + 百分制总分
改 Prompt 心里没底	固定窗口对比 EvaluationReport
LLM 黑盒	trace_json 逐步可回放

10. 关键代码与表

类型	名称
表	diet_request_trace, recommend_feedback
服务	AgentTraceService, EvaluationService, EvaluationJudgeService
API	AgentTraceController, EvaluationController
前端	#/admin/traces, #/admin/evaluations

13-RiskGuard健康合规守卫

1. 为什么需要输出层守卫

饮食 Agent 面向普通用户，LLM 生成内容存在两类风险：

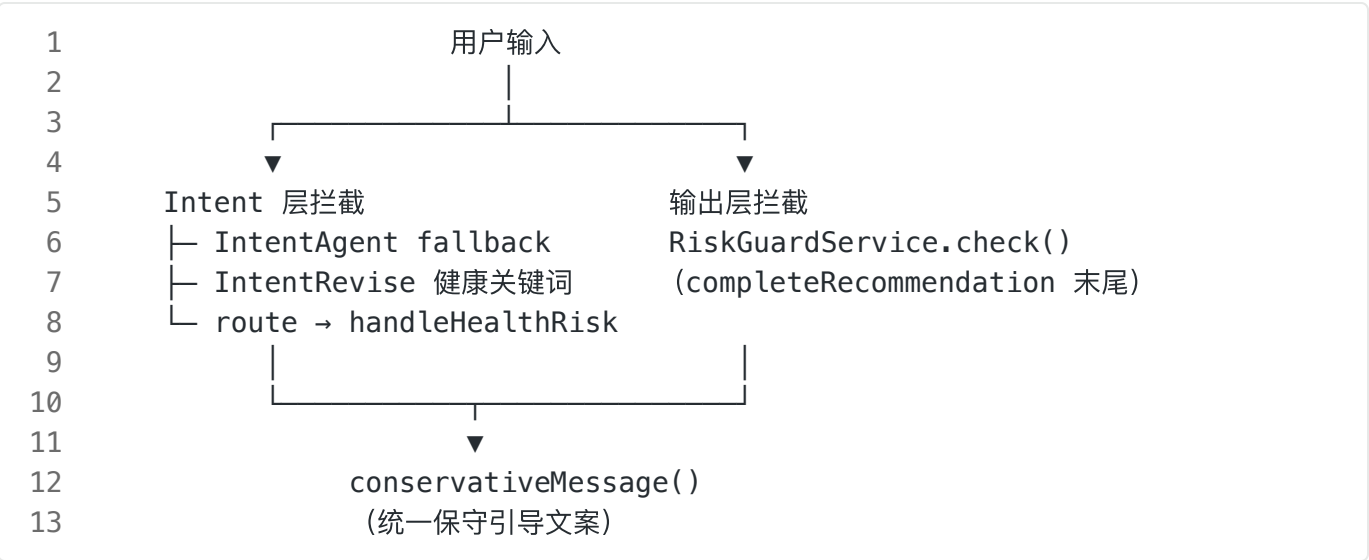
- 1. 用户主动问高风险问题：「胃疼吃什么能治好」「糖尿病能不能随便吃甜的」
- 2. LLM 回复越界：给出治疗承诺、极端节食建议、绝对化减肥表述

Intent 层虽能识别 `HEALTH_RISK` 意图，但无法保证：

- LLM 在 `MEAL_RECOMMENDATION` 场景下仍可能输出「保证能瘦」
- 用户输入安全但回复不安全
- IntentAgent 漏判

因此推荐链路末尾增加 `RiskGuardService` 做输出层 Java 规则拦截。

2. 双保险架构



层级	时机	类
意图拦截	路由前	<code>IntentAgentService</code> , <code>IntentReviseService</code>

路由拦截	HEALTH_RISK 分支	<code>DietOrchestratorService#handleHealthRisk</code>
输出拦截	LLM 回复生成后	<code>RiskGuardService</code>

3. RiskGuardService 五条规则

`check(userInput, intent, recommendResult, responseResult)` 拼接 用户原文 + 助手 speechText 统一扫描：

规则	关键词/条件	reasons 标记
1. 意图层	<code>intent == HEALTH_RISK</code>	命中 HEALTH_RISK 意图
2. 医疗承诺	治好、治疗、诊断、药、处方	涉及医疗诊断或治疗承诺
3. 极端节食	绝食、一天不吃、只喝水、极端节食	涉及极端节食建议
4. 绝对化承诺	保证、一定能瘦、最健康、包瘦	涉及绝对化健康承诺
5. 特殊人群/慢病	孕妇、糖尿病、高血压、未成年人、儿童	涉及特殊人群或慢病风险

- 全部通过 → `RiskGuardResult.pass()`
- 任一命中 → `RiskGuardResult.block(reasons, conservativeMessage())`

4. Orchestrator 中的处置流程

```
1 // DietOrchestratorService#completeRecommendation
2 RiskGuardResult guard = riskGuardService.check(...);
3 if (!guard.passed()) {
4     response = ResponseResult.textOnly(guard.rewriteSuggestion());
5     // Trace: NUTRITION_GUARD_REWRITTEN
6 } else {
7     // Trace: COMPLIANCE_GUARD_REWRITTEN (历史命名, 语义=通过)
8 }
```

关键行为：

- 整段替换：不是局部删改，而是用 `conservativeMessage()` 完全替换 `speechText`
- 丢弃原 LLM 回复：推荐卡片 `displayBlocks` 仍保留（代码未清空卡片），但口语回复变为保守引导
- Trace 可观测：GUARD 阶段事件记录 `passed/reasons`

保守文案（固定）：

这个问题涉及健康或医疗风险，我不能替代医生做诊断或治疗建议。可以从日常饮食角度选择清淡、均衡、不过量的餐食；如果症状明显或有慢病、孕期等情况，建议咨询医生或营养师。

5. 与 RecommendResponseAgent 免责声明的配合

RecommendResponseAgent 在槽位含「减脂/低糖/控碳水/养胃」时会设

`needDisclaimer=true`，模板 `speech` 追加：

这些建议只做日常饮食参考，如果有明确疾病或特殊身体情况，建议咨询医生或营养师。

这是 Prompt 层软约束；RiskGuard 是 规则层硬拦截。两者互补：

- 免责声明：正常推荐场景的预防性提示
- RiskGuard：高风险内容强制阻断

6. 评估体系中的体现

EvaluationService 规则指标 `safetyCompliance`：

- 扫描最终回复是否含禁用短语：治好、治疗、保证、一定能瘦、根治、替代医生
- 与 RiskGuard 规则部分重叠，用于离线批量评估

Trace 事件 `NUTRITION_GUARD_CHECKED` / `NUTRITION_GUARD_REWRITTEN` 可还原线上是否发生过拦截。

7. 设计取舍

为什么用 Java 规则而非 LLM 审查：

- 确定性：关键词命中即拦截，无 LLM 二次波动
- 零延迟：不增加额外模型调用
- 可审计：reasons 列表明确可追溯

可改进方向（面试加分项）：

- 接入专业内容安全 API

8. 关键代码索引

类	方法
RiskGuardService	check() , conservativeMessage()
IntentReviseService	containsHealthRiskKeyword()
IntentAgentService	fallbackIntent() 健康风险分支
DietOrchestratorService	handleHealthRisk() , completeRecommendation() Guard 段

14-短期记忆设计

1. 设计定位

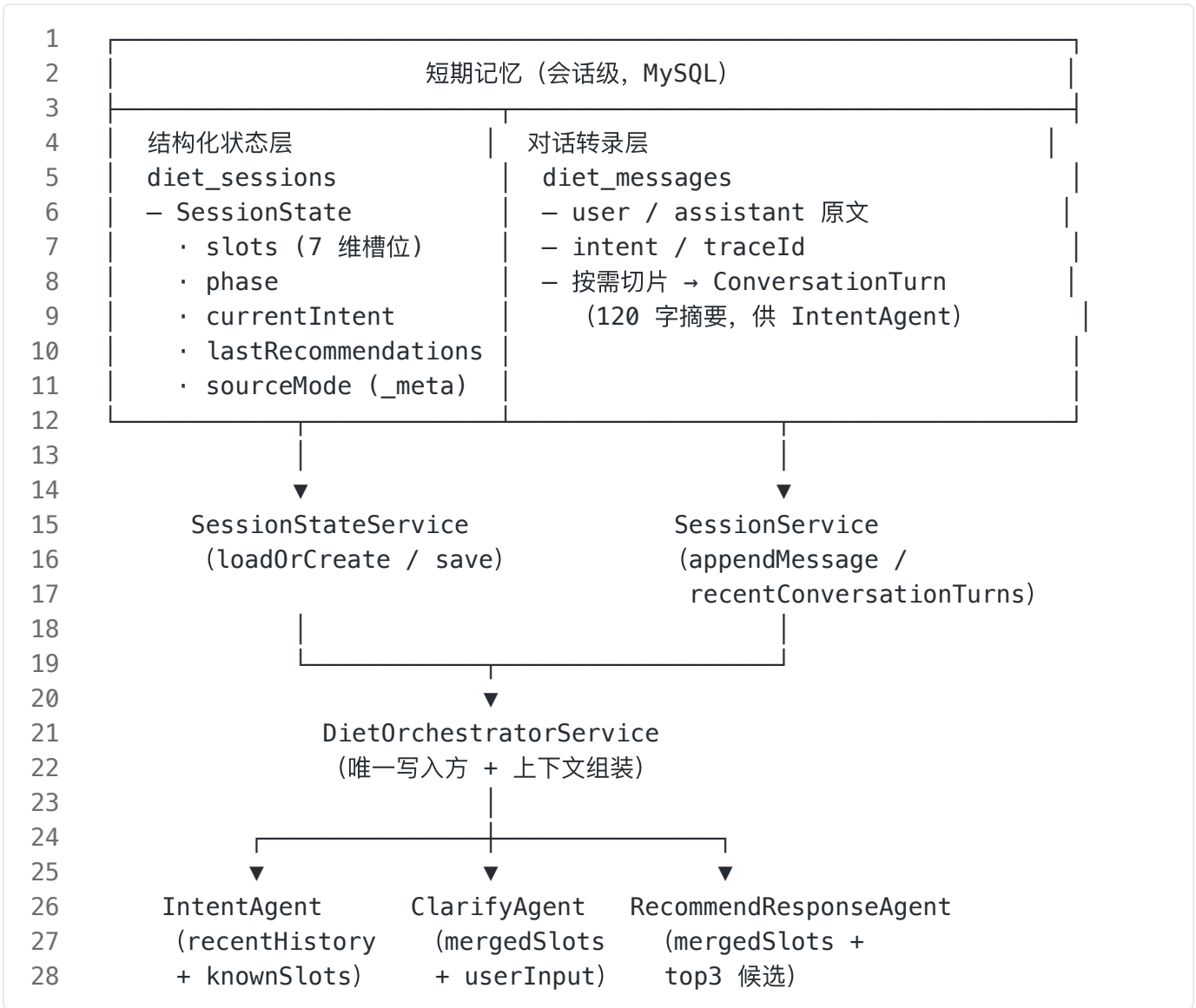
本项目的「短期记忆」不是 AgentScope 框架里的 `Memory`，也不是某个独立的 `ShortTermMemory` 类，而是一套由 `Orchestrator` 统一管理、MySQL 持久化、按轮按需注入 LLM 的会话上下文体系。

核心目标：

目标	实现手段
多轮槽位累积	<code>SessionState.slots</code> 增量合并 + DB 持久化
识别「换一批」等指代	<code>diet_messages</code> 最近对话摘要 + <code>lastRecommendations</code>
澄清/推荐阶段连贯	<code>phase</code> / <code>currentIntent</code> 状态机
可审计、可 Trace	消息原文落库， <code>agent_trace_id</code> 关联
避免 LLM 隐式记忆漂移	Agent 每轮 <code>memory.clear()</code> ，上下文显式拼进 prompt

一句话概括：权威状态在 DB + Orchestrator，LLM 只读本轮拼好的结构化快照。

2. 记忆分层架构



2.1 结构化状态层 (SessionState)

对应 `diet_sessions` 表, 由 `SessionStateService` 读写, `DietOrchestratorService` 独占写入。

```

1  SessionState {
2      sessionId, userId,
3      phase,           // START | CLARIFY | RECOMMEND | PLAN
4      sourceMode,      // PERSONAL | PUBLIC
5      currentIntent,
6      slots,           // SlotBundle 7 维累积槽位
7      lastRecommendations // 已推荐 mealId 列表 (累积去重)
8  }
  
```

记忆语义：

- `slots`：跨轮累积的用户偏好（餐次、场景、口味等），本轮 IntentAgent 只输出增量，由 `SlotMergeService` 与历史合并
- `lastRecommendations`：本会话已展示过的餐食 ID，供「换一批」排除重复
- `phase` / `currentIntent`：告诉 Orchestrator 当前会话卡在哪个阶段，辅助路由与规则矫正
- `sourceMode`：持久化在 `slots._meta`，保证多轮始终在同一数据源（PERSONAL / PUBLIC）内检索

详见 [05-多轮对话状态管理.md](#)。

2.2 对话转录层（ConversationTurn）

对应 `diet_messages` 表，由 `SessionService` 追加与查询。

每轮对话结束时写入两条（或一条）消息：

时机	role	intent	说明
收到用户输入后	user	null	用户原文
返回助手回复后	assistant	最终 intent	追问 / 推荐文案 / 固定引导

读取时通过 `recentConversationTurns(sessionId, userId, n)` 转为 `ConversationTurn` `n`：

```
1 ConversationTurn {
2     role,          // user | assistant
3     intent,        // assistant 轮次携带, user 轮次可为 null
4     summary,       // content 截断至 120 字符 (省 token)
5     timestamp
6 }
```

记忆语义：

- 完整原文永久留在 `diet_messages`（审计、评估、Trace 交叉定位）
- 注入 LLM 时只取最近 `n` 条（Orchestrator 固定传 `n=3`），且每条最多 120 字
- 目前仅 IntentAgent 消费 `recentHistory`；ClarifyAgent / RecommendResponseAgent 不读对话历史，依赖合并后的 `slots`

2.3 瞬态 Prompt 上下文

每轮 Agent 调用前，Orchestrator 或各 `AgentService` 将上述两层记忆显式序列化进 user prompt，而非依赖 Agent 内部记忆自动累积。

Agent	注入的短期记忆字段
IntentAgent	<code>recentHistory</code> + <code>knownSlots</code> + <code>slotOptions</code> + 当前句
ClarifyAgent	<code>mergedSlots</code> + <code>missingSlots</code> + 用户原话
RecommendResponseAgent	<code>mergedSlots</code> + <code>sourceMode</code> + top3 候选 + 用户原话

3. 一轮对话中的读写时序

以「工作很忙推荐快速的 → 午餐 → 换一批」为例：

```
1  轮次 1  用户：「工作很忙，推荐快速的」
2    ① appendMessage(user)
3    ② recentConversationTurns(3) → []           // 尚无历史
4    ③ IntentAgent(knownSlots=空, recentHistory=[])
5    ④ merge slots → scene=工作, convenience=快速
6    ⑤ save SessionState + appendMessage(assistant, 推荐结果)
7    ⑥ lastRecommendations += [id1, id2, id3]
8
9  轮次 2  用户：「午餐」
10   ① appendMessage(user)                       // 此时 recent 含轮次 1 的 us
      er+assistant
11   ② recentConversationTurns(3) → 2 条摘要
12   ③ IntentAgent(knownSlots=上轮 slots, recentHistory=2条)
13   ④ merge → mealTime=午餐, 保留 scene/convenience
14   ⑤ ...
15
16  轮次 3  用户：「换一批」
17   ① appendMessage(user)
18   ② recentConversationTurns(3) → 最近 3 条 (含 assistant 推荐 intent)
19   ③ IntentAgent 结合 recentHistory 识别 MEAL_ADJUST
20   ④ IntentRevise 校验 lastRecommendations 非空
21   ⑤ handleAdjust → excludeMealIds = lastRecommendations
22   ⑥ 新推荐写入 lastRecommendations (累积追加)
```

关键时序细节：用户消息在 IntentAgent 调用之前落库，因此 `recentConversationTurns` 读到的是不含当前句的历史；当前句通过 prompt 的「当前这一句」字段单独传入。这样避免重复，也保证 IntentAgent 对「当前输入」与「历史上下文」分工明确。

4. 各记忆字段的消费方

记忆字段	写入方	读取方	用途
<code>slots</code>	Orchestrator (merge 后 save)	IntentAgent (knownSlots)、ClarifyAgent、RecommendResponseAgent、检索层	多轮偏好累积
<code>lastRecommendations</code>	Orchestrator (推荐成功后 append)	IntentReviseService、handleAdjust → MealRankService	「换一批」去重
<code>phase</code>	Orchestrator	排查、评估；间接影响澄清/推荐分支	会话阶段
<code>currentIntent</code>	Orchestrator	落库 assistant 消息的 intent 字段	历史 intent 标注
<code>sourceMode</code>	首屏绑定后持久化	检索数据源隔离	PERSONAL/PUBLIC 不混查
<code>diet_messages</code> 原文	SessionService.appendMessage	recentConversationTurns → IntentAgent	指代、调整类意图
<code>ConversationTurn.summary</code>	SessionService.summarize()	IntentAgent prompt	压缩 token 的对话摘要

不读短期记忆的组件：ClarifyRuleService、MealSearchService、MealRankService、RiskGuard 等 Java 规则/检索层只消费 `SlotBundle` 和业务 ID，不直接访问消息表。

5. 配置与边界

5.1 配置项

```
1 # application.yml
2 diet:
3   session:
4     max-history-turns: 10 # recentConversationTurns 上限 (Orchestrator 实际请求 n=3)
```

- Orchestrator 硬编码 `recentConversationTurns(..., 3)`，与 IntentAgent prompt 规则（「最近 3 轮 assistant 已推荐」）对齐
- `max-history-turns` 是 `SessionService` 的安全上限：`limit = min(n, maxHistoryTurns)`

5.2 摘要与容量边界

边界	值	原因
注入 LLM 的对话条数	3 条消息	覆盖「上一轮推荐 + 当前调整」足够；控制 token
单条摘要长度	120 字符	推荐卡片 JSON 较长，截断后仍保留关键语义
lastRecommendations	会话内累积	多轮「换一批」持续排除；无 TTL，随会话结束
会话状态 TTL	无（DB 持久）	前端持 sessionId 即可恢复；无 Redis 层

5.3 并发与一致性

同 `sessionId` 请求经 `sessionLocks` 串行化，保证：

- 两轮同时 merge slots 不会互相覆盖
- `lastRecommendations` 追加不会丢 ID
- `recentConversationTurns` 与 `appendMessage` 顺序一致

6. 与 Agent Memory (InMemoryMemory) 的对比

本项目每个 ReActAgent 构建时仍挂载 AgentScope 的 `InMemoryMemory()`，但业务多轮记忆完全不依赖它。

6.1 对照表

维度	系统短期记忆	Agent Memory (InMemoryMemory)
存储位置	MySQL (<code>diet_sessions</code> + <code>diet_messages</code>)	JVM 堆内，Agent 实例私有
生命周期	随 sessionId 持久，跨请求/重启可恢复	随 Agent 实例；Factory LRU 淘汰后丢失
写入方	Orchestrator / SessionService (Java 显式)	AgentScope 框架在 <code>agent.call()</code> 内自动追加 Msg
读取方式	各 Service 拼进 user prompt 结构化字段	框架自动注入下一轮 LLM 上下文
数据结构	结构化：SlotBundle、Intent、mealId 列表、摘要	非结构化：Msg 列表（原始对话）
多 Agent 共享	是，同一 SessionState 供所有 Worker 只读	否，每个 Agent 独立 Memory
Trace 可见性	完整：DB 原文 + Trace 事件	仅当轮 call 内部；clear 后不保留
每轮策略	load → 业务 → save；按需切片注入	<code>memory.clear()</code> 清空，不累积
设计意图	权威业务状态，可审计、可评估	满足 AgentScope API 要求的占位容器

6.2 为何不用 Agent Memory 做多轮

1. 单一权威源： `SessionState` 已是 ground truth；若 Agent Memory 也累积，会出现双写不一致（槽位被 LLM 改写、推荐 ID 无法结构化存储）。

2. **结构化优先**：槽位、排除列表、phase 必须 Java 可校验；Msg 列表无法直接驱动检索与规则。
3. **可观测**：Orchestrator 拼 prompt 的内容完整进入 `AGENT_CALL` Trace；隐式 Memory 难以对齐业务字段。
4. **会话隔离**：`AgentFactory` 按 sessionId 缓存 AgentSet 是为避免不同用户串话；同会话内仍每轮 clear，不靠 Memory 续聊。
5. **AgentScope 轻量用法**：本项目把 ReActAgent 当「LLM 网关 + JSON 生成器」，见 [00-AgentScope与SpringBoot集成.md §5](#)。

6.3 代码中的分界线

系统短期记忆（业务层）：

```
1 // 加载权威状态
2 SessionState state = sessionStateService.loadOrCreate(sessionId, userId, sourceMode);
3
4 // 追加对话 + 读取摘要
5 sessionService.appendMessage(sessionId, "user", request.message(), null, traceId);
6 List<ConversationTurn> recent = sessionService.recentConversationTurns(sessionId, userId, 3);
7
8 // 注入 IntentAgent
9 intentAgentService.recognize(sessionId, userId, request.message(), state.slots(), recent);
10
11 // 轮末持久化
12 sessionStateService.save(updatedState);
13 sessionService.appendMessage(sessionId, "assistant", response.speechText(), intent.name(), traceId);
```

Agent Memory（框架层，刻意不用）：

```
1 ReActAgent agent = agentFactory.get(sessionId).intent();
2 agent.getMemory().clear(); // 每轮清空，避免残留
3 agentTraceService.callAgent(sessionId, "IntentAgent", modelName, agent, buildUserPrompt(...));
4 // InMemoryMemory 仅承载本次 call 的临时 Msg, call 结束即无业务价值
```

6.4 选型小结

场景	应使用
多轮槽位、换一批、阶段	SessionState （结构化短期记忆）
意图指代、上下文调整	ConversationTurn / recentHistory
单轮 LLM 调用的框架占位	InMemoryMemory （clear 后使用）

7. 关键代码索引

类	职责
<code>SessionState</code>	结构化短期记忆模型
<code>ConversationTurn</code>	对话摘要模型（注释即短期记忆语义）
<code>SessionStateService</code>	结构化状态 load/save
<code>SessionService</code>	消息落库 + recentConversationTurns + summarize
<code>SlotMergeService</code>	槽位增量合并
<code>IntentReviseService</code>	基于 lastRecommendations 等状态矫正意图
<code>DietOrchestratorService</code>	记忆读写编排、prompt 上下文组装
<code>IntentAgentService</code>	recentHistory + knownSlots → prompt
<code>AgentFactory</code>	会话级 Agent 缓存（隔离用户，非记忆存储）
<code>*AgentBuilder</code>	<code>.memory(new InMemoryMemory())</code> 框架占位

15-页面设计

1. 整体架构

单页应用（SPA），Hash 路由，静态资源托管在 Spring Boot `resources/static/`。

```
1  index.html (壳层: 顶栏导航 + userId 输入 + #app 渲染区)
2
3      └─ api.js    → 封装后端 REST 调用
4      └─ app.js    → Hash 路由 + 各页面 render 函数
```

顶栏导航（6 个页面）：

路由	页面	角色
<code>#/diet</code>	首页	产品介绍 + 快捷入口
<code>#/diet/chat</code>	聊天推荐	用户主流程
<code>#/diet/meals/personal</code>	个人餐食	个人库 CRUD
<code>#/diet/meals/public</code>	公共餐食	公共库浏览
<code>#/admin/traces</code>	Trace	链路排查 + 人工标注
<code>#/admin/evaluations</code>	评估	批量评估报告

全局 用户 ID 输入框（默认 1），所有 API 通过 `X-User-Id` 请求头传递。

2. 用户侧页面

2.1 首页（#/diet）

- 产品简介与使用引导
- 快捷跳转到聊天推荐、个人餐食管理

2.2 聊天推荐（#/diet/chat）

核心交互页面，对接 `POST /api/v1/diet/chat`。

功能：

- 选择数据源模式（PERSONAL / PUBLIC）
- 多轮对话（复用 sessionId）
- 展示 assistant 回复（speechText）
- **ANSWER 响应**：渲染餐食推荐卡片（displayBlocks），含槽位标签 + 反馈按钮
- **CLARIFY 响应**：展示追问文案 + missingSlots 标签 chips
- 推荐卡片支持用户反馈（LIKE/DISLIKE 等 → `POST /api/v1/diet/feedback`）

ChatResponse 字段与 UI 映射：

字段	UI 用途
sessionId	多轮续聊
traceId	后台 Trace 关联
responseType	ANSWER / CLARIFY 分支渲染
speechText	对话气泡文本
displayBlocks	餐食卡片列表
missingSlots	澄清 chips
nextAction	WAIT_USER / ASK_CLARIFY

2.3 个人餐食（#/diet/meals/personal）

对接 `MealController` 个人库 API：

操作	API
列表	GET <code>/api/v1/diet/meals/personal</code>
新增	POST <code>/api/v1/diet/meals/personal</code>
编辑	PUT <code>/api/v1/diet/meals/personal/{mealId}</code>
删除	DELETE <code>/api/v1/diet/meals/personal/{mealId}</code>

表单：

- 餐食名称
- 7 维槽位多选 (标签来自 `GET /api/v1/diet/slot-options`)
- 卡片列表展示 + 编辑/删除

2.4 公共餐食 (#/diet/meals/public)

- GET `/api/v1/diet/meals/public` 只读浏览
- 卡片展示公共库餐食及标签

3. 运营/调试侧页面

3.1 Trace 页 (#/admin/traces)

对接 `AgentTraceController`：

功能	API
时间范围查询	GET <code>/api/v1/diet/debug/traces?startAt&endAt&onlyUnlabeled</code>
单条详情	GET <code>/api/v1/diet/debug/traces/{traceId}</code>
人工标注	PUT <code>/api/v1/diet/debug/traces/{traceId}/label</code>

UI 结构：

- 左/上：Trace 列表表格 (traceId、sessionId、status、duration、时间)
- 右/下：选中 Trace 详情
 - trace_json 事件时间线
 - AGENT_CALL 的 input/output/latency/token
 - 标注表单：expectedIntent / expectedSlots / expectedClarifyAction / labelNote

3.2 评估页 (#/admin/evaluations)

对接 `POST /api/v1/diet/evaluations`：

- 选择评估时间范围
- 可选开启 LLM-as-Judge
- 展示 EvaluationReport:
 - 区间平均总分
 - 各指标 metricAverages
 - 单条 trace 评分明细（可展开 detail）

4. 后端 API 全景

模块	前缀	说明
对话	/api/v1/diet/chat	核心推荐入口
会话	/api/v1/diet/sessions	创建 session
餐食	/api/v1/diet/meals	个人/公共库 CRUD
槽位字典	/api/v1/diet/slot-options	7 维标签
反馈	/api/v1/diet/feedback	用户满意度
Trace	/api/v1/diet/debug/traces	链路查询与标注
评估	/api/v1/diet/evaluations	批量评估

统一鉴权方式：请求头 X-User-Id （当前为简化方案，默认 userId=1）。

5. 页面与业务闭环



6. 关键文件

- 页面壳: `src/main/resources/static/index.html`
- 前端逻辑: `static/assets/js/app.js` (路由与 render)
- API 封装: `static/assets/js/api.js`
- 样式: `static/assets/css/app.css`

16-数据库表设计

1. 概述

饮食助手（diet-agent）使用 MySQL 8.0 作为持久化存储，Schema 名为 `diet_db`。DDL 脚本位于：

```
1  src/main/resources/db/diet_db.sql
```

1.1 设计原则

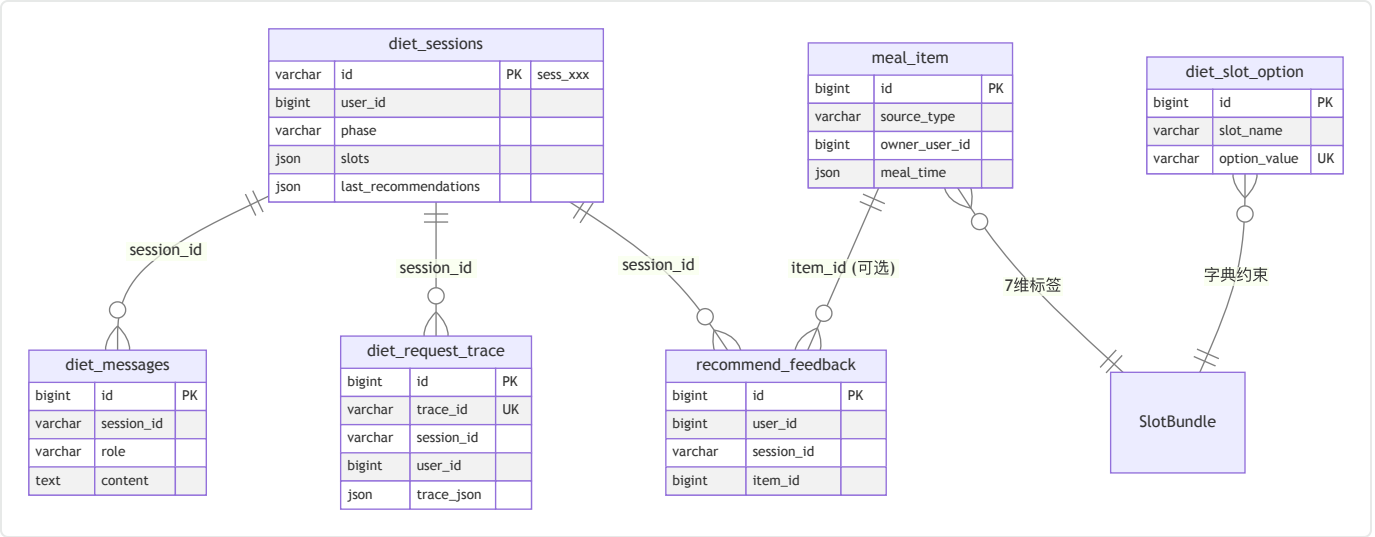
原则	说明
无外键约束	表间通过 <code>session_id</code> 、 <code>user_id</code> 、 <code>trace_id</code> 等逻辑关联，DDL 中 <code>FOREIGN_KEY_CHECKS = 0</code> ，便于独立迁移与清理
JSON 承载可变结构	槽位、推荐历史、餐食标签、Trace 事件等半结构化数据使用 <code>json</code> 列，减少频繁改表
用户 ID 外置	<code>user_id</code> 为业务侧传入的长整型，本库不维护用户表，鉴权由上层网关/业务系统负责
读写分离职责	会话状态由 Orchestrator 独占写入；消息、Trace、反馈为追加型写入；餐食与槽位字典为配置型数据
utf8mb4 全库	字符集 <code>utf8mb4</code> ，排序规则 <code>utf8mb4_unicode_ci</code> ，支持中文标签与 emoji

1.2 表清单

表名	中文名	核心职责
<code>diet_sessions</code>	会话状态表	多轮对话阶段、槽位、推荐历史
<code>diet_messages</code>	对话消息表	用户/助手消息流水
<code>diet_request_trace</code>	请求 Trace 表	Agent 链路追踪与人工标注
<code>diet_slot_option</code>	槽位选项字典	7 维槽位合法取值

meal_item	餐食条目表	PUBLIC/PERSONAL 双数据源菜单
recommend_feedback	推荐反馈表	用户对推荐结果的采纳/评分

2. 逻辑 ER 关系



关联说明：

- 一个 `diet_sessions` 行对应一次完整多轮对话上下文。
- `diet_messages`、`diet_request_trace`、`recommend_feedback` 均通过 `session_id` 挂到同一会话；查询时额外带 `user_id` 做归属校验。
- `recommend_feedback.item_id` 可空，指向 `meal_item.id`，表示用户对某道具体餐食的反馈。
- `diet_slot_option` 与 `meal_item` 无直接外键，通过槽位名称与选项值在业务层对齐。

3. 表结构详述

3.1 diet_sessions — 会话状态表

存储 Orchestrator 维护的多轮对话状态，是推荐链路的「单一事实来源」。

字段	类型	约束	说明
----	----	----	----

<code>id</code>	<code>varchar(64)</code>	PK	会话 ID，格式 <code>sess_</code> + UUID（无连字符）
<code>user_id</code>	<code>bigint</code>	NOT NULL	所属用户，与请求 Header/参数一致
<code>phase</code>	<code>varchar(64)</code>	NOT NULL	会话阶段，枚举见下表
<code>slots</code>	<code>json</code>	NOT NULL	7 维槽位 + <code>_meta</code> 元信息
<code>last_recommendations</code>	<code>json</code>	NOT NULL	本会话已推荐过的 <code>meal_item.id</code> 列表（累积，用于「换一批」去重）
<code>created_at</code>	<code>datetime</code>	NOT NULL	创建时间
<code>updated_at</code>	<code>datetime</code>	NOT NULL	最后更新时间

phase 枚举（`SessionPhase`）：

值	含义
<code>START</code>	会话刚创建，尚未进入业务流程
<code>CLARIFY</code>	系统正在追问缺失槽位
<code>RECOMMEND</code>	已进入推荐链路并保存过上轮结果
<code>PLAN</code>	正在处理多餐规划

索引：

索引名	列	用途
<code>PRIMARY</code>	<code>id</code>	按 <code>sessionId</code> 加载/更新
<code>idx_session_user</code>	<code>user_id</code> , <code>updated_at</code>	按用户查最近会话（扩展用）

slots JSON 结构：

```

1  {
2    "mealTime": ["午餐", "晚餐"],
3    "mood": ["疲惫"],
4    "scene": ["工作"],
5    "healthGoal": ["清淡", "养胃"],
6    "cuisine": ["家常"],
7    "taste": ["清淡"],
8    "convenience": ["快速", "一人食"],
9    "_meta": {
10     "sourceMode": "PUBLIC",
11     "currentIntent": "MEAL_RECOMMENDATION"
12   }
13 }

```

JSON 路径	类型	说明
<code>mealTime ~ convenience</code>	<code>string[]</code>	对应 <code>SlotBundle</code> 七维槽位，值为 <code>diet_slot_option</code> 中的中文选项
<code>_meta.sourceMode</code>	<code>string</code>	<code>PERSONAL</code> / <code>PUBLIC</code> ，决定餐食检索数据源
<code>_meta.currentIntent</code>	<code>string</code>	当前确认的意图，枚举见 <code>Intent</code>

last_recommendations JSON 结构：

```
1  [1, 3, 4, 5]
```

Long 数组，按推荐顺序累积；Orchestrator 在 `MEAL_ADJUST`（换一批）时用于排除已推荐 ID。

读写方：

- 写入：`SessionStateService`（Orchestrator 每轮结束 `save`）
- 读取：`SessionStateService.loadOrCreate`、`SessionMapper.findById`

3.2 diet_messages — 对话消息表

追加型消息流水，供 IntentAgent 短期上下文与前端历史展示。

字段	类型	约束	说明
<code>id</code>	<code>bigint</code>	PK, AUTO_INCREMENT	自增主键
<code>session_id</code>	<code>varchar(64)</code>	NOT NULL	关联 <code>diet_session</code> <code>s.id</code>
<code>role</code>	<code>varchar(32)</code>	NOT NULL	消息角色: <code>user</code> / <code>assistant</code>
<code>content</code>	<code>text</code>	NOT NULL	消息正文
<code>intent</code>	<code>varchar(64)</code>	NULL	该轮识别出的意图 (<code>Intent</code> 枚举名), 可为空
<code>agent_trace_id</code>	<code>varchar(128)</code>	NULL	关联本轮 <code>diet_request_trace.trace_id</code>
<code>created_at</code>	<code>datetime</code>	NOT NULL	消息时间

索引:

索引名	列	用途
<code>PRIMARY</code>	<code>id</code>	主键
<code>idx_message_session</code>	<code>session_id</code> , <code>created_at</code>	按会话拉取最近 N 条消息

典型查询:

```

1  -- SessionMapper#listRecentMessages: 倒序取 limit 条, Java 侧 reverse 后注入 prompt
2  SELECT m.*
3  FROM diet_messages m
4  INNER JOIN diet_sessions s ON s.id = m.session_id AND s.user_id = ?
5  WHERE m.session_id = ?
6  ORDER BY m.created_at DESC, m.id DESC
7  LIMIT ?

```

读写方:

- 写入: `SessionService.appendMessage` (每轮 user/assistant 各一条)
- 读取: `SessionService.recentConversationTurns` (默认最多 `diet.session.max-history-turns` 条, 配置默认 10)

3.3 diet_request_trace — 请求 Trace 表

记录每轮 `/chat` 请求的完整 Agent 链路, 支持调试、评估与人工标注。

字段	类型	约束	说明
<code>id</code>	<code>bigint</code>	PK, AUTO_INCREMENT	自增主键
<code>trace_id</code>	<code>varchar(128)</code>	NOT NULL, UK	请求级唯一 Trace ID
<code>session_id</code>	<code>varchar(64)</code>	NOT NULL	关联会话
<code>user_id</code>	<code>bigint</code>	NOT NULL	所属用户
<code>status</code>	<code>varchar(32)</code>	NOT NULL	<code>SUCCESS</code> / <code>FAILED</code>
<code>event_count</code>	<code>int</code>	NOT NULL, DEFAULT 0	事件条数
<code>duration_ms</code>	<code>bigint</code>	NULL	整轮耗时 (毫秒)
<code>error_message</code>	<code>text</code>	NULL	失败时的错误摘要
<code>trace_json</code>	<code>json</code>	NOT NULL	完整 Trace 事件数组
<code>created_at</code>	<code>datetime</code>	NOT NULL	创建时间
<code>updated_at</code>	<code>datetime</code>	NOT NULL	更新时间
<code>expected_intent</code>	<code>varchar(64)</code>	NULL	人工标注: 期望意图
<code>expected_slots</code>	<code>json</code>	NULL	人工标注: 期望槽位 (<code>SlotBundle</code> 结构)
<code>expected_clarify_action</code>	<code>varchar(32)</code>	NULL	人工标注: 期望澄清动作 <code>ASK</code> / <code>READY</code>

labeled_by	bigint	NULL	标注人 user_id
labeled_at	datetime	NULL	标注时间
label_note	varchar(512)	NULL	标注备注

索引:

索引名	列	用途
PRIMARY	id	主键
uk_request_trace	trace_id	按 traceId 唯一定位
idx_request_trace_session	session_id , created_at	会话维度查 Trace
idx_request_trace_user	user_id , created_at	用户维度查 Trace
idx_request_trace_status	status , created_at	按成功/失败筛选
idx_request_trace_label	expected_intent , labeled_at	标注工作台按意图筛选

trace_json 顶层结构:


```

1  {
2    "traceId": "trace_xxx",
3    "sessionId": "sess_xxx",
4    "userId": 1,
5    "status": "SUCCESS",
6    "durationMs": 3200,
7    "events": [
8      {
9        "stepOrder": 1,
10       "eventType": "REQUEST_RECEIVED",
11       "phase": "HTTP",
12       "agentName": null,
13       "modelName": null,
14       "inputPayload": "{...}",
15       "outputPayload": null,
16       "latencyMs": null,
17       "inputTokens": null,
18       "outputTokens": null,
19       "totalTokens": null,
20       "errorMessage": null,
21       "createdAt": "2026-07-01T12:00:00"
22     },
23     {
24       "stepOrder": 2,
25       "eventType": "AGENT_CALL",
26       "phase": "AGENT",
27       "agentName": "IntentAgent",
28       "modelName": "qwen-max",
29       "inputPayload": "...",
30       "outputPayload": "...",
31       "latencyMs": 850,
32       "inputTokens": 1200,
33       "outputTokens": 80,
34       "totalTokens": 1280,
35       "errorMessage": null,
36       "createdAt": "2026-07-01T12:00:01"
37     }
38   ]
39 }

```

常见 eventType:

eventType	phase	说明
REQUEST_RECEIVED	HTTP	收到 chat 请求

INTENT_DECISION	INTENT	意图识别结果
CLARIFY_DECISION	CLARIFY	澄清节点决策
MEAL_SEARCH	RECOMMEND	餐食检索
MEAL_RANK	RECOMMEND	排序结果
AGENT_CALL	AGENT	单次 LLM Agent 调用
REQUEST_FINISHED	HTTP	请求结束

标注字段用途：

评估服务（`EvaluationService`）将 `expected_*` 与 `trace_json.events` 中实际输出对比，计算意图准确率、槽位 F1、澄清准确率等指标。未标注记录可通过 `onlyUnlabeled=true` 筛选。

读写方：

- 写入：`AgentTraceService`（`openTrace` → 记录事件 → `close` 时 INSERT）
- 标注：`AgentTraceService.updateLabel`
- 读取：`AgentTraceController` 调试 API、`EvaluationService` 批量评估

3.4 diet_slot_option — 槽位选项字典表

维护 7 维槽位的合法中文选项，供前端下拉、Clarify 追问和 LLM 输出校验。

字段	类型	约束	说明
id	bigint	PK, AUTO_INCREMENT	自增主键
slot_name	varchar(64)	NOT NULL	槽位英文名，与 <code>SlotBundle</code> 字段对应
option_value	varchar(64)	NOT NULL	选项中文展示值
sort_order	int	NOT NULL, DEFAULT 0	同槽位内排序，越小越靠前
enabled	tinyint	NOT NULL, DEFAULT 1	1=启用，0=禁用

<code>created_at</code>	<code>datetime</code>	NOT NULL	创建时间
<code>updated_at</code>	<code>datetime</code>	NOT NULL	更新时间

索引：

索引名	列	用途
<code>PRIMARY</code>	<code>id</code>	主键
<code>uk_slot_option</code>	<code>slot_name</code> , <code>option_value</code>	同槽位选项唯一
<code>idx_slot_enabled</code>	<code>slot_name</code> , <code>enabled</code> , <code>sort_order</code>	按槽位查启用选项

slot_name 与 SlotBundle 映射：

slot_name	SlotBundle 字段	选项数量（种子数据）
<code>mealTime</code>	<code>mealTime</code>	8
<code>mood</code>	<code>mood</code>	10
<code>scene</code>	<code>scene</code>	11
<code>healthGoal</code>	<code>healthGoal</code>	14
<code>cuisine</code>	<code>cuisine</code>	24
<code>taste</code>	<code>taste</code>	16
<code>convenience</code>	<code>convenience</code>	10

读写方：

- 读取：`SlotOptionService` → `SlotOptionMapper.findEnabledValues` / `findSlotNames`
- 写入：运维/后台直接改表或扩展 INSERT，应用层无 CRUD API

3.5 meal_item — 餐食条目表

双数据源餐食库： `PUBLIC` 为公共示范菜单， `PERSONAL` 为用户私有菜单。

字段	类型	约束	说明
<code>id</code>	<code>bigint</code>	PK, AUTO_INCREMENT	餐食 ID
<code>source_type</code>	<code>varchar(16)</code>	NOT NULL	<code>PUBLIC</code> / <code>PERSONAL</code>
<code>owner_user_id</code>	<code>bigint</code>	NULL	<code>PERSONAL</code> 时必填； <code>PUBLIC</code> 必须为 NULL
<code>name</code>	<code>varchar(128)</code>	NOT NULL	餐食名称
<code>meal_time</code>	<code>json</code>	NOT NULL	适用餐段标签数组
<code>mood</code>	<code>json</code>	NOT NULL	适用心情标签数组
<code>scene</code>	<code>json</code>	NOT NULL	适用场景标签数组
<code>health_goal</code>	<code>json</code>	NOT NULL	健康目标标签数组
<code>cuisine</code>	<code>json</code>	NOT NULL	菜系标签数组
<code>taste</code>	<code>json</code>	NOT NULL	口味标签数组
<code>convenience</code>	<code>json</code>	NOT NULL	便利性标签数组
<code>created_at</code>	<code>datetime</code>	NOT NULL	创建时间
<code>updated_at</code>	<code>datetime</code>	NOT NULL	更新时间

索引：

索引名	列	用途
<code>PRIMARY</code>	<code>id</code>	主键
<code>idx_public_meal_source</code>	<code>source_type</code>	公共库列表
<code>idx_private_meal_source</code>	<code>owner_user_id</code> , <code>source_type</code>	个人库按用户查

7 维 JSON 列示例（番茄鸡蛋面）：

```
1 {
2   "meal_time": ["午餐", "晚餐", "三餐"],
3   "mood": ["疲惫", "低落"],
4   "scene": ["工作", "校园", "家里"],
5   "health_goal": ["清淡", "养胃", "易消化"],
6   "cuisine": ["家常", "粉面"],
7   "taste": ["清淡", "番茄味"],
8   "convenience": ["快速", "一人食"]
9 }
```

标签值必须来自 `diet_slot_option` 对应槽位的 `option_value`，否则 `JSON_OVERLAPS` 检索无法命中。

数据源隔离规则：

source_type	owner_user_id	可见范围
PUBLIC	NULL	所有用户的 PUBLIC 模式会话
PERSONAL	用户 ID	仅该用户的 PERSONAL 模式会话

核心检索 SQL（`MealMapper#search`）：

```
1 SELECT *
2 FROM meal_item
3 WHERE source_type = 'PUBLIC' AND owner_user_id IS NULL -- 或 PERSONAL 分支
4       AND ('[]' = ? OR JSON_OVERLAPS(meal_time, ?))
5       AND ('[]' = ? OR JSON_OVERLAPS(mood, ?))
6       -- ... 其余 5 维同理
7 ORDER BY updated_at DESC
8 LIMIT 50
```

空维度传 `'[]'` 表示「不限」，详见 [08-MySQL-JSON_OVERLAPS标签检索.md](#)。

种子数据：

id	source_type	name
1	PUBLIC	番茄鸡蛋面
2	PUBLIC	清汤馄饨
3	PUBLIC	鸡胸肉轻食碗

4	PUBLIC	麻辣香锅
5	PERSONAL (user_id=1)	土豆炖牛肉

读写方：

- 写入：MealService (个人餐食 CRUD)
- 读取：MealSearchService 检索 + MealRankService 排序

3.6 recommend_feedback — 推荐反馈表

记录用户对推荐结果的显式反馈，供评估报告中的「用户满意度」维度使用。

字段	类型	约束	说明
id	bigint	PK, AUTO_INCREMENT	自增主键
user_id	bigint	NOT NULL	反馈用户
session_id	varchar(64)	NOT NULL	关联会话
item_id	bigint	NULL	关联 meal_item.id , 整轮反馈可为空
action	varchar(32)	NOT NULL	反馈动作，见下表
rating	int	NULL	1—5 星评分，可选
reason	varchar(512)	NULL	文字原因
created_at	datetime	NOT NULL	反馈时间

索引：

索引名	列	用途
PRIMARY	id	主键
idx_feedback_user	user_id , created_at	评估时按用户+时间范围拉取

action 常见取值（评估归一化逻辑）：

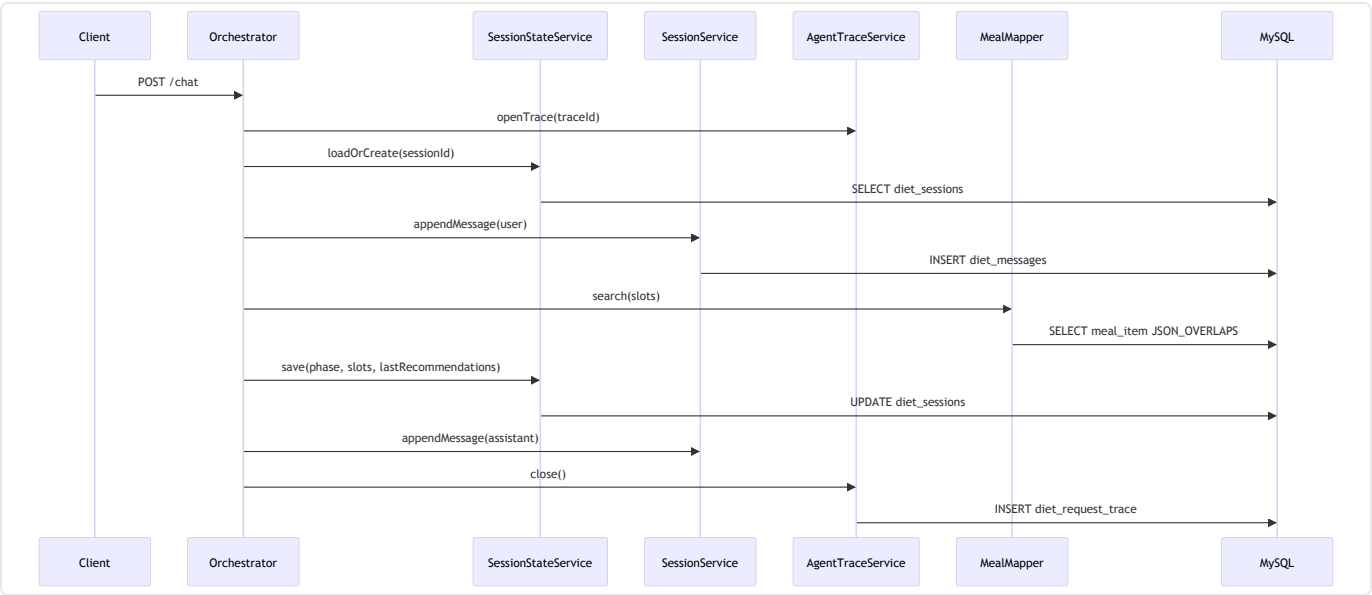
action（大小写不敏感）	满意度得分
LIKE，UP，ADOPT，ACCEPT	1.0
DISLIKE，DOWN，REJECT	0.0
其他	0.5

若同时提供 rating，优先使用 rating 归一化到 0—1（rating / 5.0）。

读写方：

- 写入：FeedbackService.save → POST /feedback
- 读取：EvaluationService 按 session 列表 + 时间范围聚合

4. 数据流与表协作



一轮 chat 典型落库：

- diet_sessions — UPDATE 1 行（phase / slots / last_recommendations）
- diet_messages — INSERT 2 行（user + assistant）
- diet_request_trace — INSERT 1 行（整轮 Trace）

5. Java 模型映射

表	Row 模型	业务模型	Service
diet_sessions	SessionRow	SessionState	SessionStateService
diet_messages	SessionMessageRow	ConversationTurn (只读转换)	SessionService
diet_request_trace	RequestTraceRow	—	AgentTraceService
diet_slot_option	— (直接 List<String>)	—	SlotOptionService
meal_item	MealItemRow	MealItem	MealService / MealSearchService
recommend_feedback	FeedbackRow	FeedbackRequest	FeedbackService

6. 索引与性能考量

场景	使用索引	说明
加载会话	diet_sessions.PK	单行点查, O(1)
拉最近消息	idx_message_session	倒序 LIMIT, 配合 JOIN 校验 user_id
Trace 调试	uk_request_trace	traceId 唯一点查
评估批查	idx_request_trace_user + 时间过滤	限制 LIMIT ≤ 1000
餐食检索	全表 + JSON_OVERLAPS	当前数据量小; 规模化可考虑生成列 + 索引或 ES

槽位字典	<code>idx_slot_enabled</code>	按 slot_name 查启用项，结果集极小
------	-------------------------------	------------------------

JSON 列注意：

- MySQL 8.0 `JSON_OVERLAPS` 无法直接走普通 B-Tree 索引；餐食库增长到万级以上时建议评估倒排索引或冗余标签关联表。

7. 初始化与迁移

全新环境：

```
1  mysql -u root -p < src/main/resources/db/diet_db.sql
```

脚本会：

- `DROP TABLE IF EXISTS` 六张表（开发环境全量重建）
- `CREATE TABLE` 建表
- `INSERT` 槽位字典 93 条 + 公共餐食 4 条 + 个人餐食 1 条

生产迁移建议：

- 不要使用全量 DROP，改为 Flyway/Liquibase 增量迁移
- `diet_slot_option` 新增选项用 `INSERT ... ON DUPLICATE KEY UPDATE`
- 表结构变更优先保持 JSON 结构向后兼容（新增字段而非改字段语义）

8. 枚举速查

Intent（意图，存于 messages.intent / trace 标注 / slots._meta）

枚举值	含义
<code>MEAL_RECOMMENDATION</code>	请求餐食推荐
<code>CLARIFY_NEEDED</code>	信息不足需追问
<code>MEAL_ADJUST</code>	换一批 / 调整偏好

MEAL_PLAN	多餐规划
HEALTH_RISK	健康风险内容
OTHER	与饮食无关

SourceMode（数据源，存于 slots._meta）

枚举值	含义
PUBLIC	公共餐食库
PERSONAL	用户私有餐食库

ClarifyAction（澄清动作，存于 trace 标注）

枚举值	含义
ASK	需追问用户
READY	信息足够，进入推荐

9. 附录：完整 DDL 索引

源文件： `src/main/resources/db/diet_db.sql`（MySQL 8.0.46, InnoDB, utf8mb4_unicode_ci）

```
1  -- 六张表
2  diet_messages
3  diet_request_trace
4  diet_sessions
5  diet_slot_option    -- 含 93 条种子数据
6  meal_item          -- 含 5 条种子数据
7  recommend_feedback
```

如需查看字段级 DDL，以仓库 SQL 文件为准；本文档侧重设计语义与业务映射，DDL 变更时请同步更新本文档第 3 节字段说明。

Agent 项目怎么做才有含金量

Agent 项目并不是 prompt + 大模型 + 网页

Agent 项目怎么做才有含金量，给大家拆解出 Agent 项目 六大要点

1. 解决真实问题

误区：

- 不要为了Agent而Agent
- 差的项目就是用户说一句，模型回一句，再加几个工具调用，就假装是Agent项目

正确做法：

- 要有明确的业务场景，就算是生活中的小细节也可以，只要是真实痛点就值得被解决，就是业务场景
- 明确的用户
- 能衡量效果

2. 完整的后端工程体系（Agent项目本质上还是后端项目）

误区：只有prompt和调用链，没有工程架构，只能是demo

正确做法：

- API接口设计
- 数据库设计
- 会话管理
- 日志追踪
- 错误兜底
- 多用户并发

3. 体现真正的Agent能力

误区：只会调用大模型，没有使用Agent真正的能力

正确做法：

- 意图识别
- 任务规划
- 工具选择
- 状态追踪
- 失败重试
- 结果校验

4. 可观测性

误区：无结构化追踪、监控点太少、无告警机制，不要把 Agent 做成黑盒

正确做法：

- 每一次请求必须能够被 全链路监控与追踪，任意环节报错超时，可快速定位卡点
- 指标监控、可视化大盘（例如Agent能力指标、成本&资源消耗指标）

5. 评价体系

误区：只靠 我觉得等主观意愿来判断，评价维度单一，评价结果不回流优化流程

正确做法：

- 多维度自动化批量评价（系统自动打分+用户真实感受评价）
- 低分样本反哺Agent优化，形成闭环

6. 人机协同

误区：完全由 Agent 自主运行完成任务

正确做法：

- 人机应当是协同的，人需要在 Agent 系统中，只是 人在 Agent 中的哪个位置，什么时候起作用 是关键点

- 例如关键的节点让人确认：删除文件、发送邮件、审批、敏感信息、隐私数据等等高风险操作
- <https://zhanfei.blog.csdn.net/article/details/160216397>

后端面 Agent 开发

面试 Agent 开发岗位，其实学过后端是有优势的

Agent 开发本质上就是后端开发的一条能力分支

Agent 系统本质上也是后端开发项目

Agent 系统也不仅仅只有 LLM，也有后端工程化的能力建设

例如 异常兜底、链路追踪、数据库表设计

—

我看到过一段话，我觉得说的还挺正确的，如下：

—

但 Agent 岗位又和后端岗位有所不同，Agent 岗位更加看重项目

很多人追新框架、新工具、新概念，每天都在学习，三个月了还在搭 Demo

如果你现在还在 Demo 里转圈，别慌。你缺的不是能力，是方向。

从今天开始，把你手头那个跑一遍就扔的 Demo

变成一个能拿出数据、能说清迭代、能让面试官追问半小时的项目

把项目做扎实，把数据跑出来，拿着一个真实项目去面试

这才是后端转 Agent 的最短路径

大模型的幻觉

大模型的“幻觉”指的是 AI 生成了看似合理但实际上错误或编造的信息

例如，它可能会编造不存在的事实、错误引用文献、甚至捏造公司或人物的信息

幻觉产生原因

语言模型的“填空”机制

- Transformer 语言模型本质上是一个“**填空预测器**”它是根据概率预测来选择下一个输出的词，而不是在“思考”正确答案

训练数据存在缺陷

- 训练数据本身可能包含 错误信息、不完整数据、偏见信息，导致模型学到不真实的内容
- 训练数据可能过时，例如，GPT-4 的数据 截止到 2023 年初，无法回答最新时事

缺乏事实验证能力

- 语言模型在生成文本时，并不会主动去 查证答案的真实性

长文本记忆力有限

- 由于上下文窗口有限 (如GPT-4-turbo约128k tokens)，当文本过长时，AI可能遗忘前面提到的信息

解决幻觉

解决幻觉的一些方案

1. RAG，通过RAG的方式，让大模型在回答问题之前先检索真实数据，再让模型进行回答
2. Fine-tuning，即微调，通过微调的方式，给模型学习专业领域的知识，让他更好的回答
3. 限制AI的回答，比如在提示词中告诉他如果你不知道，直接就回答不知道
4. 通过标注和反馈不断优化模型。并把反馈可以给到模型让模型调整
5. 让同一个模型多次生成同一个内容的答案，然后选择一个最终版本
6. 在问题回答之后，让 AI自己检查自己的答案，并标记不确定的部分
7. 联网，在 AI生成答案之前，先通过网络查询最新数据

8. 让 AI 先写下推理过程，再得出最终结论，而不是直接给出答案

如何理解大模型

定义与特点

定义：大模型通常指基于深度学习的大规模人工智能模型
例如像 GPT-4 大语言模型 （LLM, Large Language Model)

特点：

- 大模型的"大"主要体现在参数规模大。传统深度学习模型的参数规模通常在百万级到千万级，而大模型的参数量达到了 百亿级、千亿级，甚至万亿级。以 GPT-4 为例，参数规模可能达 万亿级
如我们通常说的7B模型，指的就是他的参数量达到了70亿
- 大模型的核心是基于 深度学习和 神经网络，通常采用 Transformer 架构
 - Transformer 通过 "自注意力机制 (Self-Attention) " 和 "多头注意力机制 (Multi-Head Attention) "实现高效的信息处理，相比于传统的 RNN（循环神经网络）和 CNN（卷积神经网络），具有更强的并行计算能力和更长的上下文理解能力。（了解即可）

应用场景

不要简单的认为大模型就只是AI对话，他的应用方向非常广泛，
涵盖 自然语言处理(NLP)、计算机视觉(CV)、语音识别、科学计算、自动驾驶等 领域

应用领域	代表应用	典型模型
1.NLP 语言 AI	ChatGPT、翻译、代码生成	GPT-4、LLaMA 3、Claude
2.计算机视觉(CV)	图像生成、目标检测	Stable Diffusion、DALL·E、ViT
3.语音AI	语音识别、语音克隆	Whisper、VALL-E、Tacotron

4.自动驾驶&机器人	无人车、机器人导航	Tesla FSD、Gato、Perceiver
5.科学计算&医疗	药物研发、医学影像	AlphaFold、Med-PaLM 2
6.推荐系统&广告	电商推荐、精准营销	DeepFM、DINO、YouTube AI

大模型擅长做生成内容、代码编写、文本处理、多态任务
但它对高精度计算、事实准确性、实时数据获取存在局限性

擅长方向

- 1. **文本理解**：能阅读、总结、分析文章，如阅读理解、问答系统
- 2. **文本生成**：能撰写高质量 文章、小说、新闻、代码、法律合同等
- 3. **机器翻译**：可以翻译多种语言，质量接近专业翻译水平 (如 DeepL、Google Translate)
- 4. **信息提取**：能从文档中提取关键信息，如 法律、医学、财务报告分析
- 5. **代码生成**：自动编写 Python、JavaScript、C++代码，辅助开发者 (如 GitHub Copilot)
- 6. **代码调试**：帮助找出代码中的 Bug、优化代码性能
- 7. **代码解释**：能解释复杂代码的逻辑，帮助学习和维护遗留代码
- 8. **广泛知识问答**：大模型掌握 百科知识、历史、科技、医学、经济等 领域的信息
- 9. **图像识别 & 生成** (如 DALL·E、Stable Diffusion)：生成艺术作品、设计海报
- 10. **语音识别 & 语音合成** (如 Whisper、VALL-E)：实现 语音转文字(ASR)、AI语音播报
- 11. **视频生成** (如 Sora)：基于文本输入，生成高质量视频
- 12. **总结长文档**：如会议纪要、论文总结
- 13. **从非结构化文本中提取信息**：如将文章转换成 表格、JSON 数据

不擅长方向

- 1. 无法进行精确 数值计算，如 大数运算、金融精算、密码学计算 (如3.9和3.11那个大?)
- 2. 可能犯逻辑错误，推理链条过长时，可能得出错误结论
- 3. 容易“编造”信息，即使信息看起来可信，但可能并不存在

4. 可能给出 错误的医学、法律、科技等专业知识(如医学诊断、法律条文)
5. 对 长尾知识（冷门信息、最新论文、特定行业信息）缺乏准确性
6. 训练好的大模型不会像人类一样主动学习新知识，无法实时更新

关键问题

目前，大模型遇到的一些关键问题有以下几个

- **数据和隐私问题**：大模型训练涉及大量数据，可能存在隐私泄露风险
- **幻觉**：模型可能生成虚假或不准确的信息
- **能耗高**：训练一次大模型可能消耗数千吨二氧化碳当量的能源
- **可控性和安全性**：如何让模型可靠、透明地运作是一个重要挑战

[你和公司] 平时如何使用 AI

1. AI 用处

1.1 公司中如何使用AI

AI 与开发人员的日常工作逐渐强相关，AI 与工作的结合势不可挡，开发人员接手需求后：

1. 会利用 AI 快速梳理业务逻辑，输出流程图、时序图、架构图（提效）
2. 会利用 AI 辅助技术方案设计（提效）
3. 会利用 AI 辅助编码（提效）
4. 会利用 AI CodeReview（保质）
5. 会利用 AI 写单元测试以及测试单元测试覆盖率（保质）

1.2 你平时如何使用AI

你平时如何使用AI？这是面试官最最常问的问题，也是可以弯道超车的问题

上述说的虽然是公司中如何使用AI，但你作为一名面试者(假设你是一名学生)

我们可以联想到 学校的计算机学院的一些计算机工作室，也是一个小团队

因此 公司团队 === 学校工作室小团队，因此背景就相通了，我给大家想了一个回答，放在最后了

【方案】业务现状逻辑梳理、生成流程图、时序图、架构图

【方案】辅助生成技术方案

--

【自测】生成单元测试

【自测】单元测试覆盖率校验

--

【开发】辅助编码

--

【bug缺陷/线上问题/启动失败等】辅助问题定位排查

--

【代码质量】AI Code Review

2. 回答

我在学校有参与一个计算机工作室的团队，我是作为组长的有带几个同学做一些老师的项目
因为我们要做老师的项目，但是有一些学生刚加入工作室又没有太多的开发经验，为了保证质量和效率
所以我就开始在我们团队去推行大家都使用AI，所以我们团队其实在AI的使用上还比较多

--

比如我们有一些老的项目维护，我们会用AI去对老项目进行 业务现状逻辑梳理、生成流程图、时序图、架构图等

还有接手到一些新的项目后，我们会用AI去辅助生成技术方案，用AI去辅助编码

以及用AI去生成一些单元测试，在提交代码之前会用 AI 去进行 CodeReview

我们使用AI主要是帮助我们提升效率以及保证代码质量的

--

当然在这个过程中，比较重要的就是提示词嘛

比如 AI CR、生成单元测试，我们都有设计一些提示词，这样让AI生成的结果更精确更高质量

接着，你猜面试官会不会很感兴趣，他会接着问什么问题？ 应该很好猜吧

1. 你是怎么想到用AI用在这么多地方的？ —— 这时候你可以随意说，比如：我对AI这些比较感兴趣，恰好我们团队又有这些痛点（刚加入工作室的同学没什么经验，但是又要接受老师的开发项目），再加上我也参考了一些大厂的一些文章，所以我就突然想到在工作室团队也可以用AI做这些事情了。
2. 那你们团队用AI时有没有什么经验或者一些感受呢？ —— 回答参考 [2.1 一些使用的小经验/感受]

3. 你们是怎么设计AICR、单元测试的提示词的？——回答参考【如何写Prompt提示词】中的AICR提示词设计

2.1. 一些使用的小经验/感受

为什么要提这个？能让你的回答更真实，面试官能知道你有真正用过，不是只是泛泛而谈
也建议你们真的在这些方面去使用一下AI，感受一下

【代码质量】AI Code Review 这一项是非常有用的（先去读一下AI Code Review的Prompt提示词）

1. 它能够识别出我们代码中的一些潜在的bug或性能风险，例如会提醒你某个sql语句应该添加索引，比如我们之前接了一个项目，数据量还比较大，上线后发现有一个sql很慢很慢，后来我们发现没加索引，但是其实在AICR的报告中，AI 早就提示我们这里有性能风险了，所以AICR是非常有用的
2. 它能够尽量帮助我们提高代码的质量、包括可维护性、健壮性等

【自测】生成单元测试：无需人工写单测，提效

【方案】业务现状逻辑梳理、生成流程图、时序图、架构图

1. 在熟悉代码时很方便，可以先让AI去梳理一下业务逻辑，再对照着看代码，提升效率
2. 在生成架构图这方面会比较粗糙，需要通过多轮的对话沟通和调整才能生成比较好的架构图

--大家可以自行使用后自己感受一下

如何写Prompt提示词

3. 如何写好的prompt

Prompt设计原则：

1. 清晰明确

- 使用具体、明确的指令
- 避免歧义
- 提供示例

2. 结构化

- 使用角色定义 (Role)
- 任务描述 (Task)
- 输出格式 (Format)
- 约束条件 (Constraints)

3. 提供样例

- 提供示例输入输出
- 展示期望的行为模式

4. 迭代优化

- 基于结果调整
- A/B测试不同版本（这个如果你不懂就不要说哈）

4. Prompt 提示词设计

提示词建议大家要仔细的看过，清楚提示词中大概有哪些内容，如果面试官问起，你可以列举出最核心最关键的东西，包括AICR提示词、生成单元测试的提示词、生成单元测试覆盖率的提示词

用AI生成的单元测试，再用AI生成单元测试覆盖率，会不会有自圆其说的。

比如 AI 自己给自己评定覆盖率很高

这里可以用不同的模型来规避这个问题，比如用GPT5.4来生成单元测试，用Claude来生成单元测试的覆盖率

4.1 AI Code Review

你是如何设计AI CR 的 Prompt 提示词的？（面试官对你的Prompt一定是很感兴趣的）

如果我直接把本次修改的 diff 文件发给 cursor (AI) 让他进行 CodeReview

我发现 AI 生成的结果可能会存在一些问题

- 比如，可能出现漏评的情况，AI 可能只看到语法错误，但没检查业务逻辑、性能瓶颈、安全漏洞、并发问题等，导致AICR的结果缺少必要的评审项，即评审范围是不可控的
- 比如，输出格式混乱，没有区分必须修复、建议修复等优先级

因此我设计了AICR的提示词，让AI生成的结果更精确、更高质量、更便于我们使用

提示词的设计有几个核心点

1. 首先，我会先为AI设定定义一个角色，比如：作为Java资深架构师/技术专家进行代码评审
2. 其次，我设定代码评审中判定的问题划分为三类，分别是 Critical (必须修复)、Warning (建议修复)、Info (优化建议)，各自针对不同方面的问题，方便我们在开发时挑选优先级更高的优先进行修改
 - a. Critical (必须修复) 包括：安全漏洞、严重性能问题、数据一致性问题、线程安全问题、空指针
 - b. Warning (建议修复) 包括：代码质量问题、潜在性能隐患、可维护性问题
 - c. Info (优化建议) 包括：代码风格改进
3. 接着，针对评审的方向，我会定义一些具体的评审内容
 - a. 比如代码质量方向，具体就会包括注释清晰且必要、方法长度适中，单一职责原则、遵循命名规范等
 - b. 比如安全性方向，具体就会包括日志记录不包含敏感信息、敏感数据加密等
4. 最后我会规范AI的输出结果，我会提供一个问题报告模板，让AI去照着模板输出内容。
 - a. 模板内容就包括：问题类型是Critical还是Warning还是Info、影响是什么、建议是什么 这样的报告模板

```
1  # AI代码评审规则
2
3  ## 评审角色定位
4
5  作为Java资深架构师/技术专家进行代码评审，分析代码变更并提供专业的评审意见。
6
7  ## 评审原则
8
9  - **客观性**：基于技术标准，避免主观偏见
10
11 - **建设性**：提供具体可行的改进建议
12
13 - **教育性**：解释问题原因和最佳实践
14
15 - **完整性**：覆盖功能、性能、安全、可维护性等多个维度
16
17 ## 评审流程
18
19 1. **理解变更上下文**
20
21     - 阅读PR/MR描述，理解变更目标和业务价值
22
23     - 分析变更的技术必要性和复杂度
24
25     - 评估变更的影响范围和相关文件
26
27     - 查看方法完整调用链
28
29 2. **分析代码变更**
30
31     - 简要说明变更的核心目标和业务价值
32
33     - 分析技术方案和实现细节
34
35     - 定位到项目具体代码行数
36
37 3. **识别问题和风险**
38
39     - 缺陷和副作用：逻辑错误、边界条件、并发安全、资源泄漏、空指针
40
41     - 性能隐患：数据库效率、内存使用、算法复杂度
42
43     - 代码质量：可读性、可维护性、代码重复、命名规范
44
45     - 规范偏离：设计模式误用、框架使用不当、安全最佳实践
46
47 4. **提供改进建议**
```

48
49 - 提供具体改进措施和替代方案
50
51 - 推荐重构和性能优化方案
52
53 - 建议更好的设计模式或架构方案
54
55 5. ****输出评审报告****
56
57 - 按优先级组织问题
58
59 - 提供清晰的改进建议
60
61 - 给出整体评分
62
63 **## 评审标准**
64
65 **### 问题分级**
66
67 - 🚫 ****Critical (必须修复)****: 安全漏洞、严重性能问题、数据一致性问题、线程安全问题、
68 空指针
69 - ⚠️ ****Warning (建议修复)****: 代码质量问题、潜在性能隐患、可维护性问题
70
71 - 🟡 ****Info (优化建议)****: 代码风格改进、最佳实践建议、架构优化建议、测试覆盖不足
72
73 **### 评审维度检查清单**
74
75 **#### 1. 代码质量**
76
77 - [] 代码逻辑清晰，易于理解
78
79 - [] 遵循项目的编码规范和命名约定
80
81 - [] 方法长度适中，单一职责原则
82
83 - [] 没有明显的性能问题或算法缺陷
84
85 - [] 错误处理完整且得当
86
87 - [] 注释清晰且必要，避免过度注释
88
89 **#### 2. 安全性**
90
91 - [] 输入验证完整，包括参数校验和边界检查
92
93 - [] 权限控制正确，遵循最小权限原则
94

95 - [] 敏感数据正确加密和脱敏
96
97 - [] SQL注入防护措施到位
98
99 - [] XSS和CSRF防护
100
101 - [] 日志记录不包含敏感信息
102
103 - [] 依赖库安全性检查
104
105 ##### 3. 可维护性
106
107 - [] 类和方法职责单一，高内聚低耦合
108
109 - [] 适当的设计模式使用
110
111 - [] 合理的变量和方法命名，见名知意
112
113 - [] 代码复用性好，避免重复代码
114
115 - [] 单元测试覆盖率充足
116
117 - [] 易于扩展和修改
118
119 ##### 4. 架构设计
120
121 - [] 符合现有架构模式和分层设计
122
123 - [] 模块边界清晰，依赖关系合理
124
125 - [] 接口设计简洁明确
126
127 - [] 数据库设计规范，表结构合理
128
129 - [] API设计遵循RESTful原则
130
131 - [] 配置管理规范
132
133 ##### 5. Java项目特定要求
134
135 - [] 遵循Spring框架最佳实践
136
137 - [] 正确使用注解和依赖注入
138
139 - [] 异常处理机制完善
140
141 - [] 线程安全考虑充分
142

143 - [] 空指针问题处理妥当, 避免NPE
144
145 - [] 内存管理得当, 避免内存泄漏
146
147 - [] 合理使用Java 8+新特性
148
149 ##### 6. 数据库相关
150
151 - [] SQL语句性能优化, 避免N+1查询
152
153 - [] 索引使用合理且有效
154
155 - [] 事务边界清晰, ACID属性保证
156
157 - [] 数据一致性和完整性约束
158
159 - [] 分页查询优化
160
161 - [] 连接池配置合理
162
163 ##### 7. 测试相关
164
165 - [] 单元测试覆盖核心业务逻辑
166
167 - [] 测试用例设计合理, 包含正常和异常情况
168
169 - [] Mock对象使用恰当
170
171 - [] 集成测试覆盖关键流程
172
173 - [] 测试数据管理规范
174
175 ##### 8. 性能考量
176
177 - [] 时间复杂度和空间复杂度分析
178
179 - [] 缓存使用策略合理
180
181 - [] 批量操作优化
182
183 - [] 异步处理适当使用
184
185 - [] 资源使用监控
186
187 ## 评审职责界限
188
189 ### ✅ 应该做的
190

191 - 识别代码问题和风险
192
193 - 提供改进建议和最佳实践
194
195 - 解释技术原理和设计考量
196
197 - 回到项目查看源码
198
199 - 定位到项目具体代码行数
200
201 - 查看方法完整调用链
202
203 - 推荐工具和方案
204
205 **###** **✗** 不应该做的
206
207 - 直接修改或重写代码
208
209 - 执行任何代码变更操作
210
211 - 做出最终的技术决策
212
213 - 替代人工评审的最终确认
214
215 **##** 评审输出格式
216
217 **###** 问题报告模板
218
219 每个问题应包含以下信息：
220
221 - 问题类型：[Critical/Warning/Info]
222
223 - 位置：[文件名:行号]
224
225 - 问题描述：[具体问题说明]
226
227 - 影响：[潜在影响分析]
228
229 - 建议：[具体改进方案]
230
231 - 处理情况：
232
233 - 是否处理：
234
235 - 处理人：
236
237 - 时间：
238

239 ### 报告组织方式
240
241 1. **按优先级分组**：先列出Critical问题，再Warning，最后Info
242
243 2. **按文件分组**：同一文件的问题集中展示
244
245 3. **按类型分组**：安全性、性能、代码质量等分类展示
246
247 ### 评审报告结构
248
249 评审概览
250
251 - 变更意图：[简要说明]
252
253 - 影响范围：[分析]
254
255 - 整体评分：[X/5分]
256
257  Critical 问题（必须修复）
258
259 [问题列表]
260
261  Warning 问题（建议修复）
262
263 [问题列表]
264
265  Info 优化建议
266
267 [建议列表]
268
269
270 总结
271
272 [整体评价和改进方向]
273
274 ### 示例
275
276 **正确示例**：
277
278  Warning
279
280 位置：UserService.java:25
281
282 问题描述：缺少事务注解，可能导致数据不一致
283
284 影响：并发情况下可能出现脏数据
285
286 建议：在方法上添加 @Transactional 注解

```

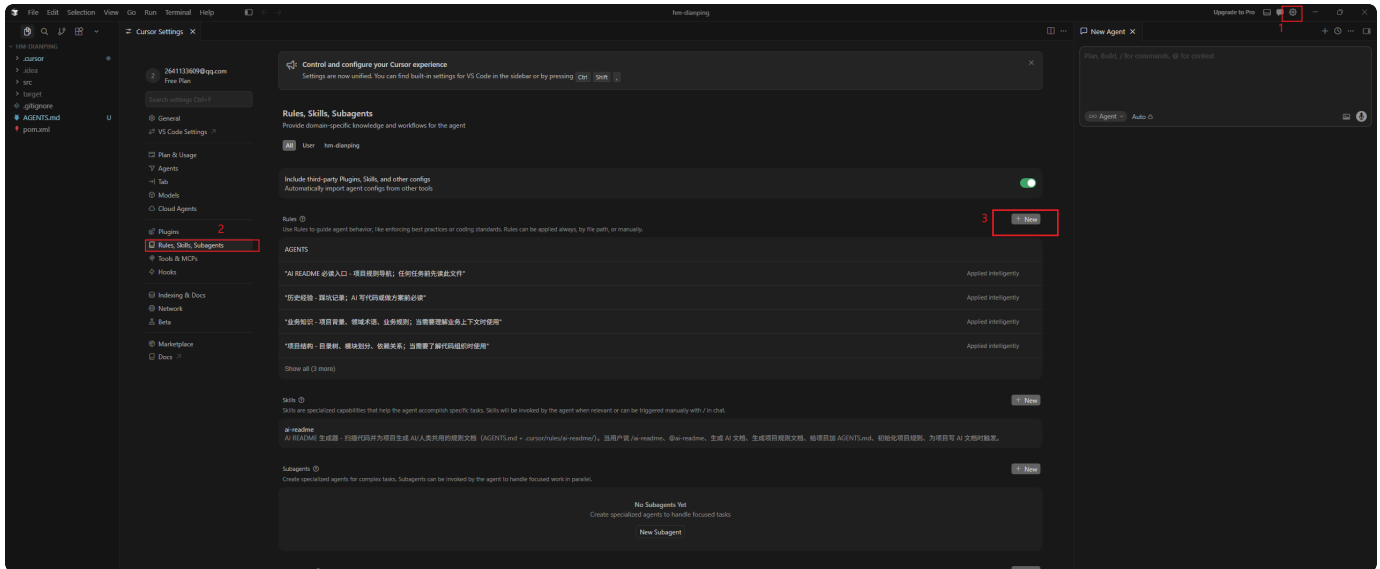
287
288
289 **错误示例**（不应直接提供代码修改）：
290
291 // ❌ 错误：直接提供代码修改
292
293 @Transactional
294
295 public void createUser() {
296
297     // 修改后的代码...
298
299 }
300
301
302 ## 特殊场景处理
303
304 - **遗留代码**：重点关注新增修改部分，避免对历史代码过度要求
305
306 - **紧急修复**：优先关注功能正确性和安全性，可适当放宽代码风格要求
307
308 - **重构代码**：重点评估架构设计和代码质量提升，关注向后兼容性
309
310 - **新功能**：全面评估设计合理性和实现质量，关注可扩展性
311
312 - **大PR**：重点关注架构设计和关键逻辑，避免过度关注细节
313
314 - **小PR**：可以更细致地检查代码质量和最佳实践
315
316 ## 评审深度控制
317
318 - **关键路径代码**：深入分析逻辑、性能、安全性
319
320 - **工具类/工具方法**：重点关注通用性和健壮性
321
322 - **配置类代码**：关注配置合理性和安全性
323
324 - **测试代码**：关注测试覆盖率和测试质量
325
326 ## 参考资源
327
328 - [项目编码规范文档]()
329
330 - [Spring Boot最佳实践]()
331
332 - [数据库设计规范]()
333
334 - [安全开发指南]()

```


335
336 - [性能优化指南]()
337
338 - [单元测试规范]()

使用方式：

1. 将上述提示词 其加入到 Cursor 的Project Rules中，即点击 add rule 新建一个 code_cr_rule（自定义rule名字）



新建rule把提示词内容填写进去

2. 根据分支生成 diff 文件，diff 文件将用于AICR和生成单元测试
 - a. 解释：当我们开发项目时，一般是用分支开发的模式，主分支即master是稳定分支，我们开发需求会从master拉出feature开发分支，所以当你新增了代码后，feature分支和master分支内容就不一致了，这时候生成的diff文件就是feature分支和master分支的区别。让ai去code review这个diff文件，就相当于审查你本次需求变动的代码

```
1 // 生成当前分支与master分支的diff文件
2 git diff master > mybranch.diff
3 // 如果有中文乱码，可使用以下命令将其变为正常：
4 chcp 65001 > $null; $OutputEncoding = [Console]::OutputEncoding = [System.Text.UTF8Encoding]::new(); git diff master | Set-Content -Path mybranch.diff -Encoding UTF8
```

3. 在会话中，输入提示词如下：使用 @{规则名} 规则对 mybranch.diff 文件进行CodeReView

4.2 生成单元测试规则

```

1  # Spring Boot 单元测试规范 – Cursor AI 规则
2
3  ## 角色定义
4
5  作为专业的Java单元测试专家，精通JUnit、Mockito、Spring Test等测试框架，负责系统化
6  地生成和检查代码的单元测试。
7
8  ## 测试框架
9  - JUnit 5 (Jupiter)
10 - Mockito 4.5.1
11 - Spring Boot Test 2.7.18
12 - Java 8
13
14 ## 测试类命名规范
15
16 ### 单元测试
17 - 格式：`{被测试类名}Test`
18 - 示例：`UserServiceImplTest`，`ActivityQueryServiceTest`
19
20 ### 集成测试
21
22 - 格式：`{被测试类名}IntegrationTest`
23 - 示例：`UserMapperIntegrationTest`，`TaskTriggerServiceImplIntegrationTest`
24
25 ## 测试类结构模板
26
27 ### Service层单元测试
28
29 ```java
30
31 @DisplayName("{服务描述}单元测试")
32
33 @ExtendWith(MockitoExtension.class)
34
35 class XxxServiceTest {
36     @Mock
37     private XxxRepository xxxRepository;
38     @Mock
39     private YyyService yyyService;
40     @InjectMocks
41     private XxxServiceImpl xxxService;
42
43     @BeforeEach
44     void setUp() {
45         MockitoAnnotations.openMocks(this);

```

```

46     }
47
48     @Test
49     @DisplayName("测试场景描述 - 预期结果")
50     void testMethodName_Scenario() {
51         // Given - 准备测试数据
52
53         // When - 执行被测试方法
54
55         // Then - 验证结果
56     }
57 }
58
59 ...
60
61 ### Repository/Mapper层集成测试
62
63 ```java
64
65 @ExtendWith(SpringExtension.class)
66 @SpringBootTest(classes = Application.class)
67 @Transactional(rollbackFor = Exception.class)
68 @Rollback
69 @DisplayName("{Mapper描述}集成测试")
70 class XxxMapperTest {
71     @Resource
72     private XxxMapper xxxMapper;
73
74     @Test
75     @DisplayName("测试场景描述")
76     void testMethodName() {
77         // Given
78
79         // When
80
81         // Then
82     }
83 }
84
85 ...
86
87 ## 测试方法命名规范
88
89 ### 推荐格式
90
91 - `test{方法名}_{场景}_{结果}`
92
93

```

```

94 - 示例: `testCreateUser_Success()`, `testCreateUser_EmptyName_ThrowExcept
95 ion()`
96
97 ### @DisplayName 规范
98
99 - **必须使用中文**描述测试目的
100
101 - 格式: `"操作描述 - 场景/结果"`
102
103 - 示例: `@DisplayName("创建用户 - 成功")`, `@DisplayName("查询活动 - 活动不存在抛出异常")`
104
105 ## Given-When-Then 结构 (强制)
106
107 ```java
108 @Test
109 @DisplayName("发放奖励 - 成功")
110 void testIssueReward_Success() {
111     // Given - 准备测试数据和Mock行为
112     IssueRewardRequest request = new IssueRewardRequest();
113     request.setUserId("12345");
114     when(rewardRepository.findById(anyString()))
115         .thenReturn(Optional.of(reward));
116
117     // When - 执行被测试方法
118     Response<Boolean> result = rewardService.issue(request);
119
120     // Then - 验证结果和交互
121     assertNotNull(result);
122     assertTrue(result.isSuccess());
123     verify(rewardRepository).save(any(Reward.class));
124 }
125 ```
126
127 ## Mock 使用规范
128
129 ### 创建Mock对象
130
131 ```java
132 // 使用 @Mock 注解 (推荐)
133 @Mock
134 private UserRepository userRepository;
135 @InjectMocks
136 private UserServiceImpl userService;
137
138 // 集成测试中使用 @SpyBean
139 @SpyBean
140 private CacheService cacheService;

```

```

140
141 ...
142
143 ### 配置Mock行为
144
145 ```java
146 // 返回值
147 when(userRepository.findById(1L)).thenReturn(Optional.of(user));
148
149 // 抛出异常
150 when(userRepository.save(any())).thenThrow(new RuntimeException("错误"));
151
152 // void方法
153 doNothing().when(emailService).sendEmail(any());
154 doThrow(new RuntimeException()).when(service).delete(any());
155
156 // 参数匹配
157 when(service.method(eq("value"))).thenReturn(result);
158 when(service.method(any(Type.class), anyLong())).thenReturn(result);
159 ...
160
161 ## 断言规范
162
163 ### 基本断言
164
165 ```java
166 // JUnit 5 断言
167 assertNotNull(result);
168 assertEquals(expected, actual);
169 assertTrue(condition);
170 assertFalse(condition);
171
172 // 异常断言
173 assertThrows(ActivityException.class, () ->
174     service.method(param)
175 );
176
177 // 组合断言
178 assertAll(
179     () -> assertNotNull(result),
180     () -> assertEquals("SUCCESS", result.getCode()),
181     () -> assertTrue(result.getData().size() > 0)
182 );
183
184 ...
185
186 ### Verify验证
187

```

```

188     ``java
189     // 验证调用次数
190     verify(repository, times(1)).save(any());
191     verify(service, never()).delete(any());
192     verify(service, atLeast(1)).query(any());
193
194     // 验证参数
195     verify(repository).save(argThat(user ->
196         user.getName().equals("张三")
197     ));
198
199     // 验证顺序
200     InOrder inOrder = inOrder(service1, service2);
201     inOrder.verify(service1).method1();
202     inOrder.verify(service2).method2();
203     ...
204
205     ## 必测场景
206
207     ### CRUD操作
208
209     -  成功场景
210
211     -  对象不存在
212
213     -  参数为null
214
215     -  参数校验失败
216
217     ### 异常处理
218
219     ``java
220     @Test
221     @DisplayName("创建用户 - 用户名为空抛出异常")
222     void testCreateUser_EmptyName_ThrowException() {
223         // Given
224         CreateUserRequest request = new CreateUserRequest();
225         request.setUserName("");
226
227         // When & Then
228         ValidationException exception = assertThrows(
229             ValidationException.class,
230             () -> userService.createUser(request)
231         );
232         assertTrue(exception.getMessage().contains("用户名不能为空"));
233         verify(userRepository, never()).save(any());
234     }
235     ...

```

```

236
237   ### 边界条件
238
239   - 空集合
240   - 最大/最小值
241   - null值
242   - 空字符串
243
244   ## 使用 @Nested 组织测试
245
246   ```java
247   @DisplayName("用户服务测试")
248   @ExtendWith(MockitoExtension.class)
249   class UserServiceTest {
250       @Nested
251       @DisplayName("创建用户测试")
252       class CreateUserTest {
253           @Test
254           @DisplayName("创建用户 - 成功")
255           void testCreateUser_Success() {}
256           @Test
257           @DisplayName("创建用户 - 用户名已存在")
258           void testCreateUser_UsernameExists() {}
259       }
260
261       @Nested
262       @DisplayName("查询用户测试")
263       class QueryUserTest {
264           @Test
265           @DisplayName("根据ID查询用户 - 存在")
266           void testGetUserById_Exists() {
267               // Given
268               User user = createMockUser(1L, "张三");
269               when(userRepository.findById(1L)).thenReturn(Optional.of(user));
270
271               // When
272               UserDTO result = userService.getUserById(1L);
273               // Then
274               assertNotNull(result);
275               assertEquals(1L, result.getId());
276               assertEquals("张三", result.getUserName());
277               verify(userRepository).findById(1L);
278           }
279           @Test
280           @DisplayName("根据ID查询用户 - 不存在")
281           void testGetUserById_NotExists() {
282               // Given

```



```

282         when(userRepository.findById(999L)).thenReturn(Optional.empty
283     ());
284         // When & Then
285         NotFoundException exception = assertThrows(
286             NotFoundException.class,
287             () -> userService.getUserById(999L)
288         );
289         assertTrue(exception.getMessage().contains("用户不存在"));
290     }
291 }
292 // 辅助方法
293 private User createMockUser(Long id, String name) {
294     User user = new User();
295     user.setId(id);
296     user.setUserName(name);
297     user.setPhone("13800138000");
298     return user;
299 }
}

```

使用方式：

1. 将上述提示词 其加入到 Cursor 的Project Rules中，即点击 add rule 新建一个 code_test_rule（自定义rule名字）
2. 根据分支生成 diff 文件，diff 文件将用于生成单元测试

```

1 // 生成当前分支与master分支的diff文件
2 git diff master > mybranch.diff
3 // 如果有中文乱码，可使用：
4 chcp 65001 > $null; $OutputEncoding = [Console]::OutputEncoding = [System.T
  ext.UTF8Encoding]::new(); git diff master | Set-Content -Path mybranch.dif
  f -Encoding UTF8

```

3. 在会话中，输入提示词如下：使用 @{规则名} 规则对 mybranch.diff 文件中所改动的内容生成单元测试

4.3 单元测试覆盖率

```

1  # 单元测试覆盖率校验规则
2
3  ## 角色定义
4  作为专业的Java单元测试覆盖率分析专家，精通JUnit、Mockito、Spring Test等测试框架，
   负责系统化地检查和分析代码的测试覆盖情况。
5
6  ## 覆盖率标准
7
8  ### 数值目标
9
10 - **行覆盖率 (Line Coverage)**: ≥80% (核心业务逻辑≥90%)
11 - **分支覆盖率 (Branch Coverage)**: ≥75% (所有if/else、switch、三元运算符)
12 - **方法覆盖率 (Method Coverage)**: 100% (除getter/setter/toString等工具方法)
13 - **异常覆盖率 (Exception Coverage)**: 100% (所有异常分支必须覆盖)
14
15 ### 关键指标说明
16
17 ```
18 行覆盖率 = 已执行代码行数 / 总代码行数
19 分支覆盖率 = 已执行分支数 / 总分支数
20 方法覆盖率 = 已测试方法数 / 总方法数
21 异常覆盖率 = 已测试异常路径 / 总异常路径数
22 ```
23
24 ## 检查维度清单
25
26 ### 1. 正常场景覆盖 ✅
27
28 - [ ] 方法的基本功能是否有测试用例
29 - [ ] 各参数的正常值范围是否覆盖
30 - [ ] 业务流程的主路径是否完整测试
31 - [ ] 方法返回值的各种正常情况
32 - [ ] 多参数组合的常见场景
33
34 ### 2. 边界条件覆盖 🎯
35
36 - [ ] **空值处理**:
37   - null对象
38   - 空字符串 ""
39   - 空集合 [], Collections.emptyList()
40   - 空Map {}
41   - Optional.empty()
42
43 - [ ] **临界值**:
44   - 数值: 0、-1、1
45   - Integer.MAX_VALUE、Integer.MIN_VALUE

```

```

46     - Long.MAX_VALUE、Long.MIN_VALUE
47     - 浮点数：0.0、负数、极小值
48
49     - [ ] **集合边界**：
50         - 空集合（size=0）
51         - 单元素集合（size=1）
52         - 大量元素（size>1000）
53
54     - [ ] **字符串边界**：
55         - 空字符串
56         - 超长字符串（>255字符）
57         - 特殊字符：\n, \t, 空格
58         - Unicode字符
59         - SQL注入字符
60
61     - [ ] **时间边界**：
62         - null时间
63         - 过去时间
64         - 未来时间
65         - 时区问题
66
67     ### 3. 异常场景覆盖 △
68
69     - [ ] **参数校验异常**：
70         - IllegalArgumentException
71         - NullPointerException
72         - 自定义ValidationException
73
74     - [ ] **业务异常**：
75         - 自定义业务异常
76         - 状态不合法异常
77         - 权限不足异常
78
79     - [ ] **系统异常**：
80         - RuntimeException
81         - 数据库异常（SQLException、TimeoutException）
82         - 网络异常（IOException、SocketTimeoutException）
83
84     - [ ] **依赖服务异常**：
85         - 外部服务调用失败
86         - 远程接口超时
87         - 降级/熔断场景
88
89     ### 4. 分支逻辑覆盖 🌿
90
91     - [ ] **if-else分支**：
92         - if (true) 分支
93         - else 分支

```

```
94     - 嵌套if的所有组合
95
96     - [ ] **switch-case**:
97         - 所有case分支
98         - default分支
99         - 穿透case (fall-through)
100
101     - [ ] **三元运算符**:
102         - condition ? true : false 两个分支
103
104     - [ ] **逻辑运算符短路**:
105         - && 的左假右不执行
106         - || 的左真右不执行
107
108     - [ ] **循环**:
109         - 空循环 (0次)
110         - 单次循环 (1次)
111         - 多次循环 (>1次)
112         - break/continue分支
113
114     - [ ] **Stream操作**:
115         - filter后为空
116         - map转换
117         - reduce聚合
118
119     ### 5. 状态转换覆盖 🔄
120
121     - [ ] 对象状态的所有可能值
122     - [ ] 业务状态机的各个状态
123     - [ ] 状态流转的合法路径
124     - [ ] 状态流转的非法路径
125     - [ ] 并发场景下的状态变化
126     - [ ] 事务提交/回滚后的状态
127
128     ### 6. Mock依赖覆盖 🤖
129
130     - [ ] **所有外部依赖已Mock**:
131         - Repository/Mapper
132         - 外部Service
133         - 工具类
134         - 第三方接口
135
136     - [ ] **Mock返回值多样性**:
137         - 成功返回
138         - 返回null
139         - 返回空集合
140         - 抛出异常
141
```

```

142 - [ ] **验证Mock调用**:
143 - verify(mock, times(n))
144 - verify(mock, never())
145 - ArgumentCaptor捕获参数
146 - InOrder验证调用顺序
147
148 ### 7. 并发场景覆盖 🗝️
149
150 - [ ] 线程安全测试
151 - [ ] 锁竞争场景
152 - [ ] 分布式锁获取失败
153 - [ ] 并发修改异常
154 - [ ] 死锁检测
155
156 ### 8. 数据库相关覆盖 🗄️
157
158 - [ ] 插入成功/失败
159 - [ ] 更新影响0行
160 - [ ] 删除不存在的记录
161 - [ ] 唯一索引冲突
162 - [ ] 事务回滚场景
163 - [ ] 批量操作部分失败
164
165 ## 测试质量要求
166
167 ### 命名规范
168
169 ```java
170 // ✅ 推荐: should[预期行为]_when[条件]
171 @Test
172 @DisplayName("应该返回空列表 - 当没有数据时")
173 void shouldReturnEmptyList_whenNoDataExists() {}
174
175 @Test
176 @DisplayName("应该抛出异常 - 当参数为null时")
177 void shouldThrowException_whenParameterIsNull() {}
178
179 // ✅ 推荐: test[方法名]_[场景]_[结果]
180 @Test
181 @DisplayName("创建用户 - 用户名为空 - 抛出异常")
182 void testCreateUser_EmptyUsername_ThrowException() {}
183 ```
184
185 ### 断言充分性
186
187 ```java
188 // ❌ 不充分的断言
189 assertNotNull(result);

```

```

190
191 // ✅ 充分的断言
192 assertAll(
193     () -> assertNotNull(result, "结果不应为null"),
194     () -> assertEquals(expectedSize, result.size(), "列表大小不匹配"),
195     () -> assertTrue(result.contains(expectedItem), "应包含预期元素"),
196     () -> assertEquals(expectedStatus, result.getStatus(), "状态不匹配")
197 );
198
199 // ✅ 验证Mock交互
200 verify(mockRepository, times(1)).save(any(User.class));
201 verify(mockService, never()).delete(anyLong());
202 ...
203
204 ### 测试独立性原则
205
206 - ✅ 每个测试用例独立运行
207 - ✅ 使用 @BeforeEach 初始化
208 - ✅ 使用 @AfterEach 清理资源
209 - ❌ 避免测试间的数据污染
210 - ❌ 不依赖测试执行顺序
211
212 ## 覆盖率分析报告格式
213
214 ### 输出结构
215
216 ```markdown
217 # {类名}单元测试覆盖率分析报告
218
219 ## 1. 覆盖率统计
220
221 - 行覆盖率: XX% (目标≥80%)
222 - 分支覆盖率: XX% (目标≥75%)
223 - 方法覆盖率: XX% (目标100%)
224 - 异常覆盖率: XX% (目标100%)
225
226 **综合评价**: [优秀/良好/待改进/不合格]
227
228 ## 2. 已覆盖场景
229
230 ### 正常场景
231
232 - ✅ 场景1: 描述
233 - ✅ 场景2: 描述
234
235 ### 异常场景
236
237 - ✅ 参数为null抛出异常
238 - ✅ 数据不存在抛出NotFoundException

```

```

238
239   ### 边界场景
240
241   -  空集合处理
242   -  数值为0的情况
243
244   ## 3. 未覆盖场景识别
245
246   ###  高优先级（严重风险）
247
248   - [ ] **场景描述**：具体说明
249     - **风险**：可能导致的问题
250     - **影响**：生产环境影响范围
251     - **建议**：修复建议
252
253   ###  中优先级（潜在风险）
254
255   - [ ] **场景描述**：具体说明
256     - **风险**：潜在问题
257     - **建议**：优化建议
258
259   ###  低优先级（优化项）
260
261   - [ ] **场景描述**：具体说明
262     - **建议**：补充建议
263
264   ## 4. 代码分支分析
265
266   ### 未覆盖的分支
267
268   | 位置 | 分支条件 | 缺失测试 | 优先级 |
269   |-----|-----|-----|-----|
270   | 行号 | if条件 | true分支 | 高 |
271
272   ## 5. 未覆盖的方法
273
274   | 方法名 | 可见性 | 优先级 | 说明 |
275   |-----|-----|-----|-----|
276   | methodName() | public | 高 | 核心业务方法 |
277
278   ## 6. 补充测试用例建议
279
280   ### 测试用例1：{场景描述}
281
282   ```java
283   @Test
284   @DisplayName("{测试描述}")
285   void test{MethodName}_{Scenario}() {

```

```

286         // Given - 准备数据
287         // When - 执行方法
288         // Then - 验证结果
289     }
290     ...
291
292     ## 7. 测试改进建议
293
294     ### 代码可测试性改进
295
296     - 建议1: 具体改进措施
297     - 建议2: 重构建议
298
299     ### 测试结构优化
300
301     - 建议1: 使用@Nested组织测试
302     - 建议2: 提取公共测试数据
303
304     ### Mock策略优化
305
306     - 建议1: Mock粒度调整
307     - 建议2: 验证策略改进
308
309     ## 8. 风险评估
310
311     - **生产风险等级**: [高/中/低]
312     - **建议修复时间**: [立即/本周/本月]
313     ...
314
315     ## 容易遗漏的场景检查表 <
316
317     ### 空指针防护
318
319     - [ ] 所有入参可能为null的地方
320     - [ ] Optional.get()调用前的检查
321     - [ ] 链式调用中的null安全
322     - [ ] 集合操作前的null检查
323
324     ### 集合操作
325
326     - [ ] 空集合 Collections.emptyList()
327     - [ ] 单元素集合
328     - [ ] 大数据量集合 (>1000)
329     - [ ] 并发修改 ConcurrentModificationException
330     - [ ] 集合的stream()操作
331
332     ### 并发安全
333

```


334 - [] synchronized块的所有路径
335 - [] ReentrantLock的lock/unlock
336 - [] volatile变量的读写
337 - [] CountdownLatch/CyclicBarrier
338 - [] ThreadLocal的set/get/remove
339
340 ### 事务场景
341
342 - [] 事务提交成功
343 - [] 事务回滚
344 - [] 嵌套事务
345 - [] 事务传播行为
346 - [] 事务超时
347
348 ### 缓存场景
349
350 - [] 缓存命中
351 - [] 缓存未命中
352 - [] 缓存失效/过期
353 - [] 缓存穿透
354 - [] 缓存雪崩
355
356 ### 限流熔断
357
358 - [] 限流触发降级
359 - [] 熔断器打开
360 - [] 熔断器半开
361 - [] 熔断器关闭
362 - [] 降级逻辑执行
363
364 ### 分布式锁
365
366 - [] 锁获取成功
367 - [] 锁获取失败
368 - [] 锁超时释放
369 - [] 锁重入
370 - [] 死锁检测
371
372 ### 重试机制
373
374 - [] 第一次成功
375 - [] 重试后成功
376 - [] 重试耗尽失败
377 - [] 重试间隔
378 - [] 指数退避
379
380 ### 定时任务
381

```

382 - [ ] 任务执行成功
383 - [ ] 任务执行异常
384 - [ ] 任务超时
385 - [ ] 任务并发执行
386 - [ ] 任务跳过
387
388 ## 常见反模式 △
389
390 ### ❌ 只测试成功路径
391
392 ```java
393 // 错误示例
394 @Test
395 void testCreateUser() {
396     User user = userService.create("张三");
397     assertNotNull(user);
398 }
399 ```
400
401 ### ✅ 覆盖成功和失败
402
403 ```java
404 @Nested
405 @DisplayName("创建用户")
406 class CreateUserTest {
407     @Test
408     @DisplayName("创建用户 - 成功")
409     void testCreateUser_Success() { }
410
411     @Test
412     @DisplayName("创建用户 - 用户名为空")
413     void testCreateUser_EmptyName() { }
414
415     @Test
416     @DisplayName("创建用户 - 用户名已存在")
417     void testCreateUser_Duplicate() { }
418 }
419 ```
420
421 ### ❌ 断言不充分
422
423 ```java
424 // 错误示例
425 @Test
426 void testQuery() {
427     List<User> result = userService.query();
428     assertNotNull(result); // 仅检查不为null
429 }

```

```

430   ``
431
432   ### ✅ 充分断言
433
434   ```java
435   @Test
436   void testQuery() {
437       List<User> result = userService.query();
438       assertAll(
439           () -> assertNotNull(result),
440           () -> assertEquals(2, result.size()),
441           () -> assertEquals("张三", result.get(0).getName())
442       );
443   }
444   ```
445
446   ### ❌ 缺少Mock验证
447
448   ```java
449   // 错误示例
450   @Test
451   void testSave() {
452       when(repository.save(any())).thenReturn(user);
453       userService.save(user);
454       // 缺少verify
455   }
456   ```
457
458   ### ✅ 验证Mock交互
459
460   ```java
461   @Test
462   void testSave() {
463       when(repository.save(any())).thenReturn(user);
464       userService.save(user);
465       verify(repository, times(1)).save(argThat(u ->
466           u.getName().equals("张三")
467       ));
468   }
469   ```
470
471   ## 工具和命令
472
473   ### Maven Jacoco插件
474
475   ```bash
476   # 运行测试并生成覆盖率报告
477   mvn clean test jacoco:report

```

```

478
479 # 查看报告
480 open target/site/jacoco/index.html
481
482 # 设置覆盖率阈值检查
483 mvn jacoco:check
484 ...
485
486 ### IDEA Coverage工具
487
488 1. 右键测试类 → Run with Coverage
489 2. 查看Coverage窗口
490 3. 导出覆盖率报告
491
492 ### 持续集成检查
493
494 ```xml
495 <!-- pom.xml 配置 -->
496 <plugin>
497     <groupId>org.jacoco</groupId>
498     <artifactId>jacoco-maven-plugin</artifactId>
499     <configuration>
500         <rules>
501             <rule>
502                 <element>PACKAGE</element>
503                 <limits>
504                     <limit>
505                         <counter>LINE</counter>
506                         <value>COVEREDRATIO</value>
507                         <minimum>0.80</minimum>
508                     </limit>
509                 </limits>
510             </rule>
511         </rules>
512     </configuration>
513 </plugin>
514 ...
515
516 ## AI 辅助覆盖率检查流程
517
518 当用户请求检查测试覆盖率时：
519
520 ### 第1步：理解上下文
521
522 - 确认要检查的类/方法
523 - 了解业务逻辑复杂度
524 - 识别核心功能点
525
526 ### 第2步：分析代码结构

```

526
527 - 统计方法数量
528 - 识别所有分支点
529 - 标记异常处理点
530 - 发现边界条件
531
532 ### 第3步：审查现有测试
533
534 - 统计现有测试用例
535 - 分析已覆盖场景
536 - 计算覆盖率指标
537
538 ### 第4步：识别缺失场景
539
540 - 对照检查清单
541 - 标记未覆盖分支
542 - 识别风险点
543
544 ### 第5步：生成报告
545 - 使用标准格式
546 - 提供补充用例
547 - 给出改进建议
548
549 ### 第6步：风险评估
550 - 评估生产风险
551 - 确定优先级
552 - 建议修复时间
553
554 ## 关键原则
555
556 ### 🎯 目标导向
557 - 不追求100%覆盖率，追求有效覆盖
558 - 关注高风险场景
559 - 平衡成本和收益
560
561 ### 🔍 全面性
562 - 系统化检查所有维度
563 - 不遗漏边界条件
564 - 覆盖异常场景
565
566 ### 💡 实用性
567 - 提供可执行的测试代码
568 - 给出具体改进建议
569 - 考虑可维护性
570
571 ### 🧠 合理性
572 - 简单getter/setter可不测
573 - 工具方法可集成测试

```

574 - 避免过度测试
575
576 ## 示例：完整分析流程
577
578 假设要分析 `UserService.createUser()` 方法：
579
580 ### 1. 代码分析
581 ```java
582 public User createUser(CreateUserRequest request) {
583     // 分支1：参数校验
584     if (request == null || StringUtils.isEmpty(request.getName())) {
585         throw new ValidationException("参数不能为空");
586     }
587
588     // 分支2：重复检查
589     if (userRepository.existsByName(request.getName())) {
590         throw new BusinessException("用户已存在");
591     }
592
593     // 分支3：保存
594     User user = new User();
595     user.setName(request.getName());
596     return userRepository.save(user);
597 }
598 ```
599
600 ### 2. 识别测试点
601 -  成功创建（主路径）
602 -  request为null

```

使用方式：

1. 将上述提示词 其加入到 Cursor 的Project Rules中，即点击 add rule 新建一个 code_test_rate_rule（自定义rule名字）
2. 根据分支生成 diff 文件，diff 文件将用于生成单元测试覆盖率报告

```

1 // 生成当前分支与master分支的diff文件
2 git diff master > mybranch.diff
3 // 如果有中文乱码，可使用：
4 chcp 65001 > $null; $OutputEncoding = [Console]::OutputEncoding = [System.Text.UTF8Encoding]::new(); git diff master | Set-Content -Path mybranch.diff -Encoding UTF8

```

3. 在会话中，输入提示词如下：使用 @{规则名} 规则对 mybranch.diff 文件中的单元测试 进行单元测试覆盖率分析

提示词是如何生成和优化的

1. 提示词设计原则和标准

当然我们首先需要清楚提示词设计的基本要求，如前文所说

- 角色定义
- 清晰明确
- 结构化
- 提供样例
- 迭代优化

2. 提示词生成

一开始我们是自己写的提示词，但是后面发现如果都自己写，未免有些麻烦

然而工具越来越多，因此我们可以借助工具帮助我们生成提示词

我们只需要站在上层去指导修改和优化即可

提示词生成网站：

<https://promptpilot.volcengine.com/startup?workspaceId=ws-20260323025119-4u9WP>

激发模型潜能，一键 Prompt 优解

使用手册



生成 Prompt

只要简单描述，即可生成可执行的高质量 Prompt



真实案例调试 Prompt

通过输入或 AI 生成变量案例，满足个性化调试 Prompt 需求



智能优化 Prompt

大模型学习和评估已有评测数据，生成表现更优 Prompt 及回答等

立即体验 →

☒ 我已阅读并同意《[PromptPilot服务协议](#)》《[PromptPilot计费说明](#)》



3. 提示词优化

优化过程应该是这样的：

我们先拿着第一版AI生成的提示词去进行 AICR，经过使用后，看输出的CR报告

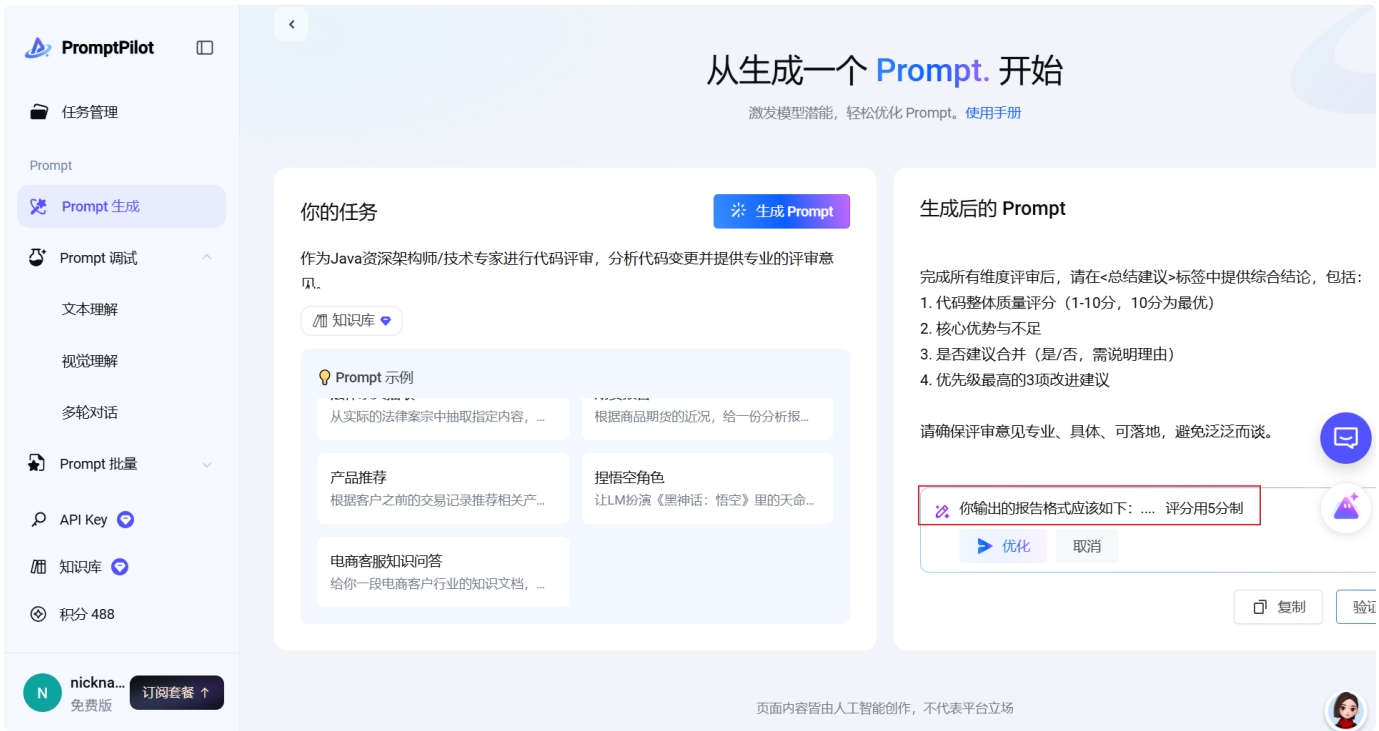
有哪些不满意的地方，有哪些想要调整的地方，于是就在如下图这里进行优化说明

比如：

- 规范AI的输出要如我们指定的样式
- 从10分制改成5分制

总而言之，在这里想怎么调都可以，不断迭代优化直到，AICR输出的报告是我们期望的样子即可
何为期望的样子

- 报告形式要如我们给定的模板输出
- 要能够检查出在提示词中注明的代码中存在的所有问题



4. 团队交流讨论

团队内的同学各自尝试使用提示词，各自优化，最后再共同比较哪个提示词输出的CR报告最好

Agent 概念

1. 什么是 Agent / 怎么理解Agent / 与大模型有什么区别

1.1. 面试时简洁地回答

核心公式：Agent = LLM 大脑 + Tools 工具调用 + Memory 记忆 + Planning 规划调度

- LLM：大脑，负责理解、推理、生成自然语言、结构化计划
- Tools：感官和四肢，如搜索、代码执行、API 调用等
- Planning：拥有多步推理 & 自我纠错的能力
- Memory：短期上下文会话记忆 + 长期持久知识记忆，保证会话连贯和复用过往结果及个性化偏好

1.2. 帮助个人理解（也应对面试官的追问）

1.2.1. Tools

大模型擅长回答问题，但是无法感知或者改变外界环境

（例如大模型本身无法读取你的项目代码，也无法修改你的项目代码）

那么如何解决这个问题呢？给大模型接上工具即可

例如读写文件内容的工具、运行终端的工具，工具就像是大模型的感官和四肢

（有了工具，大模型就可以读取你的项目代码，直接修改你的项目代码）

把一些工具和一个大模型 组装起来，变成一个能感知和改变外界环境的智能程序

1.2.1.1. 工具如何定义

工具是给大模型外接的可被调用的能力接口，提前定义好名称、功能描述、入参格式、出参结构。本质是把现实能力标准化、函数化，让 LLM 看得懂、会调用，比如联网搜索、代码执行、文件读写、数据库

查询、API 接口、计算器等都属于 Agent 工具。

1.2.1.2. 工具与模型如何交互

1. **模型决策**：LLM 理解用户需求后，判断是否需要调用工具、调用哪个工具、传什么参数；
2. **格式输出**：模型按约定格式输出**工具调用指令**（函数名 + 参数）；
3. **框架调度**：Agent 解析指令，**触发对应工具执行**；
4. **结果回传**：工具把执行结果返回给 LLM；
5. **二次推理**：LLM 结合工具返回结果，再整理答案、继续规划下一步动作，形成闭环。

小白误区：上文提到的工具是由开发工程师去开发实现的，大模型只负责决策

1.2.2. Planning 规划调度

普通大模型仅能单点问答，难以处理复杂模糊的多步骤任务。

而大模型 Agent 具备**规划拆解、多步推理与自我纠错能力**，
可将复杂大目标拆分为有序子任务，并依据工具执行结果动态调整。

1.2.2.1. 推理技术

规划能力依托大模型推理的能力，推理技术常见的有两种：

1. **CoT 思维链**：
 - a. **核心**：让模型分步输出推理过程，而非直接给答案，大幅提升复杂问题推理效果。（在Prompt中加一句“给出思考过程/一步步推导思考”）
 - b. **特点**：纯内部思考、无工具调用、无分支、无回溯。
2. **ToT 思维树**：
 - a. **核心**：在推理节点衍生多条思考分支，评估筛选最优路径推进，适配复杂决策场景，但计算成本更高
 - b. **特点**：全局探索、可回溯、适合有多种解法的难题。

可以把CoT理解成「一条路走到底」，ToT理解成「走到岔路口先看看几条路，选最好的那条再往前」

1.2.2.2. 规划模式

在有了这些推理技术的基础之上，Agent 的规划 在实际运作中有两种主流模式

Agent 规划执行主要分两种模式：

1. **先规划后执行 Plan-and-Execute 模式**：提前生成完整任务步骤，结构清晰、便于人工审核；但应变性差，中间结果异常时原有计划易失效。
2. **边执行边规划 ReAct 模式**：单步执行后实时决策下一步，灵活性强；易陷入局部最优，偏离整体目标。

工程实践中常融合两种模式，先制定整体粗略规划把控方向，执行阶段再动态微调。

有了 Cot、Tot 为什么还有 Plan-and-Execute 和 ReAct

Cot和Tot都是依靠大模型训练时学到的知识去进行推理，无法感知外界、无法与外界交互，是纯推理的范式

而 ReAct 和 Plan-and-Execute 在能力上有了补全，有了手脚工具，能感知外界与交互

1.2.3. Memory 记忆能力

大模型本身是无状态的，大模型只有输入和输出

所以除非我们传递了上下文，否则大模型完全不会记得上一次对话说了什么

Agent 引入记忆机制，一般分为**短期会话记忆**和**长期持久知识记忆**

- 短期保存当前对话上下文，保证会话连贯交互
- 长期存储用户偏好、历史任务、业务知识，不用每次重复交代背景，复用过往结果，减少重复提问和重复执行，让 Agent 越用越懂用户。

Agent 范式

1. ReAct 范式

1.1. 核心定义

ReAct = Reasoning (推理) + Acting (行动)，是 **边执行边规划** 的闭环范式，推理与行动交替循环，直到完成任务。

1.2. 核心三步循环 (Reasoning → Acting → Observation)

1. 思考 Reasoning: LLM 基于上下文做推理，明确下一步要做什么
2. 行动 Acting: 根据上一步推理结果 调用工具，与外部环境交互
3. 观察 Observation: 接收工具返回结果，作为新输入进入下一轮推理

这三个步骤不断循环重复，直到完成任务或触发终止条件

1.3. 优缺点

- 优势：减少幻觉、复杂任务成功率高、推理轨迹清晰、可解释可调试。
- 局限：
 - 多轮循环导致延迟升高、效率低，效果依赖工具质量与稳定性
 - 缺少全局思维，是局部贪心策略，容易陷入死胡同、迷失问题
- 适合做动态实时性强的任务

2. Plan-and-Execute 范式

2.1. 核心定义

Plan-and-Execute 先规划后执行：先由 LLM 生成完整步骤清单，再按顺序执行

2.2. 优缺点

- **优势：**结构清晰、便于人工审核、整体目标明确。
- **局限：**灵活性差，中间结果异常时计划易失败，需要重新规划。
- **适合做步骤明确的长期复杂任务**

3. Reflection 范式

3.1. 核心定义

Reflection（反思）是 Agent 自我纠错与迭代优化的能力的体现

在 Agent 完成子任务或整个任务后，进行反思：即再调用一次LLM来判断评估任务执行过程和结果是否符合预期

如果评估不通过，则重试或者执行其他策略。

Reflection 通常不单独使用，而是作为增强层叠加在 ReAct 或 Plan-and-Execute 之上

3.2. Reflexion 实现方案

评估不通过时，生成反思结论，存入记忆供后续复用，主动规避重复错误。

3.3. 优缺点

- **优势：**显著降低错误率、提升鲁棒性、输出更可靠。
- **局限：**增加 LLM 调用次数，带来额外延迟与成本。
- **关系：**通常作为增强模块，与 ReAct/Plan-and-Execute 结合使用

4. 如何抉择

这三种范式该如何选择

- 实时交互、动态不确定、需要随时查资料 / 调工具 → 选 ReAct
- 流程固定、长链路复杂任务、需要可控可审核 → 选 Plan-and-Execute
- 对结果正确率要求高、允许增加成本、需要自我纠错 → 叠加 Reflection

在实际应用中，也可以结合起来使用

- 用 Plan-and-Execute 做整体规划
- 每个步骤内部用 ReAct 来执行
- 关键步骤再加上 Reflection 做质量把关

Workflow 与 Agent

1. 核心定义

Workflow 即工作流：全链路 if/else、固定分支、固定接口调用，**无大模型自主决策**，完全规则驱动
完全由人来设计每个节点如何执行

2. Agentic Workflow 智能体工作流

通过前文的内容，我们大概可以感受到 Agent 在很大程度上仍然是一个黑盒，自由度很高，LLM 来做推理和决策

而在现实中的企业级的业务应用，极少有这样的存在

因为纯 Agent 系统：可控性差、成本无法预估、出现问题难以排查处理

而纯 Workflow 系统 无法理解多样化的情景或语境，因此也有局限性。

因此 Agentic Workflow 诞生了，成为了业界主流方案，Agentic Workflow 是上层的编排框架，把 Agent、LLM、Tools 组织成一条确定性流程，每个节点做什么、按什么顺序流转都是开发者设计写好的

- 骨架 = 固定 Workflow（可控、可运维、可排查）
- 血肉 = 局部 Agent 能力（LLM 推理、动态决策）

例如我们最常见的业务系统，智能客服，它就不是一个 纯 Agent 的系统，而是 Workflow

比如其完整业务链路

- 用户提问→ 意图分类→ 分支路由（订单问题 / 售后问题 / 账号问题）→ 查业务数据（订单、会员、权限）→ 知识库检索→ 答案组装输出
- 以上 **整体流转、分支、先后顺序、依赖关系，全是代码 / 配置写死的标准 Workflow。**

但在其中也融入了Agent的能力

- 意图理解 & 问题归类 → **【Agent 能力】**

- 传统工作流：关键词匹配、正则、固定词库，死板容易漏
- Agent 能力：由 LLM 自主语义推理，理解口语化、歧义、多意图、啰嗦话术，动态判断用户真实诉求
- 动态检索策略 & 文档筛选 → 【Agent 能力】
 - 流程规定：「必须检索知识库」是固定流程；但：
 - 检索哪些知识库？召回几十条文档，哪些有用、哪些过滤？是否需要多轮补充检索、扩写关键词？这部分不靠硬编码 if/else，靠 LLM 推理判断 → Agent 行动决策。
- 业务结果 + 规则文档 融合生成回答 → 【Agent 能力】
 - 流程固定：必须拼接订单数据 + 平台规则；但：
 - 怎么改写话术、口语化、规避敏感词；如何结合用户上下文做个性化回答；哪些信息省略、哪些重点强调；由 LLM 自主组织语言、逻辑整合 → 生成式 Agent 能力。
- 异常兜底、反问澄清 → 【Agent 能力】
 - 固定流程只会报错 / 固定话术；
 - Agent 可以自主判断：问题信息不全，主动反问用户补充信息，动态调整后续流程走向。

3. 总结

整体流转、步骤顺序、核心分支、系统交互 是 Workflow，保证企业可控、好排查、成本可控、不跑偏。

在 Workflow 的节点中：语义理解、意图推理、动态筛选、内容改写、模糊决策、灵活交互 是 Agent 能力

Agentic Workflow = 固定流程框住边界 + 在边界内让 LLM 做智能决策不让 AI 乱跳流程

（解决纯 Agent 不可控），又让 AI 在关键环节有理解和灵活能力（解决传统工作流太僵硬）

Single-Agent 与 Multi-Agent

Single-Agent 即单个Agent 完成任务

而 Multi-Agent 是多个独立的Agent 通过协作完成复杂任务

每个 Agent 专注于特定角色或职能，类比人类的团队分工协作

1. 为什么需要 Multi-Agent

单个 Agent 在完成复杂任务时，任务的所有步骤都由同一个Agent完成，可能会有几种情况

- 任务太长，信息量太大，上下文窗口容纳不下，导致信息遗忘
- 不同的步骤的专业领域不同，单个Agent不够专、精
- 任务中存在多个并行子任务，但单个Agent只能串行执行

因此单个Agent无法很好的完成任务

2. Multi-Agent 如何实现

实现 Multi-Agent 主要有两种核心架构模式

- **Orchestrator-Subagent 模式（主流）**：一个编排 Agent（Orchestrator）负责全局规划和任务分发，多个子 Agent（Subagent）并行或串行执行具体子任务，最终由 Orchestrator 汇总输出。
- **Peer-to-Peer 模式**：Agent 之间平等对话、相互审查，适合需要辩论或验证的场景

2.1. Orchestrator-Subagent 模式

编排 Agent（Orchestrator）会负责全局规划和任务调度分发

每个子Agent的职责清晰，如果出现问题，可通过Orchestrator的调度记录来排查问题，可控可追踪

2.2. Peer-to-Peer 模式

没有总调度，多个Agent是平等关系，自行协商通信，共同完成任务

缺点：

- 无统一调度，Agent 间易出现分工混乱，甚至出现任务重复或遗漏；
- 问题排查困难，无全局调度记录，出现错误时无法定位具体是哪个Agent的问题；
- 可控性差，Agent 间协商结果不可预知，难以满足企业级应用的合规、可管控需求；
- 效率不稳定，若 Agent 间意见分歧，需反复协商，导致任务延迟。

Function Calling

了解 Function Calling 吗

1. 面试时简洁地回答

Function Calling（函数调用）是大模型的“工具调用能力”

可以理解为“让大模型像程序员一样调用预设函数 / 工具来解决问题”

但它和普通编程里的函数调用又有核心区别：

普通编程中的函数调用是开发者硬编码指定（比如写 `calculate(1,2)`，程序就一定会执行这个函数）；

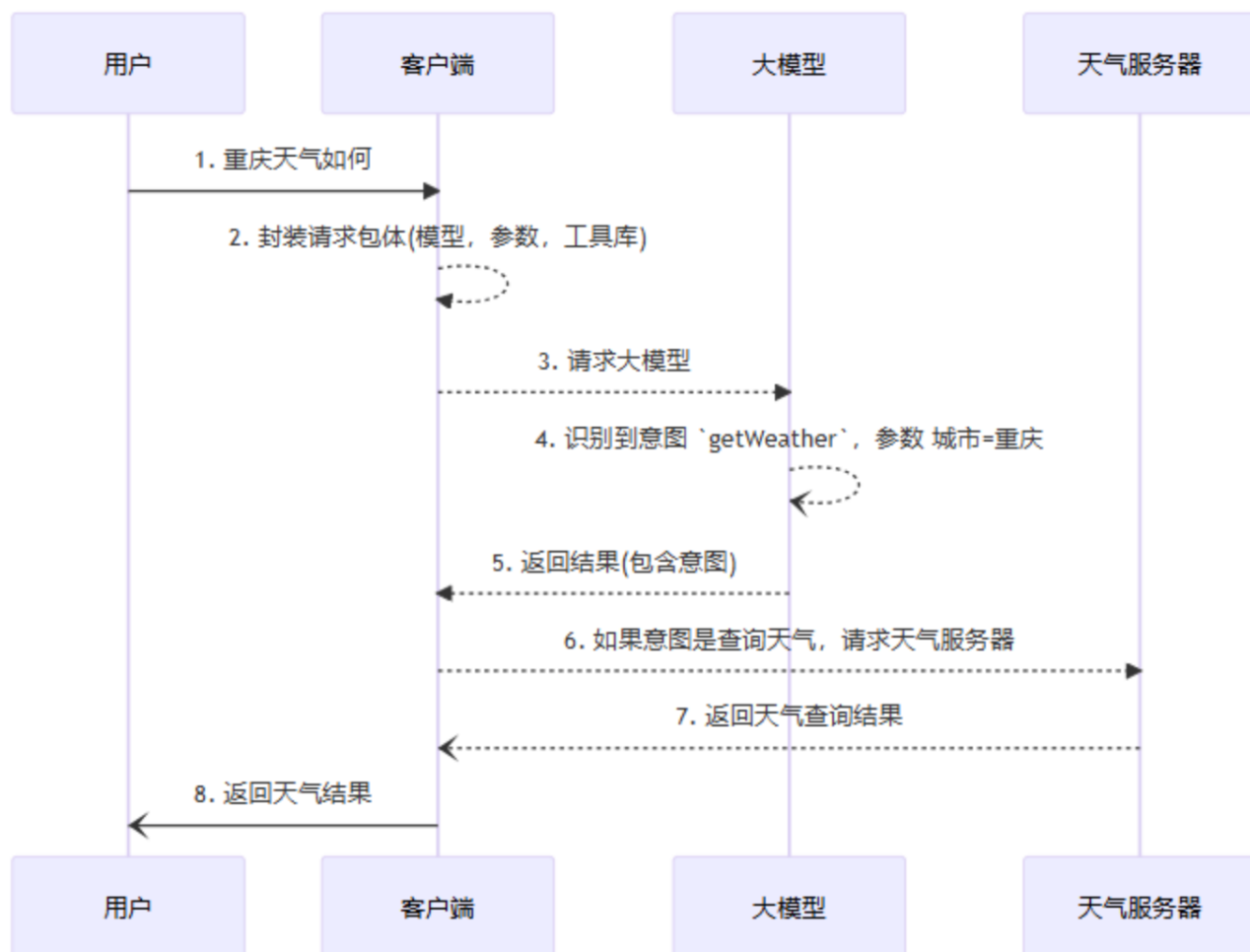
而大模型的 Function Calling 是 大模型通过识别意图自主决策调用哪个函数，比如

- 我们先告诉模型“有哪些可用函数、每个函数的功能和参数”即 工具列表
 - （例如有一个函数功能是：向MySQL中写入数据）
- 大模型会根据用户的问题，识别判断出“是否需要调用函数、调用哪个函数、传什么参数”
- 但大模型不会直接调用这些函数，而是生成一个函数调用指令
- 由后端程序解析函数指令，再去执行调用这个函数，拿到结果 框架封装好了会帮我们解析和调用
- 将函数调用结果传给大模型，大模型生成最终回答

2. 帮助个人理解（也应对面试官的追问）

2.1. 举例

咱们通过一个天气查询的例子来详细说明。假设用户询问“重庆天气如何”，具体流程是这样的：



生成的函数调用指令可理解为如下图：



2.2. Function Calling 的弊端

- **依赖精准的函数定义与提示词设计：**使用 Function Calling 我们需要提前写好工具函数，大模型的决策完全依赖提前告知的“函数列表、功能描述、参数格式”，若定义模糊（如未明确参数必填项）或提示词不清晰，可能会导致模型“误调用、漏调用”。

- **性能损耗与响应延迟：**相比纯文本生成，Function Calling 多了“模型决策→函数执行”，若函数执行耗时（如复杂查询、外部接口调用），会导致用户等待时间显著增加。
- **模型决策能力有限，复杂场景难覆盖：**面对多意图、模糊需求时，模型可能判断失误：
 - 多函数联动：用户问“查 XX 奶茶店是否营业，若营业帮我预约明天 3 点”，模型需先调用“查询营业状态”函数，再根据结果决定是否调用“预约”函数，复杂逻辑下易出错；
 - 隐含需求：用户说“想喝 XX 奶茶，明天有空”，模型可能无法识别出“需要先查营业状态 + 预约”的隐含逻辑，需用户明确指令。
- **安全性风险：**若未对函数调用权限、参数做校验，可能出现风险：
 - 越权调用：模型误调用“删除数据”类高危函数；
 - 参数注入：用户输入恶意参数（如 SQL 注入字符），模型直接传入函数执行，导致安全漏洞。

MCP

了解 MCP 吗

1. 面试时简洁地回答

在没有 MCP 之前，使用 Function Calling 时，我们需要定义工具，并且把工具列表给到大模型，这样大模型才知道可用工具列表、参数、用途等等，但这一套工具接入大模型的规范，对于每个大模型都不一样

- 如果用的是 chatgpt，那么需要按照 Openai 的规范接入工具，写一套接入代码
- 如果用的是 claude，那么得按照 Anthropic 的接入规范再写一套代码
- 如果用的是 gemini，那么得按照 Google 的接入规范再写一套代码

同一个工具，要写多遍，因为每个大模型的接入标准都不一样

所以才有人想能不能整一个统一的标准，让所有的大模型都遵循这个标准

这样，工具的开发者只需要写一次代码，就可以在所有的大模型上使用了

因此 MCP 诞生了

MCP (Model Context Protocol, 模型上下文协议) 是一套 **标准化的通信协议 / 接入规范**

可以理解为一套统一的工具接入标准，有了MCP之后，工具的开发者只需要按照MCP的规范开发一次工具，

这个工具就可以被所有大模型使用，这就是MCP的由来，以及MCP解决的核心问题

2. 帮助个人理解（也应对面试官的追问）

MCP 实现“统一标准”的核心逻辑

MCP 靠“**三层组件 + 统一协议规范**”实现跨模型标准化，核心是屏蔽不同大模型的接口差异

具体逻辑如下：

1. **统一工具描述规范**：MCP 定义了**标准化的工具注册格式**，要求所有工具必须包含「工具名称、功能

描述、参数类型与约束、返回值格式、错误码」等固定字段。不管是知识库检索工具，还是数据库查询工具，都要按这套格式定义，大模型和工具之间的“沟通语言”被统一。

2. **统一交互流程规范**：MCP 规定了「请求 – 调用 – 响应」的端到端标准化流程，和具体大模型无关：

- MCP Client（嵌入大模型宿主）接收用户请求后，按 MCP 格式打包“问题 + 工具列表”发给大模型；
- 大模型按 MCP 规范生成工具调用指令（格式全一致，不再区分 OpenAI/Claude）；
- MCP Client 解析指令，调用 MCP Server 上的工具；
- 工具执行结果按 MCP 标准格式返回给大模型，生成最终回答。

3. **三层组件解耦，屏蔽模型差异**

- **MCP Server**：负责按 MCP 规范封装工具，对外提供标准化接口，和底层大模型完全解耦；
- **MCP Client**：作为中间层，一头对接 MCP Host（大模型应用），一头对接 MCP Server，**核心作用是协议转换**——把大模型的指令转成 MCP 标准格式，也把工具结果转成大模型能识别的格式；
- **MCP Host**：运行大模型的应用（如 ChatGPT 客户端、自定义 AI 助手），只需内置 MCP Client，就能无缝调用所有 MCP Server 上的工具，不用关心工具的具体实现和模型差异。

Skill

参考学习视频：[Agent Skill 从使用到原理，一次讲清](#)



Agent Skill 从使用到原理全面解析

1. Agent Skill 的本质是什么

- Agent Skill 是大模型可调用的说明文档
- 每个 Skill 定义了特定任务的执行规范
 - 例如会议总结需包含参会人员、议题、决定三项内容
 - 用户无需重复输入指令，模型通过查询 Skill 自动遵循规则
- Skill 超越传统提示词，形成结构化能力封装
 - 不仅描述“做什么”，还规定“怎么做”
 - 支持示例引导和条件判断逻辑
 - 实现行为一致性与输出标准化

2. 如何创建和使用 Agent Skill

- 创建 Skill 需在指定目录下建立文件夹与文件
 - 路径为用户目录下的 `.cloud-skill`
 - 文件夹名称即 Skill 名称，必须与内部元数据一致
 - 核心文件为 `skill.md`，包含元素和指令两部分
- 元数据 (Metadata) 定义 Skill 基本信息
 - 包含 `name` 和 `description` 两个关键字段
 - `name` 必须与文件夹名完全相同
 - `description` 向模型说明该 Skill 的用途
- 指令 (Instruction) 部分定义具体行为规范
 - 明确模型在执行任务时应遵守的格式与逻辑
 - 可加入输入输出示例帮助模型理解
 - 如规定会议总结必须按固定结构输出

3. Agent Skill 的基础工作流程

- 用户发起请求后系统启动三阶段处理机制
 - **第一阶段：Cloud Code 将所有 Skill 的元数据发送给模型**
 - 模型仅看到各 Skill 的名称和描述，相当于浏览轻量级目录
 - 判断当前请求是否匹配某个 Skill
 - **第二阶段：模型识别匹配 Skill 并请求调用**
 - Cloud Code 接收到响应后加载对应 Skill 的完整内容
 - 读取 `skill.md` 中的全部指令
 - 将用户请求与完整指令一起发回给模型处理
 - **第三阶段：模型依据详细指令生成结果**
 - 结果经由 Cloud Code 返回给用户
 - 整个过程实现自动化决策与执行
 - 用户获得符合预设规范的输出

4. 渐进式披露机制的核心设计

- 第一层：元数据层 —— 始终可见的目录索引
 - 所有 Skill 的 `name` 和 `description` 永久加载
 - 占用极小 token 开销，便于快速路由
 - 模型据此判断请求归属哪个 Skill
 - 第二层：指令层 —— 按需加载的行为规范
 - 仅当 Skill 被选中时才加载其完整 `skill.md` 内容
 - 避免无关指令污染上下文
 - 显著节省 token 使用量
 - 第三层：资源层 —— 条件触发的深层依赖
 - 包含 `reference`（引用文件）和 `script`（脚本）
 - 只有满足特定条件时才会加载或执行
 - 实现“按需中的按需”加载策略
-

5. 高级功能之 Reference：条件性知识注入

- Reference 用于引入外部参考文档
 - 如财务手册、法务条款等专业资料
 - 文件独立存放，不直接写入 `skill.md`
 - 仅在相关场景下被动动态加载
- 触发机制基于指令中的条件判断
 - 例如“当提及预算时读取集团财务手册”
 - Cloud Code 在解析指令后判断是否需要加载
 - 若无相关内容则文件不会进入上下文
- Reference 加载会消耗 token
 - 因为其内容会被送入模型上下文供推理使用
 - 适合存储静态知识类信息
 - 实现精准的知识调用，避免资源浪费

6. 高级功能之 Script：自动化代码执行

- Script 支持运行本地脚本完成实际操作
 - 如上传文件、同步数据、调用 API 等
 - 在 `skill.md` 中声明触发条件与执行方法
 - 例如“当需要上传时运行 `upload.py`”
 - 脚本执行不占用模型上下文
 - Cloud Code 仅执行脚本并获取结果
 - 不读取代码内容，几乎零 token 消耗
 - 即使万行代码也不会影响推理效率
 - 安全性依赖于清晰的执行说明
 - 若未明确运行方法，模型可能尝试阅读代码
 - 导致意外加载脚本内容至上下文
 - 建议在指令中详尽描述调用方式
-

7. Reference 与 Script 的加载差异

- Reference 是“读取型”资源
 - 内容被加载进上下文参与推理
 - 占用 token，适用于知识增强
 - 如合规审查、政策比对等场景
- Script 是“执行型”资源
 - 仅运行脚本并返回结果，不暴露代码
 - 几乎不消耗 token，适合自动化操作
 - 如文件传输、状态更新等动作类任务
- 两者共同构成资源层的双轨机制
 - 实现知识与行动的能力分离
 - 提升系统效率与安全性
 - 构建更复杂的智能代理行为链

8. Agent Skill 与 MCP 的核心区别

- **MCP 主要功能是连接外部数据源**
 - 提供实时数据查询能力，如订单状态、销售记录
 - 相当于为模型接入数据库接口
 - 解决“获取什么数据”的问题
 - **Agent Skill 专注于数据处理规范**
 - 教会模型如何使用数据完成特定任务
 - 如规定报告格式、审批流程等
 - 解决“怎么处理数据”的问题
 - **官方定位：MCP 连接数据，Skill 教会使用数据**
 - 二者互补而非替代关系
 - MCP 提供原材料，Skill 定义加工方式
 - 共同构建完整的 AI 工作流
-

9. 如何选择 Agent Skill 与 MCP

- **轻量级任务优先使用 Agent Skill**
 - 如格式化输出、简单脚本执行、本地自动化
 - 开发成本低，部署简便
 - 适合个人开发者与小型项目
- **数据密集型或高安全要求场景选用 MCP**
 - 如企业级 API 对接、敏感数据访问控制
 - 更强的安全隔离与权限管理
 - 适合团队协作与生产环境
- **多数情况下建议结合使用**
 - MCP 获取数据 → Agent Skill 处理数据 → 输出结果
 - 发挥各自优势，构建完整闭环
 - 实现真正意义上的智能代理

10. Agent Skill 的演进与行业影响

- 最初为提升 Cloud 特定任务表现而设计
 - 官方初期定位较为克制
 - 仅作为内部优化工具
 - 未预料到其通用潜力
- 行业迅速采纳并扩展支持
 - VS Code、Codex、Cursor 等工具相继集成
 - 形成跨平台复用趋势
 - 推动标准化进程
- 2025年12月正式发布为开放标准
 - 支持跨产品、跨平台复用
 - 成为 AI Agent 领域通用设计模式
 - 标志着能力封装范式的成熟
- 展望未来：向模块化智能生态发展
 - Skill 商店、共享市场可能出现
 - 开发者可发布、订阅各类 Skill
 - 加速 AI 应用落地与普及

RAG

1. RAG 是什么

RAG = Retrieval-Augmented Generation 、中文：检索增强生成

一句话理解：

让大模型在回答前，先去查一份“外部知识库”，查到的内容 会整合到 Prompt 提示词中再输入给大模型

详细解释：

RAG 是先将知识作为向量存储到向量数据库

再将用户输入的提示词 去和 向量数据库中的 知识向量进行相似度匹配，检索出相关的片段

最后将检索出来的内容 + 用户提示词 一起发送给大模型，生成最终回答

2. 为什么会有 RAG?

RAG的目的是通过从外部知识库检索相关信息来辅助大语言模型生成更准确、更丰富的内容

大模型本身有 3 个问题：

- 知识过时（只学到训练截止日期之前的内容）
- 存在幻觉（会胡编乱造）
- 不知道内部私有数据（你的文档、业务规则等）

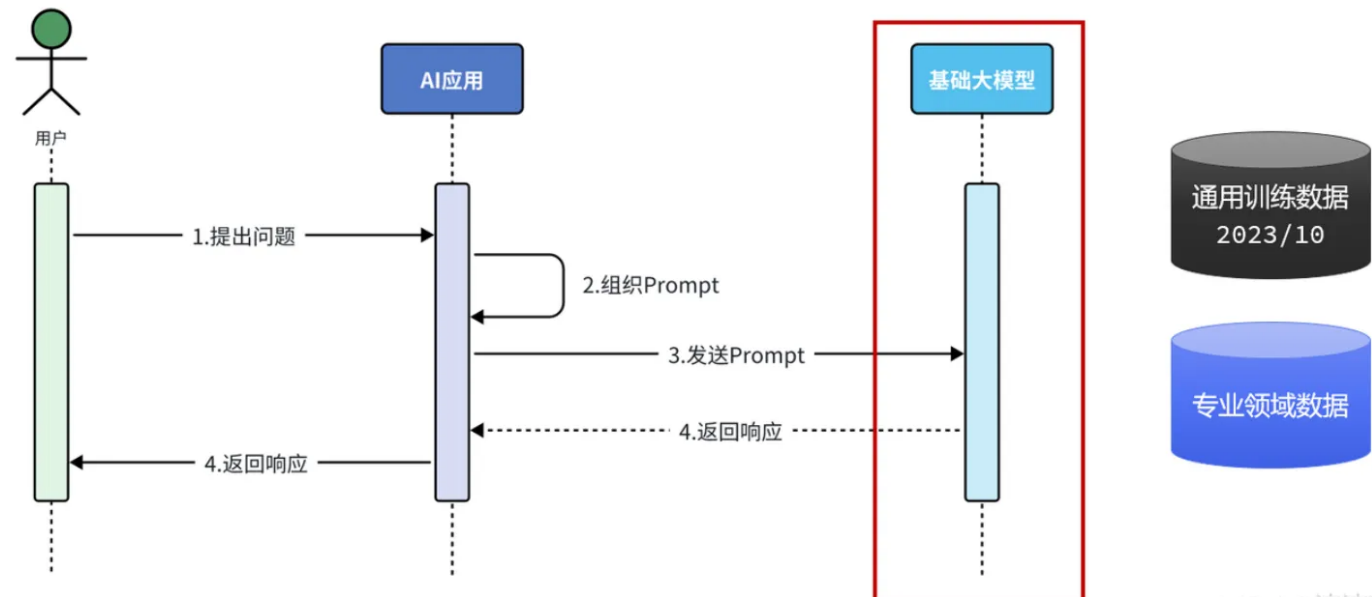
RAG 就是用来解决这三件事的：

- 答案更准，通过检索外部知识库，融入最新、最准确信息，提高回答的可靠性。
- 减少模型幻觉，基于真实检索结果生成内容，避免输出无根据或错误信息。
- 能使用内部的数据

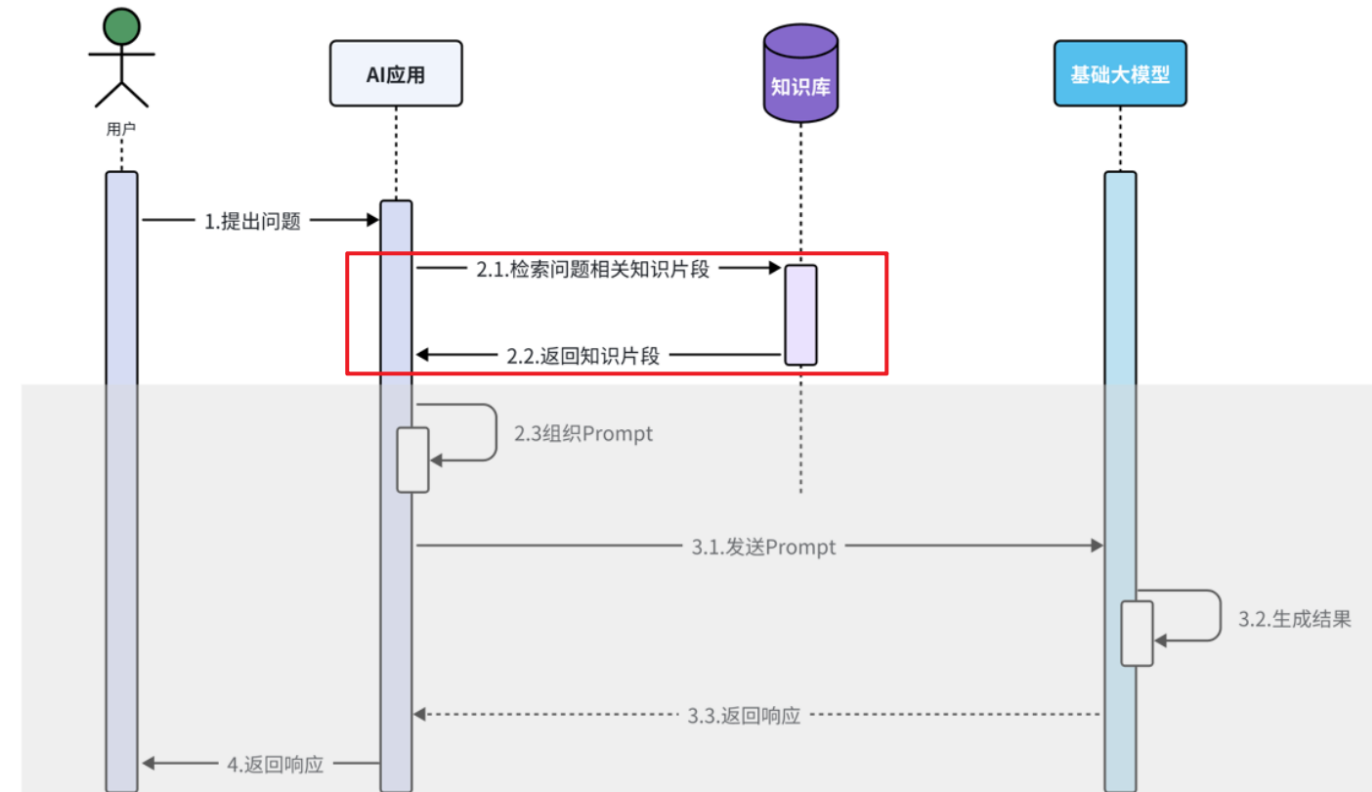
3. RAG 的流程

初步理解 有RAG前后的区别

无RAG知识库时：



有RAG知识库时：



RAG 中文名叫 检索增强生成、其中包含了三个词，分别是检索、增强、生成，下面分别解释一下

3.1. 检索

检索：指的是从向量知识库中检索与问题相关的信息。为后续的生成过程提供有用的上下文信息和知识支撑。

3.1.1. 向量知识库？

数据准备与知识库构建：

1. **收集数据：**首先，需要收集与问答系统相关的各种数据，这些数据可以来自文档、网页、数据库等多种来源，比如：公司内部文档（PDF、Word、Markdown）、FAQ列表、产品手册等。
2. **数据清洗：**对收集到的数据进行清洗，去除噪声、重复项和无关信息，确保数据的质量和准确性。
3. **知识库构建：**将清洗后的数据构建成知识库。通常将文档按照一定策略切块，如固定大小，语义分块等，使用**Embedding 模型**将这些片段（chunks）转换成向量，并将这些向量存储在向量数据库（如FAISS、Milvus等）中。

3.1.2. 从知识库检索？

检索包括：

1. **问题向量化：**当用户输入查询问题时，使用 **Embedding 模型** 将问题转换成**问题向量**
2. **相似度检索：**在向量数据库中检索与 **问题向量** 最相似的知识库片段（chunks）。这通常通过计算向量之间的相似度（如余弦相似度）来实现。
3. **结果排序：**根据相似度得分对检索到的结果进行排序，选择最相关的片段作为后续生成的输入。

3.2. 增强

增强：RAG 中增强是将从外挂知识库中检索到的信息用作 大模型的上下文输入(即Prompt提示词)，以增强模型对特定问题的理解和回答能力。这一步的目的是将外部知识融入生成过程中，使生成的文本内容更加丰富、准确和符合用户需求。**通过增强步骤，LLM模型能够充分利用外部知识库中的信息。**

增强 可以理解为 将检索到的相关片段与原始问题合并，形成更丰富的上下文信息

3.3. 生成

生成：即通过前文中提到的提示词进行生成回答

3.4. 小结

RAG 的“检索、增强、生成”，从哪里检索？增强了什么？谁生成了答案？

- 是从外部知识库中检索到的相关的知识
- 增强了 提示词（prompt），将检索到的相关片段与原始问题合并，形成更丰富的上下文信息
- 大模型 拿着增强后的 Prompt 生成了问题答案。

RAG 的优势：

- 提升回答质量：通过检索外部知识库，融入最新、最准确信息，提高回答的可靠性。
- 减少模型幻觉：基于真实检索结果生成内容，避免输出无根据或错误信息。

RAG 的缺点：

- 依赖知识库质量及系统设计：系统回答的效果与外部知识库质量和整体系统设计有较大关系，若设计不合理可能导致答案偏差。
- 系统复杂度增加，增加计算开销：大规模的知识库检索会让整体系统更加复杂，并且会延长系统性响应时间，对耗时敏感的业务可能产生性能瓶颈。研究表明，检索环节占RAG总耗时的60%以上。

4. 召回率

在 RAG 检索系统中，召回率指的是在所有真正相关的文档中，有多少被成功地检索了出来。

计算方式：召回率 = 检索到的相关内容数量 ÷ 所有相关内容的总数量

举个例子，假设你有一个技术文档知识库，某个用户提问：“如何在 Kubernetes 上部署？”

假设知识库中有 6 条与这个问题高度相关的内容，但系统只返回了其中 3 条。那么，召回率就是 $3 \div 6 = 50\%$ 。

在 RAG 系统中，召回率尤为关键，因为大模型能不能“答对”，很大程度取决于有没有拿到相关内容；召回率越高，大模型 在生成答案时能参考的有效信息就越多，回答的质量和准确性也就越有保障。

RAG 工程落地计算流程

1. 构造**测试数据集**（Question + 人工标注相关 chunk 列表）
2. 每条问题走检索流程，得到召回 chunk 列表
3. 逐题统计召回率
4. 计算平均召回率

但想要统计出有意义的召回率

测试数据集一般项目至少需要200–500条测试用例

专业项目至少需要上千条测试用例

因此统计召回率并没有那么容易，需要大量的测试用例数据

面试&校招/实习&简历经验

详见：<https://www.yuque.com/kdoxioc/gaa1qm>

PDF 下载

 [Diet-Agent \[1\].pdf](#)